CS6140 MACHINE LEARNING

Midterm Open-Notes Reference

Lookup-first synthesis of all 10 lecture decks (W2, W4–W12)

Nguyen Ha · Northeastern University, Silicon Valley

Exam: Thursday, November 12, 2026 · 3:00 PM · Room 912 · pen and paper · no calculator

How to use this in the exam (goal: any concept in about 20 seconds)

1. **Know the term?** Go to the **Concept Index** on page 5 → page number.
2. **Know the week, not the term?** Use the **Contents** (next page). Each week has the same four blocks: *Core definitions → Formulas → Mechanisms (why/how) → Comparisons.*
3. **Question shows a figure?** Use the **Figure Gallery** on page 67.
4. **Looks like an in-class quiz question?** Use the **Quiz types and model solutions** on page 78.
5. **Need just one formula?** Use the **Master Formula Sheet** on page 87.
6. **Still don't get a topic?** Use **Learn More** (IBM first, then DeepLearning.AI / Andrew Ng) on page 92.

Source tags such as [**W4 p45**] mean *deck W4, slide 45.* Boxes marked **Not in deck — standard definition** contain textbook material that the decks do not state. When an answer must match the professor, quote the deck wording first.

Contents

Concept Index 5

Orientation 8
- Exam facts and strategy 8
- Cross-reference to Géron, Hands-On ML, 3rd ed. 8
- Math survival kit: how to read the symbols 9
- Known deck errors and inconsistencies 11
- Notation conventions used here 11

W2 — Math Review: Probability, Statistics, Linear Algebra 13
- Core definitions 13
- Formulas 15
- Mechanisms: why and how 18
- Comparisons 18

W4 — Regression (Linear, Polynomial, Regularized, Logistic) 20
- Core definitions 20
- Formulas 22
- Mechanisms: why and how 24
- Comparisons 25

W5 — Model Performance Evaluation 26
- Core definitions 26
- Formulas 27
- Mechanisms: why and how 29
- Comparisons 29

W6 — Supervised Learning: kNN, Naive Bayes, SVM 31
- Core definitions 31
- Formulas 32
- Mechanisms: why and how 34
- Comparisons 35

W7 — Unsupervised Learning: Clustering, DBSCAN, Factor Analysis, PCA 36
- Core definitions 36
- Formulas 38
- Mechanisms: why and how 39
- Comparisons 40

W8 — Decision Trees and Random Forests 41
- Core definitions 41
- Formulas 42
- Mechanisms: why and how 43
- Comparisons 43

W9 — Ensemble Learning: Boosting, Bagging, Stacking 45
- Core definitions 45
- Formulas 46
- Mechanisms: why and how 46
- Comparisons 47

W10 — Recommender Systems 48
- Core definitions 48
- Formulas 50
- Mechanisms: why and how 53

Comparisons 53

W11 — Deep Learning 55
Core definitions 55
Formulas 57
Mechanisms: why and how 59
Comparisons 59

W12 — Intro to NLP (incl. LDA = Latent Dirichlet Allocation) 61
Core definitions 61
Formulas 64
Mechanisms: why and how 65
Comparisons 65

Figure Gallery: read any plot in 20 seconds 67
Hinge loss (flat part vs. rising part) 67
Scree plot (elbow / leveling-off point) 68
Which fits better: k = 1, 3, or 6? (W4 p37 layout) 68
Training vs. test error and the bias–variance tradeoff 69
L1 (V) vs. L2 (parabola) penalty 70
ROC curve and AUC 71
Precision–recall vs. threshold 71
Loss curves during training (underfit / overfit / optimal) 72
Sigmoid (logistic) curve 72
Activation functions and their derivatives 73
Why gradients vanish 73
Gradient descent and the learning rate 74
K-Means: objective per iteration and elbow over k 74
Dendrogram cut 75
Impurity: entropy vs. Gini 75
SVM margin and support vectors 76
Kernel trick 76
LDA: topics over words, documents over topics 77
Cosine similarity vs. dot product 77

Quiz Types Seen in Class and Model Solutions 78
Q1. Why is overfitting stated with n+1 points instead of n? 78
Q2. From the three curves L / M / R (W4): which k fits better, 1, 3, or 6? 78
Q3. Does K-Means converge? Why? 79
Q4. PCA: what are the principal components / principal vectors, and how do they relate to eigenvectors and eigenvalues? 79
Q5. What is the role of the kernel trick in SVM? 80
Q6. What does “naive” mean in naive Bayes? 80
Q7. How do you prevent zero likelihood in a classification model? 80
Q8. What is a scree plot for, and when do you use it? 81
Q9. LDA in NLP (Latent Dirichlet Allocation) 81
Q10. Figure: hinge loss. What are the flat part and the rising part? 81
Q11. Figure: L1 vs. L2 penalty shapes 82
Likely next variants (same styles, not yet asked) 82

Worked Numeric Examples (by hand, no calculator) 83
Confusion matrix → metrics 83
One K-Means iteration (assign + update) 83
Entropy and information gain for a split (deck data) 84
A short Bayes probability (deck numbers) 85
One perceptron update and one gradient-descent step 85
AdaBoost re-weighting (deck numbers) 86

Master Formula Sheet (all weeks, one place) **87**

Learn More: IBM and DeepLearning.AI (outside the deck) **92**

The sources **92**

Topic map by week **93**

Concept Index

Alphabetical. "§" = section number in this document; "p." = page of this PDF. Deck slide numbers appear inside each section.

A

Accuracy — W5 — **28**
Activation derivatives (sigmoid, tanh, ReLU) — W11 — **58**
Activation functions — W11 — **55**
Activation functions figure — Figures — **73**
AdaBoost (adaptive boosting) — W9 — **45**
AdaBoost weight update (ε, $\sqrt{((1-\varepsilon)/\varepsilon)}$) — W9 — **46**
AdaBoost worked numbers — Numeric — **86**
Adam / AdaGrad / momentum — W11 — **56**
Adjusted R^2, AIC, BIC (model selection) — W5 — **26**
Agglomerative vs. divisive clustering — W7 — **37**
Alpha and beta in LDA — W12 — **62**
Andrew Ng / DeepLearning.AI courses — More — **92**
Arithmetic / harmonic / geometric mean — W2 — **15**
Assumptions of linear regression (OLS) — W4 — **20**
AUC (area under ROC) — W5 — **27**
Autoencoder — W11 — **57**

B

Backpropagation — W11 — **55**
Bag of words — W10 — **49**
Bagging (bootstrap aggregating) — W9 — **45**
Bagging vs. boosting vs. stacking table — W9 — **47**
Batch normalization — W11 — **56**
Bayes worked examples — Numeric — **85**
Bayes' theorem — W2 — **16**
Bernoulli distribution — W2 — **15**
BERT vs. GPT vs. BART — W12 — **62**
Bias (perceptron term) — W11 — **55**
Bias–variance figure — Figures — **69**
Bias–variance tradeoff — W5 — **26**
Binomial distribution — W2 — **15**
Bootstrapping — W5 — **27**
Border point (DBSCAN) — W7 — **37**

C

CART algorithm — W8 — **41**
CBOW (continuous bag of words) — W10 — **50**
Chain rule — W11 — **56**
Chain rule worked gradient — W11 — **58**
CNN (convolutional neural network) — W11 — **57**
Cold start problem — W10 — **49**
Collaborative filtering — W10 — **50**
Communality — W7 — **37**
Complement rule — W2 — **15**
Conditional probability — W2 — **13**
Confusion matrix — W5 — **27**
Confusion matrix worked example — Numeric — **83**
Content-based recommendation — W10 — **48**
Core point (DBSCAN) — W7 — **37**
Cosine similarity — W10 — **50**
Cross-validation (k-fold) — W5 — **27**

D

DBSCAN — W7 — **37**
Decision tree anatomy (root, leaf, depth) — W8 — **41**
Deck errors and inconsistencies — Intro — **11**
Dendrogram — W7 — **37**
Dendrogram cut figure — Figures — **75**
Dice coefficient — W10 — **51**
Discriminative vs. generative models — W11 — **57**
Distance-weighted kNN — W6 — **32**
Dot product (similarity) — W10 — **50**
Dropout — W11 — **56**

E

Eigenvalues / eigenvectors (linear algebra) — W2 — **15**
Eigenvalues / eigenvectors in PCA — W7 — **38**
Elastic Net — W4 — **21**
Elbow (knee) method — W7 — **36**
ELIZA — W12 — **61**
Embedding matrix — W10 — **49**
Entropy — W8 — **42**
Entropy / information gain worked example — Numeric — **84**
Epoch — W11 — **55**
Ethics in NLP (bias, privacy) — W12 — **64**
Euclidean distance — W6 — **32**
Exam facts and question styles — Intro — **8**
Expectation — W2 — **15**
Experiment vs. observational study — W2 — **13**
Exploding gradients — W11 — **56**

F

F1 score — W5 — **28**
Factor analysis — W7 — **37**
False positive / false negative — W5 — **27**
Feature extraction problem (recsys) — W10 — **49**

G

GAN (generative adversarial network) — W11 — **57**
Gaussian (normal) distribution — W2 — **15**
Géron book cross-reference — Intro — **8**
Gini index — W8 — **42**
GloVe — W10 — **50**
Gradient descent (W4) — W4 — **20**
Gradient descent figure (learning rate) — Figures — **74**
Gradient descent update rule (minus sign) — W4 — **22**
Gradient descent variations — W11 — **56**

H

Hierarchical clustering — W7 — **37**
Hinge loss — W6 — **32**

Hinge loss figure ..... Figures 67
Hinge loss quiz answer ..... Quiz 81
Homoscedasticity W4 20

I

IBM AI Engineering courses ..... More 92
IDF (inverse document frequency) W10 51
Independent events W2 14
Information gain W8 42

J

Jaccard similarity W10 51

K

k = 1, 3, 6 figure ..... Figures 68
k = 1, 3, or 6 (which fits better) ..... Quiz 78
K-Means W7 36
K-Means algorithm steps W7 36
K-Means convergence W7 36
K-Means convergence quiz answer ..... Quiz 79
K-Means objective J (WCSS) W7 38
K-Means one-iteration example ..... Numeric 83
K-Means strengths / weaknesses W7 36
K-modes / k-prototype W7 36
Kernel trick W6 32
Kernel trick quiz answer ..... Quiz 80
kNN (k-nearest neighbors) W6 31
kNN for collaborative filtering W10 50
Knowledge-based recommendation W10 48
Kurtosis W2 13

L

L1 / L2 / max norms W2 14
L1 vs. L2 figure ..... Figures 70
L1 vs. L2 quiz answer ..... Quiz 82
Language model W12 63
Laplace smoothing W6 32
Laplace smoothing quiz answer ..... Quiz 80
Lasso (L1) regression W4 21
Law of total probability W2 13
LDA (Latent Dirichlet Allocation) W12 61
LDA algorithm W12 62
LDA example (5 sentences) W12 62
LDA quiz answer ..... Quiz 81
Learning rate W4 20
Lemmatization W12 61
Linear algebra rules W2 15
Linearity of expectation W2 15
Linkage (single, complete, centroid) W7 37
Log loss (cross-entropy) W5 26
Logistic regression W4 21
Logit / log-odds / odds W4 23
Loss curves (under/overfit) W11 56
Loss curves figure ..... Figures 72
LSTM W12 64

M

MAE / MSE / RMSE W5 27
MAP vs. MLE W4 22
Margin (SVM) W6 32
Markov assumption W12 63
Math symbols explained ..... Intro 9
Matrix factorization W10 49
Matrix inverse / invertibility W2 14
Mini-batch K-Means W7 36
Mini-batch vs. stochastic GD W11 56
MLP (multilayer perceptron) W11 55
Moments (mean, variance, skewness, kurtosis) W2 13
Momentum W11 56

N

N-gram model W12 63
n+1 points (overfitting) quiz answer ..... Quiz 78
Naive (meaning of) W6 31
Naive Bayes W6 31
Naive Bayes for collaborative filtering W10 50
Naive Bayes formula W6 33
Naive Bayes quiz answer ..... Quiz 80
Neuron (artificial) W11 55
Non-linear activation (why needed) W11 55
Normal equation W4 22
Notation conventions ..... Intro 11

O

Out-of-bag (OOB) W9 45
Outside reading: IBM / DeepLearning.AI (Andrew Ng) ..... More 92
Overfitting W4 20

P

p-value W2 14
PCA W7 38
PCA quiz answer ..... Quiz 79
Pearson correlation (CF) W10 51
Perceptron (single-layer) W11 55
Perceptron learning rule W11 58
Perceptron worked update ..... Numeric 85
Polynomial regression W4 20
Precision / recall W5 28
Precision–recall tradeoff figure ..... Figures 71
Pseudoinverse / SVD W2 13

R

Random forest W8 42
Random variable W2 13
Recommender system types W10 48
Regression tree vs. linear regression W8 41
Regular expressions W12 61
Regularization W4 21
Regularized loss formulas W4 23
ReLU / leaky ReLU W11 58
Ridge (L2) regression W4 21
RNN W12 63
ROC curve W5 27
ROC figure ..... Figures 71

S

Scree plot W7 **38**
Scree plot figure Figures **68**
Scree plot quiz answer Quiz **81**
Serendipity problem W10 **49**
Sigmoid W4 **23**
Sigmoid figure Figures **72**
Skewness W2 **13**
Skip-gram W10 **50**
Stacking W9 **46**
Stemming W12 **61**
Stopping criteria (trees) W8 **41**
Support vectors W6 **32**
SVM W6 **32**
SVM formulas (margin, hinge) W6 **34**

T

TF-IDF W10 **49**
Tokenization W12 **61**
Train / validation / test split W5 **26**
Training cycle W11 **56**
Transformer / CNN / RNN pros & cons W11 **57**
Transformers W12 **64**

U

Underfitting W4 **21**
Unit step function W11 **57**

V

Vanishing gradient figure Figures **73**
Vanishing gradients W11 **56**

W

Why not linear regression for classification W4 **21**
Word embeddings W12 **62**
Word2Vec W10 **50**

Z

z-score W2 **15**

Orientation

Exam facts and strategy

Item	Detail
When / where	Thursday Nov 12, 2026, regular class time 3:00–6:20 PM (exam designed for about 1.5 hours), Room 912, in person
Weight	25% of the course grade
Format	Handwritten, pen and paper. Conceptual and short-answer questions; light hand-computable numbers only; **no calculator, no code**
Allowed	Any paper materials. **No** phones, laptops, or AI
Scope	Module 0–9: Intro, Math Review (W2), Regression (W4), Model Evaluation (W5), Supervised Learning (W6), Unsupervised Learning (W7), Decision Trees (W8), Ensemble Learning (W9), Recommender Systems (W10), Deep Learning (W11), Intro to NLP (W12)

The professor's five question styles (from his in-class surprise quizzes, most frequent first):

1. **One-line "what / why" with a twist.** He tests the *mechanism*, not the memorized definition.
2. **Figure reading.** Name each part of a curve and say what it means (hinge loss, scree plot, error curves, L1/L2 shapes).
3. **Compare and judge.** Pick the better option and justify it (k = 1 vs 3 vs 6, Lasso vs Ridge, bias vs variance).
4. **Mechanism "how".** How Laplace smoothing works; why K-Means converges.
5. **Light math.** Short calculations by hand: confusion-matrix metrics, one K-Means step, entropy/Gini, a Bayes probability, one gradient or perceptron step.

> **EXAM ANSWER**
>
> **Answer template that scores well:** (1) state the term in the deck's words → (2) one sentence of mechanism ("because . . .") → (3) a consequence or example. Three short sentences usually beat one long paragraph.

Cross-reference to Géron, *Hands-On ML*, 3rd ed.

Use the book as a deeper fallback. Always answer in the professor's notation first.

Topic in this document	Géron 3rd ed.	Coverage in book
Classification metrics, confusion matrix, precision/recall, ROC (W5)	Ch 3	Strong
Cross-validation, train/validation/test (W5)	Ch 2 (and Ch 3)	Good

Topic in this document	Géron 3rd ed.	Coverage in book
Linear regression, normal equation, gradient descent, polynomial regression, learning curves, Ridge/Lasso/Elastic Net, logistic regression (W4)	Ch 4	Strong
SVM, margins, kernel trick, hinge loss (W6)	Ch 5	Strong
Decision trees, CART, Gini, entropy (W8)	Ch 6	Strong
Ensembles: bagging, OOB, random forests, AdaBoost, stacking (W8–W9)	Ch 7	Strong
PCA, explained variance, choosing dimensions (W7)	Ch 8	Strong
K-Means, elbow, DBSCAN, hierarchical clustering (W7)	Ch 9	Good (hierarchical is brief)
Neural networks, perceptron, MLP, backpropagation, activations (W11)	Ch 10	Strong
Vanishing/exploding gradients, optimizers (momentum, AdaGrad, Adam), dropout, batch normalization (W11)	Ch 11	Strong
CNN (W11)	Ch 14 (check book TOC)	Good
RNN / LSTM (W11, W12)	Ch 15 (check book TOC)	Good
NLP, attention, Transformers, BERT/GPT (W12)	Ch 16	Good
Autoencoders, GANs (W11)	Ch 17 (check book TOC)	Good
Naive Bayes, Laplace smoothing (W6)	Not covered	**Self-contained here, p. 31**
Recommender systems, TF-IDF, similarity measures, collaborative filtering (W10)	Not covered	**Self-contained here, p. 48**
NLP basics: tokenization, stemming/lemmatization, regex, LDA, n-grams (W12)	Thin	**Self-contained here, p. 61**
Probability, statistics, linear algebra (W2)	Not in book	**Self-contained here, p. 13**
Factor analysis, k-modes, DBSCAN details (W7)	Partial	Use p. **36**

Math survival kit: how to read the symbols

*If a formula looks scary, read it aloud using this table. Every formula in this document also has an **In plain words** line.*

Symbol	Read it as	Tiny example
$\sum_{i=1}^{n} x_i$	"add up x_1 through x_n"	$\sum_{i=1}^{3} i = 1+2+3 = 6$
$\prod_{i=1}^{n} x_i$	"multiply x_1 through x_n"	$\prod_{i=1}^{3} i = 1 \cdot 2 \cdot 3 = 6$
$\bar{x}$	"x-bar": the average of the x values	$\bar{x}$ of 2, 4, 6 is 4
$\hat{y}$	"y-hat": the model's **prediction** of y	true $y = 5$, $\hat{y} = 4.5$
$\lvert a \rvert$	absolute value: drop the minus sign	$\lvert -3 \rvert = 3$
$\lVert w \rVert_2$	length of vector w: $\sqrt{w_1^2 + w_2^2 + \dots}$	$\lVert (3,4) \rVert_2 = 5$
$\lVert w \rVert_1$	sum of absolute values	$\lVert (3,-4) \rVert_1 = 7$
$w^T x$	dot product: multiply matching entries, then add	$(1,2) \cdot (3,4) = 3 + 8 = 11$
$P(A \mid B)$	"probability of A **given** B happened"	$P(\text{rain} \mid \text{clouds})$
$A \cap B$, $A \cup B$	"A **and** B", "A **or** B"	
$\arg\min_w f(w)$	"the w that makes f smallest" (the location, not the value)	$\arg\min_x (x-2)^2 = 2$
$\frac{\partial L}{\partial w}$	"slope of L when you nudge only w"	$L = w^2 \Rightarrow \frac{\partial L}{\partial w} = 2w$
∇f	gradient: the vector of all those slopes; points **uphill**	
e^x, $\exp(x)$	$e \approx 2.718$ to the power x; always > 0	$e^0 = 1$
log, ln	inverse of exp; $\log(1) = 0$; log of a number in $(0,1)$ is negative	$\log_2 8 = 3$
$\propto$	"is proportional to" (equal up to a constant)	
$\in$, $\forall$	"belongs to", "for all"	$x \in \mathbb{R}^d$: x is a list of d real numbers
$\leftarrow$	"replace the old value with" (an update)	$w \leftarrow w - 0.1$
$\mathbf{1}(\cdot)$	indicator: 1 if the condition is true, else 0	$\mathbf{1}(3 > 2) = 1$

NOT IN DECK — STANDARD DEFINITION

Hand-computation helpers (no calculator). $\log_2 1 = 0$, $\log_2 2 = 1$, $\log_2 4 = 2$, $\log_2 8 = 3$, $\log_2 3 \approx 1.585$, $\log_2 5 \approx 2.322$, $\log_2 6 \approx 2.585$, $\log_2 7 \approx 2.807$. Quotients: $\log_2(1/2) = -1$, $\log_2(1/4) = -2$, $\log_2(3/4) \approx -0.415$, $\log_2(1/3) \approx -1.585$, $\log_2(2/3) \approx -0.585$. Binary entropy $H(p)$: $H(1/2) = 1$, $H(1/4) = H(3/4) \approx 0.811$, $H(1/3) \approx 0.918$, $H(1/5) \approx 0.722$, $H(0) = H(1) = 0$. Also $e^0 = 1$, $e^1 \approx 2.718$, $e^{-1} \approx 0.368$, $\sigma(0) = 0.5$, $\sigma(2) \approx 0.88$, $\sigma(-2) \approx 0.12$, $\sqrt{2} \approx 1.414$, $\sqrt{3} \approx 1.732$, $\sqrt{5} \approx 2.236$.

Known deck errors and inconsistencies

DECK ERROR / INCONSISTENCY

1. Sign in the weight update, W11 p11 (backpropagation). The slide prints weight = weight + $r\,\frac{\partial L}{\partial w_i}$ with a **plus** sign. This is almost certainly a slide error. Moving *with* the gradient goes **uphill** and makes the loss larger. This document uses the **minus** sign everywhere:

$$w_i \leftarrow w_i - r\,\frac{\partial L}{\partial w_i}$$

The minus sign matches W4 p15 ($x_{i+1} = x_i - \eta \nabla f(x_i)$), W11 p18 ($w_i^* = w_i - \nu \frac{\partial L}{\partial w_i}$), and W11 p19 (momentum). If asked, write the minus form and add one line: *"deck W11 p11 prints a plus — likely a slide typo."*

DECK ERROR / INCONSISTENCY

2. The perceptron rule on W11 p7 is NOT a typo. The slide gives weight = weight + learning_rate × error × input with error = actual − predicted. The plus sign is correct *here*, because the error is defined as actual − predicted. Written in the standard "minus" form, it is the same rule: $w \leftarrow w - \eta(\hat{y} - y)x$. Both forms push the weights so that the prediction moves toward the true class.

DECK ERROR / INCONSISTENCY

3. Other small inconsistencies. (a) W4 p32 gives the target as $7.5\sin(2.5\pi x)$, while the axis label on W4 p37 says $7.5\sin(2.25\pi x)$. The conclusion is the same either way. **(b)** W10 p61 writes cosine similarity as "$\text{sim}(x, y) = \arccos(r_x, r_y) = \frac{r_x \cdot r_y}{\|r_x\|\|r_y\|}$". The fraction is the **cosine**; arccos would turn it into an *angle*. **(c)** W11 p20 labels the tanh derivative "$f(z) =$" instead of "$\partial f / \partial z =$". The value $\frac{4\exp(2z)}{(\exp(2z)+1)^2}$ is right. **(d)** W8 p16 says the Gini index ranges "from 0 to 1". For k classes the maximum is actually $1 - 1/k$ (0.5 for two classes). **(e)** W12 p34 "Misinformation and Manipulation" repeats the privacy text of p33. **(f)** W9 p6 averages 3, 10, 90, 15, 25 and prints $\frac{158}{5} = 31.6$. The correct sum is 143, so the average is $\frac{143}{5} = 28.6$. **(g)** In W9 p16, bootstrap sample 1 (9 9 2 6 9 8 2 1 7 2) actually lacks $\{3, 4, 5, 10\}$; the slide lists $\{3, 4, 7, 10\}$.

Notation conventions used here

Convention	Meaning	Source
α	regularization strength (Ridge/Lasso/Elastic Net)	W4 p41–p47
η, ν, r	learning rate (the decks use all three letters)	W4 p15, W11 p18, W11 p11
J	objective function (K-Means: J; linear regression GD: $J(w)$)	W7 p10, W4 p24
$E(w)$, SSE, RSS	sum of squared errors $\sum_i (\hat{y}_i - y_i)^2$	W4 p7, p41
log loss	cross-entropy for binary classification	W5 p18
z_{nk}	1 if point n is in cluster k, else 0	W7 p9

Convention	Meaning	Source
μ_k	centroid (mean) of cluster k	W7 p8
w_0 / b	intercept / bias	W4 p4, W11 p5

W2 — Math Review: Probability, Statistics, Linear Algebra

*Deck W2_Math_Review.pdf, 53 slides. Géron does **not** cover this, so this section is fully self-contained. **Outside reading (IBM / DeepLearning.AI): p. 93.***

Core definitions

Statistics. Techniques for analyzing data to gain insights. ML uses statistics to make data-driven decisions [W2 p2]. Four supporting pieces: *sampling* gathers data while reducing bias; *descriptive statistics* explore data with visuals and summaries; *inferential statistics* draw conclusions about a population from a sample, using probability.

Statistic vs. parameter [W2 p3]. A **statistic** describes a characteristic of a **sample**. A **parameter** describes a characteristic of a **population**.

Population vs. sample [W2 p4]. The *population* is all relevant individuals, items, or events. A *sample* is a subset, taken because of resource limits, made of n *observational units.*

Observational study vs. experiment [W2 p5]. An *observational study* records responses as they occur, with no direct influence (example: collecting home-feature data). An *experiment* first assigns treatments to the units and then records responses (example: an A/B test of webpage layouts). **Only an experiment with random treatment assignment supports causal conclusions.**

Feature; distribution; cluster; tails [W2 p6]. A *feature* is a measurable trait of an observational unit. Exploring its *distribution* shows which values are possible and how often they occur. A *cluster* is a distinct, frequent group inside a distribution. *Tails* are the ends of the distribution: the left tail holds low values, the right tail high values.

The four moments [W2 p7]

Moment	Name	What it measures
1st	**Mean** μ	Average: sum of values ÷ number of values. (The **median** ν, the middle value of the ordered data, is *not* a moment.)
2nd	**Variance** σ^2	Average squared distance of the values from the mean
3rd	**Skewness** γ	Amount and direction of departure from symmetry. **Positive = extended right tail; negative = extended left tail**
4th	**Kurtosis** κ	Tail heaviness. **Larger kurtosis = more extreme values**

Random variable [W2 p10–p11] . A rule that assigns a number to every outcome in the sample space (coin toss: H→1, T→0). It is written with a capital letter (X). A *discrete* random variable takes countably many values and is typically **counted** (a die roll gives $\{1,\ldots,6\}$). A *continuous* one can take

any value in a range and is typically **measured** (temperature).

Probability vocabulary [W2 p14–p15]. *Experiment*: a procedure with one of several possible outcomes. *Outcome*: the result. *Sample space* S: the set of all outcomes (a die gives $S = \{1, \ldots, 6\}$). *Event*: a subset of S (rolling an even number gives $\{2, 4, 6\}$). $P(A)$ is the sum of the probabilities of the outcomes in A, or, when all outcomes are equally likely, $\frac{\#\text{desired outcomes}}{\#\text{total outcomes}}$.

PMF and CDF [W2 p18]. A *probability mass function* gives $p(X = x)$, the probability that a discrete X equals exactly x. The *cumulative distribution function* is $F(x) = p(X \leq x)$. For a fair die, $F(3) = p(1) + p(2) + p(3) = \frac{3}{6} = \frac{1}{2}$.

Expectation [W2 p12]. The first moment of a distribution, and a useful summary of it. *Some distributions have no moments at all, for example the Cauchy distribution.*

Conditional probability [W2 p21]. The probability of one event given that another has occurred.

Law of total probability [W2 p22]. If S is split into mutually exclusive pieces S_1, S_2, S_3, then $P(B)$ is the sum over the pieces of $P(B \mid S_i)P(S_i)$.

Independent events [W2 p23–p24] . The probability of one event does not affect the other (flipping a nickel and a dime). **Independent ≠ mutually exclusive.** Mutually exclusive events are *dependent* by definition, because if one occurs the other cannot.

Bayes' theorem [W2 p25]. Lets you compute $P(A \mid B)$ from $P(B \mid A)$. The four parts are named **posterior** $P(A \mid B)$, **likelihood** $P(B \mid A)$, **prior** $P(A)$, and **marginal** $P(B)$.

Bernoulli trial and binomial distribution [W2 p26–p27]. A Bernoulli trial has two outcomes (1/0, T/F, H/T, −1/1), like one if-then-else. A sequence of n independent Bernoulli trials has 2^n possible 0/1 tuples. The number of successes in such a sequence follows the **binomial** distribution.

z-score [W2 p30]. A signed number of standard deviations from the mean. Positive means above the mean, negative below. A large $|z|$ means the value is far from the mean, so it is more unusual.

p-value [W2 p31] . *Technical:* the probability of the observed outcome, or something more extreme, computed **assuming the null hypothesis is true**. The lower the p-value, the more confident we are that the alternative hypothesis is true. *Non-technical:* it "quantifies surprise in a universe where there is no effect or no change."

Linear algebra objects [W2 p32–p35]. *Scalar*: a single number, in italics (a, n, x). *Vector*: a 1-D array, $x \in \mathbb{R}^n$. *Matrix*: a 2-D array, $A \in \mathbb{R}^{m \times n}$. *Tensor*: an array with any number of dimensions (0-D = scalar, 1-D = vector, 2-D = matrix, or more).

Systems of equations [W2 p39–p42] . $Ax = b$ can have **no solution, many solutions, or exactly one solution**. Exactly one solution means multiplication by A is invertible. A matrix **cannot be inverted** if it is **not square** (#rows ≠ #columns) or has **redundant rows/columns** ("linearly dependent", "low rank"). Using the inverse is *numerically unstable but useful for abstract analysis.*

Norms [W2 p43–p44] . Functions that measure how "large" a vector is, like its distance from zero. Three required properties: $f(x) = 0 \Rightarrow x = 0$; the **triangle inequality** $f(x + y) \leq f(x) + f(y)$; and $f(\alpha x) = |\alpha| f(x)$.

Special matrices [W2 p45]. *Unit vector*: $\|x\|_2 = 1$. *Symmetric*: $A = A^T$. *Orthogonal*: $A^T A = AA^T = I$, so $A^{-1} = A^T$ (inverting it is free).

Eigenvectors and eigenvalues [W2 p46–p47] . An *eigenvector* v of A is a direction that A only **stretches**, without rotating it. The stretch factor is the *eigenvalue* λ. The figure on W2 p47 shows the unit circle turning into an ellipse: each eigenvector $v^{(i)}$ is scaled by its λ_i.

SVD and pseudoinverse [W2 p48–p50]. SVD is like eigendecomposition but **more general: the matrix need not be square**. The **Moore–Penrose pseudoinverse** $A^\dagger$ solves $x = A^\dagger y$ in all three cases. With exactly one solution it equals the inverse. With no solution it gives the x with the **smallest error** $\|Ax - y\|_2$. With many solutions it gives the solution with the **smallest norm** $\|x\|$.

Formulas

Means [W2 p8]

$$\text{Arithmetic: } \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{Harmonic: } \bar{x} = n\Big(\sum_{i=1}^{n} \frac{1}{x_i}\Big)^{-1} \qquad \text{Geometric: } \bar{x} = \Big(\prod_{i=1}^{n} x_i\Big)^{1/n}$$

> **IN PLAIN WORDS**
>
> **In plain words.** *Arithmetic*: add, then divide by n ("good for sharing things equally"). *Harmonic*: average the reciprocals, then flip the result ("an average that considers different parts of a whole", for example rates). *Geometric*: multiply, then take the n-th root ("how things grow when they are multiplied", for example growth rates). Example with 1, 4: arithmetic $= 2.5$, geometric $= \sqrt{4} = 2$, harmonic $= 2/(1 + \frac{1}{4}) = 1.6$.

Variance and standard deviation [W2 p8, p20]

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2 \ \text{(population)} \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \ \text{(sample)} \qquad \sigma = \sqrt{\sigma^2} \qquad \text{precisio}$$

> **IN PLAIN WORDS**
>
> **In plain words.** Variance asks: "on average, how far (squared) is each value from the mean?" It is in **squared units**. The standard deviation takes the square root to get back to the **same units** as the data [W2 p9]. The sample version divides by $n - 1$ instead of n. *Precision* (in W2) means the reciprocal of the variance. That is a different meaning from precision in W5 metrics!

Skewness and kurtosis [W2 p8] (population versions; the sample versions add small-n correction factors)

$$\gamma = \sqrt{n} \times \frac{\sum_i (x_i - \bar{x})^3}{\left(\sum_i (x_i - \bar{x})^2\right)^{3/2}} \qquad \kappa = n \times \frac{\sum_i (x_i - \bar{x})^4}{\left(\sum_i (x_i - \bar{x})^2\right)^2}$$

Sample versions [W2 p8]: $\gamma_s = \frac{n\sqrt{n-1}}{n-2} \times \frac{\sum (x_i - \bar{x})^3}{(\sum (x_i - \bar{x})^2)^{3/2}}$, and $\kappa_s = \frac{n(n+1)(n-1)}{(n-2)(n-3)} \times \frac{\sum (x_i - \bar{x})^4}{(\sum (x_i - \bar{x})^2)^2}$.

> **IN PLAIN WORDS**
>
> **In plain words.** Cubing keeps the sign, so values far to the right push skewness **positive**. Raising to the 4th power makes big deviations dominate, so heavy tails push kurtosis **up**.

Expectation and variance of a random variable [W2 p12, p19–p20]

$$E[X] = \sum_i x_i\, p(x_i) \quad \text{(discrete)} \qquad E[X] = \int_{-\infty}^{\infty} x f(x)\, dx \quad \text{(continuous)}$$

$$V[X] = \sum_i (x_i - \mu)^2\, p(x_i) \qquad V[X] = \int_{-\infty}^{\infty} (x-\mu)^2 f(x)\, dx$$

IN PLAIN WORDS

In plain words. Expectation is a *probability-weighted average*: each value counts as much as its probability. A fair die gives $E[X] = \frac{1+2+\cdots+6}{6} = 3.5$. In the continuous case, $f(x)$ is the probability **density**.

Linearity of expectation [W2 p13]: $E[X+Y] = E[X] + E[Y]$ and $E[cX] = cE[X]$.

WORKED EXAMPLE / MODEL ANSWER

Deck example [W2 p13]: expected number of aces in a 5-card hand. Let $A_j = 1$ if card j is an ace. Then $E[A_1] = 1 \cdot \frac{1}{13} + 0 \cdot \frac{12}{13} = \frac{1}{13}$. Every card is the same, so $E[A] = E[A_1] + \cdots + E[A_5] = 5 \cdot \frac{1}{13} = \frac{5}{13}$. **Linearity works even though the cards are dependent.**

Union, complement, conditional, total probability, independence [W2 p16–p23]

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) \qquad P(\bar{E}) = 1 - P(E) \qquad P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

$$P(B) = P(B \mid S_1)P(S_1) + P(B \mid S_2)P(S_2) + P(B \mid S_3)P(S_3) \qquad \text{independent: } P(A \cap B) = P(A).$$

WORKED EXAMPLE / MODEL ANSWER

Deck examples. *Complement* [W2 p17]: two dice; E = at least one 6. Then $\bar{E}$ = no 6 on either die, with probability $\frac{5 \cdot 5}{36} = \frac{25}{36}$, so $P(E) = 1 - \frac{25}{36} = \frac{11}{36}$. *Union* [W2 p16]: $P(\geq 4 \text{ face cards}) = P(F_4) + P(F_5) - P(F_4 \cap F_5) = \frac{\binom{12}{4}\cdot 40}{\binom{52}{5}} + \frac{\binom{12}{5}}{\binom{52}{5}} - 0$. The overlap is 0 because "exactly 4" and "exactly 5" cannot both happen.

Bayes' theorem [W2 p25]

$$\underbrace{P(A \mid B)}_{\text{posterior}} = \frac{\overbrace{P(B \mid A)}^{\text{likelihood}}\overbrace{P(A)}^{\text{prior}}}{\underbrace{P(B)}_{\text{marginal}}}$$

IN PLAIN WORDS

In plain words. You start with a belief, the **prior** $P(A)$. You see evidence B. You re-weight the prior by how well A explains the evidence (the **likelihood**), then divide by the overall chance of the evidence (the **marginal**) so the probabilities add up to 1. The marginal often comes from the law of total probability: $P(B) = P(B \mid A)P(A) + P(B \mid \bar{A})P(\bar{A})$.

Bernoulli and binomial [W2 p26–p27; Bernoulli PMF from review file]

$$P(Y=y)=p^y(1-p)^{1-y},\ y\in\{0,1\} \qquad E[X]=p,\ \ \mathrm{Var}[X]=pq,\ \ q=1-p$$

$$b(k;n,p)=\binom{n}{k}p^k q^{n-k} \qquad E[X]=np \qquad \mathrm{Var}[X]=npq$$

IN PLAIN WORDS

In plain words. The Bernoulli formula is a trick that gives p when $y=1$ and $1-p$ when $y=0$. The binomial counts the successes k in n independent tries: $\binom{n}{k}$ counts the orderings, p^k covers the k successes, q^{n-k} the failures. Example: $n=3, p=\frac{1}{2}$ gives $P(k=2)=3\cdot\frac{1}{4}\cdot\frac{1}{2}=\frac{3}{8}$. **Why ML cares:** a binary class label is a Bernoulli variable, and that is the bridge to logistic regression [W4 p54].

Gaussian (normal) [W2 p28] and **z-score** [W2 p30]

$$\mathcal{N}(x;\mu,\sigma^2)=\sqrt{\frac{1}{2\pi\sigma^2}}\ \exp\Big(-\frac{1}{2\sigma^2}(x-\mu)^2\Big) \qquad z=\frac{x-\mu}{\sigma}$$

IN PLAIN WORDS

In plain words. The bell curve: μ is the center and σ^2 the width. The density is highest at $x=\mu$ and falls off with the squared distance from μ. A z-score of $+2$ means "2 standard deviations above the mean".

Linear algebra rules [W2 p36–p51]

Rule	Formula	In plain words
Transpose	$(A^T)_{ij}=A_{ji}$; $(AB)^T=B^TA^T$	flip rows ↔ columns; the order reverses for a product
Matrix product	$C=AB$, $C_{ij}=\sum_k A_{ik}B_{kj}$	entry (i,j) = row i of A dotted with column j of B
Identity	$I_nx=x$ for all x	the "1" of matrices
Inverse	$A^{-1}A=I_n$; $Ax=b\Rightarrow x=A^{-1}b$	undo A
L^p norm	$\|x\|_p=\left(\sum_i\|x_i\|^p\right)^{1/p}$	general "size" of a vector
L^2 (most popular)	$\|x\|_2=\sqrt{\sum_i x_i^2}$	straight-line length
L^1	$\|x\|_1=\sum_i\|x_i\|$	city-block length
Max norm	$\|x\|_\infty=\max_i\|x_i\|$	largest entry
Eigen	$Av=\lambda v$	A only stretches v by λ
Eigendecomposition	$A=V\,\mathrm{diag}(\lambda)V^{-1}$ (diagonalizable)	
Real symmetric A	$A=Q\Lambda Q^T$, Q orthogonal	real eigenvalues, perpendicular eigenvectors (this is why PCA works)
SVD	$A=UDV^T$ (any shape)	
Pseudoinverse	$A^\dagger=VD^\dagger U^T$; $D^\dagger=$ reciprocals of the non-zero entries	
Trace	$\mathrm{Tr}(A)=\sum_i A_{ii}$; $\mathrm{Tr}(ABC)=\mathrm{Tr}(CAB)=\mathrm{Tr}(BCA)$	sum of the diagonal; cyclic shifts are allowed

WORKED EXAMPLE / MODEL ANSWER

Mini examples (by hand). $\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}5\\ 6\end{bmatrix} = \begin{bmatrix}1\cdot 5 + 2\cdot 6\\ 3\cdot 5 + 4\cdot 6\end{bmatrix} = \begin{bmatrix}17\\ 39\end{bmatrix}$. Eigen check: $A = \begin{bmatrix}2 & 0\\ 0 & 3\end{bmatrix}$ has $v = (1, 0)$ with $Av = (2, 0) = 2v$, so $\lambda = 2$. For $x = (3, -4)$: $\|x\|_1 = 7$, $\|x\|_2 = 5$, $\|x\|_\infty = 4$.

NOT IN DECK — STANDARD DEFINITION

Marginal distribution (W2 p29 shows only the figure). For a joint distribution $p(x, y)$, the marginal of x is obtained by summing (or integrating) out the other variable: $p(x) = \sum_y p(x, y)$. In the 3-D figure, it is the "shadow" of the bell surface projected onto one axis.

Mechanisms: why and how

- **Why the sample variance divides by $n - 1$:** the sample mean $\bar{x}$ is computed from the same data, so the deviations are slightly too small; dividing by $n - 1$ corrects for that. *(Reason: not in deck — standard.)*
- **Why the median is not a moment:** moments are averages of powers of the values (x, $(x - \mu)^2$, ...). The median is a position in the sorted data [W2 p7].
- **Why mutually exclusive events are dependent:** if A happens, $P(B)$ drops to 0, so knowing A changes B [W2 p24].
- **Why an experiment gives causality and an observational study does not:** random treatment assignment balances every other factor between the groups [W2 p5].
- **Why the inverse is avoided in practice:** solving through A^{-1} is numerically unstable [W2 p41]. The pseudoinverse or iterative methods are used instead.
- **Why the eigen-picture matters for ML:** PCA's principal components are the eigenvectors of the covariance matrix, and the eigenvalues are the variance along each one [W7 p41].
- **Why a low p-value counts as evidence:** it says the data would be very surprising *if there were no effect* [W2 p31].

Comparisons

A	B	Key difference
Statistic	Parameter	describes a **sample** vs. a **population** [W2 p3]
Observational study	Experiment	no intervention vs. assigned treatment; only an experiment supports **causal** claims [W2 p5]
Discrete RV	Continuous RV	counted, PMF vs. measured, density $f(x)$ [W2 p11, p18]
Variance σ^2	Std. deviation σ	average squared deviation, in squared units vs. root-mean-square deviation, in the data's units [W2 p9]
Independent events	Mutually exclusive events	$P(A \cap B) = P(A)P(B)$ vs. $P(A \cap B) = 0$ (which makes them dependent) [W2 p24]

A	B	Key difference
Population variance	Sample variance	divide by n vs. $n-1$ [W2 p8]
Eigendecomposition	SVD	square, diagonalizable matrix only vs. **any** matrix [W2 p46, p48]
Inverse A^{-1}	Pseudoinverse $A^{\dagger}$	needs square, full rank vs. always exists (least-error or least-norm solution) [W2 p49]
L^1 norm	L^2 norm	sum of absolute values vs. root of the sum of squares (the basis of Lasso vs. Ridge in W4)
Skewness	Kurtosis	direction and asymmetry of the tail vs. heaviness of the tails

W4 — Regression (Linear, Polynomial, Regularized, Logistic)

Deck W4_Regression.pdf, 66 slides. Géron Ch 4. ***Outside reading (IBM / DeepLearning.AI): p. 94.***

Core definitions

The house-price task [W4 p2–p4]. Predict the market value y (in units of \$10K) from the number of rooms x with N examples, using the model $y = w_0 + w_1 x$. Here w_0 is the **intercept** and w_1 the **slope**. Learning means finding w_0 and w_1.

Overdetermined system [W4 p5]. N data points give N equations but only 2 unknowns, so **no line satisfies all of them**. We look for the *best-fit* (approximate) solution instead.

Loss function / SSE [W4 p7]. The error of one point is $(\hat{y}_i - y_i)^2$. The **sum of squared errors** over the dataset, $E(w)$, measures "goodness of fit". Between two candidate lines l_1 and l_2, choose the one with the lower SSE [W4 p6].

OLS assumptions of linear regression [W4 p9] . **Linearity** (the relationship is linear), **independence** (the residuals are independent), **homoscedasticity** (the residuals have constant variance), **normality** (the residuals are approximately normal), and **no multicollinearity** (the features are not highly correlated). Check linearity with a scatter plot first.

Gradient vector [W4 p14]. For a function of m variables, $\nabla_x f = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_m}\right]^T$. It **points in the direction of maximum increase**.

Gradient descent / method of steepest descent [W4 p13–p18] . Start at x_0 and repeatedly step in the direction of the **negative** slope until a convergence criterion is met. The **learning rate** sets how far each step goes. *Too small:* it takes a long time to converge ("stalls", p18). *Too large:* it overshoots and oscillates between values (p17).

Multiple features; bias column [W4 p22–p23]. $y_i = w_0 + \sum_{k=1}^{m} w_k x_{ik}$ has $m+1$ coefficients, all estimated together. Adding a constant feature $x_{i0} = 1$ lets you write $y_i = w^T x_i$. The **design matrix** X has a first column of 1s.

Training set vs. test set [W4 p30]. Both come from the same distribution. The model *trains* on the training set (computing statistics, estimating parameters), so training error is **not a good indicator of true performance**. The model sees the test set **only to evaluate**, so test error is the better indicator.

Overfitting [W4 p31] . The model learns the *idiosyncrasies (noise)* of the training data and generalizes poorly: **low training error, test error orders of magnitude higher**. It happens when there is *little training data relative to the number of parameters* or the *model is too complex.* The deck's example: **a degree-N polynomial exactly fits $N+1$ data points.**

Polynomial regression [W4 p33]. Fit a degree-K polynomial by **transforming the features** (for $K=2$: $x_1, x_2, x_1^2, x_2^2, x_1 x_2$) and then running ordinary linear regression. The model stays *linear in the parameters* [W4 p29].

Underfit / overfit / good fit [W4 p34–p37] . Degree 1 has a large training error and **underfits** the target. Degree 6 has negligible training error but **overfits**: it differs wildly from the target, and small changes in the data change it drastically. **Degree 3 has some training error but closely resembles the target function, so it is the best of the three.**

Avoiding overfitting [W4 p38]. (1) **More training data**: always works, but can be costly. (2) **Cross-validation**: tune parameters by splitting the training data into training and test parts. (3) **Regularization**: a mathematical framework for constraining model complexity.

Why large weights signal overfitting [W4 p39]. More complex models have **larger-magnitude parameters**. In the deck's table the weights for $P = 11$ reach tens of thousands (e.g. $w_5 = 35776.97$), while $P = 1$ has $w_0 = 4.03$, $w_1 = -6.79$. With large weights, a slight change in the input causes a large change in the prediction. So we **discourage large weights**.

Regularization [W4 p41] . It controls model complexity by **adding a term that shrinks the weights toward 0**. Because the penalty depends on weight size, **features must be standardized first**. The three methods are Ridge, LASSO, and Elastic Net. (W4 p42 shows the Lasso drives the "baths" weight to exactly $0x_3$.)

Ridge (L2) [W4 p43–p44] . Minimize RSS plus an L2 penalty. $\alpha \geq 0$ is a hyperparameter: **larger α shrinks the weights toward 0**. A good α is chosen by model tuning. In the deck's example the slope falls from $w_1 = 600.66$ (least squares) to 570.63 ($\alpha = 1$), 475.53 ($\alpha = 5$), 393.54 ($\alpha = 10$), and 95.90 ($\alpha = 100$).

LASSO (L1) [W4 p45–p46] . Minimize RSS plus an L1 penalty. A larger α shrinks the weights, and **some reach exactly 0, so LASSO also performs feature selection.**

Elastic Net [W4 p47] . Mixes the two penalties. $\lambda \in [0, 1]$ is the weight on L1: $\lambda = 0$ gives **Ridge**, $\lambda = 1$ gives **LASSO**. Tuning finds the best L1/L2 ratio.

Pros and cons of regularization [W4 p48]. It shrinks the weights, which prevents overfitting. LASSO and Elastic Net can set weights to 0, giving **simpler, cheaper, easier-to-interpret** models; Ridge only shrinks. **It lowers variance but adds bias**, so the hyperparameter controls the bias–variance tradeoff.

Logistic regression [W4 p49–p61] . A **binary classification** algorithm. A binary outcome is modeled as **Bernoulli** [W4 p54]: $P(Y = 1 \mid X) = p$ and $P(Y = 0 \mid X) = 1 - p$. The mean of the Bernoulli distribution *is* the probability of success, so we model p as a function of x.

Why not plain linear regression for a 0/1 target? [W4 p53–p55] . The deck lists three problems: (1) **non-normal error terms** ε; (2) **non-constant error variance** $\sigma^2(\varepsilon)$; (3) the **constraint on the response**: $0 \leq p \leq 1$, but a line goes outside $[0, 1]$. *(The review file adds that squared error is very sensitive to outliers, which shifts the boundary.)*

Odds, log-odds, sigmoid [W4 p59–p60]. The **odds** are $\frac{p}{1-p}$, success vs. failure. The **log-odds** (logit) $\ln \frac{p}{1-p}$ is the *inverse of the sigmoid.* The **sigmoid** maps the log-odds to a probability between 0 and 1.

Logistic regression pipeline [W4 p61]. Inputs → linear model $w^T x$ → sigmoid gives a probability → threshold classifier outputs 0 or 1.

Properties of the logistic curve [W4 p58]. **Sigmoidal (S-shaped), monotonic, and linearizable** through the logit transformation. *The log of the odds is linearly related to x.*

Parameter estimation: MLE vs. MAP [W4 p62–p65] . **MLE** picks the parameters that maximize the *conditional data likelihood*, $P(Y \text{ observed} \mid X)$. **MAP** treats w as a random variable with a **prior** (e.g. $w \sim \mathcal{N}(0, \sigma^2)$) and maximizes $P(w \mid \text{Data}) \propto P(\text{Data} \mid w)P(w)$. **MLE is the special case of MAP with an uninformative prior.** Neither has a closed-form solution in general, so any gradient-based optimizer is used. *(The 2015 exception: a closed form exists when all predictors are categorical and coded as orthogonal binary variables.)*

Formulas

Model and SSE loss [W4 p4, p7–p8]

$$\hat{y}_i = w_0 + w_1 x_i \qquad E(w) = \sum_{i=1}^{N} (\hat{y}_i - y_i)^2 = \sum_{i=1}^{N} (w_0 + w_1 x_i - y_i)^2 \qquad w^* = \arg\min_{w_0, w_1} E(w)$$

> **IN PLAIN WORDS**
> **In plain words.** For each house, measure how far the line's guess is from the real price, square it so that misses in both directions count, and add everything up. Learning means finding the slope and intercept that make this total smallest. Recipe [W4 p8]: take the derivative, set it to zero, solve.

Analytical solution, one feature [W4 p21]

$$w_1 = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{N}(x_i - \bar{x})^2} = \frac{\text{Cov}(x, y)}{\text{Var}(x)} \qquad w_0 = \bar{y} - w_1 \bar{x}$$

> **IN PLAIN WORDS**
> **In plain words.** Slope = "how x and y move together" ÷ "how much x moves on its own". The intercept makes the line pass through the point of averages $(\bar{x}, \bar{y})$.

Closed form, many features (normal equation) [W4 p27]

$$y = Xw \;\Rightarrow\; X^T y = X^T X w \;\Rightarrow\; w = (X^T X)^{-1} X^T y$$

> **IN PLAIN WORDS**
> **In plain words.** Multiply both sides by X^T to get a square system, then invert. This needs $X^T X$ to be invertible (W2: square and no redundant columns). Strongly correlated features (multicollinearity) make that inverse unstable.

Analytic minimization, deck example [W4 p11–p12]: $f(x) = 3x^2 + 4x + 1$, $f'(x) = 6x + 4 = 0 \Rightarrow x^* = -\frac{2}{3}$.

Gradient descent update [W4 p15]

$$x_{i+1} = x_i - \eta \nabla f(x_i)$$

IN PLAIN WORDS

In plain words. The gradient points uphill, so step **against** it (hence the **minus**). η is the step size. If the slope is positive, x decreases; if negative, x increases. At the minimum the slope is 0, so the updates stop changing x.

Gradient descent for linear regression [W4 p24]

$$J(w) = \frac{1}{2N}\sum_{i=1}^{N}(w^T x_i - y_i)^2 \qquad \frac{\partial J}{\partial w_0} = \frac{1}{N}\sum_{i=1}^{N}(w^T x_i - y_i) \qquad \frac{\partial J}{\partial w_1} = \frac{1}{N}\sum_{i=1}^{N}(w^T x_i - y_i)\, x_i$$

then $w_j \leftarrow w_j - \eta \frac{\partial J}{\partial w_j}$ for every j at the same time.

IN PLAIN WORDS

In plain words. The $\frac{1}{2}$ is there only to cancel the 2 that comes from differentiating a square. The gradient for w_0 is the **average error**. The gradient for w_1 is the **average error times the input** x_i. Points with big errors pull the weights hardest. W4 p25–p26 show the SSE surface as a bowl (convex) and the GD path walking down the contour plot to its center.

Regularized objectives [W4 p41–p47]

$$\text{Ridge: } RSS + \alpha\|w\|_2^2 = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \alpha\sum_{j=1}^{p} w_j^2$$

$$\text{LASSO: } RSS + \alpha\|w\|_1 = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \alpha\sum_{j=1}^{p} |w_j|$$

$$\text{Elastic Net: } RSS + \alpha\lambda\|w\|_1 + \alpha(1-\lambda)\|w\|_2^2, \quad \alpha \geq 0,\ 0 \leq \lambda \leq 1$$

IN PLAIN WORDS

In plain words. Total cost = "how badly I fit the data" + α × "how big my weights are". $\alpha = 0$ gives plain least squares; a huge α pushes all weights toward 0, which flattens the model. **Ridge** charges *weight squared*: big weights are very expensive, tiny ones almost free, so weights shrink but rarely hit 0. **LASSO** charges *absolute weight*: even tiny weights cost a steady amount, so the cheapest move is often to set a weight exactly to 0. The sum over j starts at 1: the intercept w_0 is not penalized.

Logistic regression [W4 p56–p60, p63–p64]

$$p = \frac{e^{\alpha+\beta x}}{1+e^{\alpha+\beta x}} = \frac{1}{1+e^{-(\alpha+\beta x)}} \qquad \sigma(w^T x) = \frac{1}{1+e^{-w^T x}}$$

$$\text{odds} = \frac{p}{1-p} = e^{\alpha+\beta x} \qquad \text{logit: } \ln\left(\frac{p}{1-p}\right) = \alpha + \beta x$$

IN PLAIN WORDS

In plain words. Compute a score $z = \alpha + \beta x$, which can be any number. The sigmoid squashes it into $(0, 1)$: $z = 0$ gives $p = 0.5$, large positive z gives $p \to 1$, large negative z gives $p \to 0$. Reading backwards, the *log of the odds* is a straight line in x, which is why it is still "regression". Increasing x by 1 multiplies the odds by e^{β}. *(Odds interpretation: standard.)* Note that $\frac{e^{-x}}{1+e^{-x}} = \frac{1}{1+e^{x}}$ is the mirrored, decreasing S-curve [W4 p57].

$$\text{MLE: } w, b = \arg\min_{w,b} \sum_{i=1}^{n} \log\left(1 + e^{-y_i(w^T x_i + b)}\right) \qquad \text{MAP: } \arg\min_{w} \sum_{i=1}^{n} \log\left(1 + e^{-y_i w^T x_i}\right) + \lambda w^T w$$

IN PLAIN WORDS

In plain words. Here the labels are coded $y_i \in \{-1, +1\}$. When $y_i(w^T x_i)$ is large and positive (confidently correct), $e^{-\text{large}} \approx 0$ and the loss is $\log 1 = 0$. When it is negative (wrong), the loss grows. **MAP = MLE + an L2 penalty** $\lambda w^T w$. The Gaussian prior on w acts *exactly like Ridge regularization*, with λ a linear function of $\frac{1}{2\sigma^2}$ [W4 p63].

Mechanisms: why and how

- **Why "degree N fits $N+1$ points" is the overfitting example (the n+1 quiz):** a degree-N polynomial has $N+1$ coefficients, so with $N+1$ points you can solve for a curve through every point. Training error $= 0$ no matter what the true pattern is, even for pure noise. So a perfect training fit tells you nothing about generalization [W4 p31].
- **Why degree 3 beats 1 and 6 [W4 p37]:** degree 1 is too rigid (high bias). Degree 6 chases the noise (high variance; on new data its error explodes). Degree 3 matches the *shape* of the true sine-like target. Judge by **test/generalization error, not training error**.
- **Why gradient descent needs the minus sign:** ∇f points to the steepest *increase*; to minimize, go the opposite way [W4 p13–p14].
- **Why the learning rate matters:** too small means tiny steps and a long time to converge; too large means jumping across the valley, overshooting and oscillating, possibly diverging [W4 p16–p18].
- **Why regularization reduces overfitting:** overfit models have huge weights [W4 p39]. Penalizing weight size forces a smoother function that reacts less to small input changes, so variance falls (at the cost of some bias) [W4 p48].
- **Why standardize before regularizing:** the penalty is based on weight size, and weight size depends on the feature's units. Without scaling, a feature measured in small units would be penalized unfairly [W4 p41].
- **Why Lasso does feature selection and Ridge does not:** see the L1 vs. L2 figure (p. **67**). The $|w|$ penalty has a sharp corner at 0, so the optimum often lands exactly on 0. The w^2 penalty is flat near 0, so weights shrink but stay non-zero [W4 p45–p48].
- **Why logistic regression uses the sigmoid:** it keeps p in $[0, 1]$, gives the S-shape probabilities really have, and makes the log-odds linear [W4 p55–p58].
- **Why MLE/MAP use gradient methods:** there is no closed-form solution in general [W4 p65].

Comparisons

A	B	Key difference
Analytical (closed form)	Iterative (gradient descent)	solve $\nabla = 0$ exactly, $w = (X^T X)^{-1} X^T y$ vs. take repeated small downhill steps; GD scales to huge data and works when no closed form exists [W4 p11–p13, p27]
Too-small learning rate	Too-large learning rate	stalls, slow convergence vs. overshoots, oscillates [W4 p16–p18]
Underfitting	Overfitting	too simple, high train *and* test error (high bias) vs. too complex, low train but high test error (high variance) [W4 p34–p36]
Ridge (L2)	**LASSO (L1)**	penalty $\alpha \sum w_j^2$ vs. $\alpha \sum \lvert w_j \rvert$; shrinks, never exactly 0 vs. shrinks and **sets some weights exactly to 0** (feature selection, sparse); keeps correlated features together vs. tends to keep one of them; cost like linear regression vs. simpler, cheaper, more interpretable model [W4 p43–p48]
Elastic Net, $\lambda = 0$	Elastic Net, $\lambda = 1$	equals Ridge vs. equals LASSO [W4 p47]
Linear regression	Logistic regression	numeric target, unbounded output, SSE loss vs. binary target, probability in (0,1), log loss / MLE [W4 p51–p64]
Odds	Probability	$\frac{p}{1-p} \in (0, \infty)$ vs. $p \in [0, 1]$ [W4 p59]
MLE	MAP	maximize the likelihood only vs. likelihood $\times$ prior; MAP with a Gaussian prior = MLE + L2 penalty; MLE = MAP with an uninformative prior [W4 p62–p64]
Training error	Test error	measured on the data used for fitting (optimistic) vs. on unseen data (honest) [W4 p30]

W5 — Model Performance Evaluation

Deck W5_Model_Evaluation.pdf, 30 slides. Géron Ch 2–3 (metrics, cross-validation) and Ch 4 (bias/variance).
Outside reading (IBM / DeepLearning.AI): p. 94.

Core definitions

Model specification and complexity [W5 p2–p3]. A model is a function $f(x)$ with $y = f(x)$. A key choice is its **complexity**, for example whether $y = \beta_0 + \beta_1 x$ is better than $y = \beta_0 + \beta_1 x + \beta_2 x^2$. Underfit (too simple) and overfit (too complex) models both miss the pattern. Model selection aims for a **moderately complex** model that captures the trend without fitting noise.

Breaking down error [W5 p4] . Total error = **bias** (how far the model's assumptions deviate from the observed values) + **variance** (how spread out the model's predictions are) + **irreducible error** (inherent and unavoidable).

Bias vs. variance as darts [W5 p5]. Bias is the shots' average distance **from the bullseye**. Variance is their average distance **from one another**. Low bias + low variance means a tight cluster on the bullseye.

Bias–variance tradeoff [W5 p6]. As complexity increases, **variance increases and bias decreases**. The optimal model **minimizes total error** by balancing the two (a U-shaped total-error curve; see the gallery).

ML basics [W5 p7]. Data has labeled instances (features + output label) or unlabeled ones. Supervised learning does **classification** (categorical output) or **regression** (numeric output). *Training* estimates the parameters, for example the slope and intercept for predicting insurance premiums.

Training / validation / test split [W5 p8–p10] . **Training data** fits the model. **Validation data** evaluates candidate models: *tuning parameters and selecting features.* **Test data** evaluates the **final** model and compares different models. All three should have a **similar distribution**, with the largest share for training (the deck shows 80/10/10 and 70/10/20). Fitting on the full sample risks overfitting.

Loss function vs. metric [W5 p11, p15]. A loss function measures the gap between predictions and observations (lower is better). The **metric** is the loss function's observed value for a fitted model. Classification losses penalize misclassifications, *and even correct but uncertain predictions.* A perfect classifier has loss 0.

Absolute loss for classification [W5 p16–p17]. $L_{abs}(y, \hat{p}) = |y - \hat{p}|$, where $\hat{p}$ is the predicted probability. Deck example: a benign tumor ($y = 0$) at radius 14.05 mm has $\hat{p} = 0.286$, so $L_{abs} = |0 - 0.286| = 0.286$. It is still classified as 0 at threshold 0.5, but the loss records the uncertainty. The overall loss is the average over all 171 instances, $= 0.183$.

Log loss [W5 p18] . The **negative log-likelihood** of the predicted probability. The likelihood measures model–data consistency (higher = better fit); its values are tiny, so the *log*-likelihood is used. **Higher likelihood gives smaller log loss.** The closer $\hat{p}$ is to the true class (0 or 1), the closer the loss is to 0.

Confusion-matrix vocabulary [W5 p19] . **TP**: correctly identified as positive. **TN**: correctly identified as negative. **FP**: identified as positive but actually negative (a "false alarm"). **FN**: identified as negative but actually positive (a "miss"). The **confusion matrix** summarizes predicted vs. actual; for a binary classifier it is 2×2.

Accuracy, precision, recall [W5 p20]. **Accuracy**: correct predictions out of all predictions. **Precision**: correct positive predictions among all **predicted** positives. **Recall (sensitivity)**: correct positive predictions among all **actual** positives.

Precision–recall tradeoff [W5 p21]. Increasing precision often decreases recall, and vice versa.

ROC and AUC [W5 p22] . An ROC curve assesses how well a model separates the two classes **across probability cutoffs** by plotting the **true positive rate against the false positive rate**. **AUC** (area under the curve) compares two binary models: **larger AUC = better** class prediction.

Cross-validation [W5 p23–p25] . A single train/validation/test split may leave key instances out of training. *Resampling* methods give a better assessment. **k-fold CV** divides the data into k folds and trains/validates repeatedly on different fold combinations. *Exhaustive* CV uses every possible way of splitting (the deck writes "k = #instances in the training data − 1", i.e. leave-one-out-style). *Non-exhaustive* CV does not.

Choice of k in k-fold [W5 p24]. **Larger k**: smaller validation sets, **more variability**, and more computation (more models, each trained on larger folds). **Smaller k**: **more bias**. For complex models or large datasets a smaller k may be needed.

Bootstrapping [W5 p26–p27] . When population information is missing, bootstrapping **simulates sampling by drawing with replacement** from the existing sample. It gives the distribution of a statistic and enables interval estimation, for example a confidence interval for an incumbent's vote share from exit polls. For model evaluation [W5 p27]: (1) draw bootstrap samples; (2) the instances not drawn form the **out-of-bag** sample; (3) train on each bootstrap sample and compute the error on its out-of-bag sample; (4) examine the error distribution (a small error variance plus a correctly specified model means it performs well).

Model selection [W5 p28–p29]. Pick a model by validation error (the error-bar plot on p28 selects polynomial **degree 3**: low MSE and a small error bar), or by other indicators: the **lowest information criterion** (AIC or BIC, balancing fit against complexity), or the **highest adjusted R^2** (which corrects R^2 for the number of features).

Formulas

Regression losses [W5 p13–p14]

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \qquad MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

IN PLAIN WORDS

In plain words. MSE averages the **squared** misses. It is in squared units and is **sensitive to outliers**, because one huge miss squared dominates. RMSE takes the root to return to y's units, so models can be compared in real units. MAE averages the **absolute** misses: same units as y, **less affected by outliers**, but it "lacks some math properties" (the absolute value has no derivative at 0), so it is used less. Example: errors $1, -1, 4$ give MAE $= \frac{1+1+4}{3} = 2$ and MSE $= \frac{1+1+16}{3} = 6$, so RMSE ≈ 2.45.

Classification losses [W5 p16, p18]

$$L_{abs}(y, \hat{p}) = |y - \hat{p}| \qquad L_{log}(y, \hat{p}) = -\big(y \log \hat{p} + (1-y)\log(1-\hat{p})\big)$$

IN PLAIN WORDS

In plain words (log loss). Only one of the two terms is "on". If the truth is $y = 1$, the loss is $-\log \hat{p}$; if $y = 0$, it is $-\log(1-\hat{p})$. So the loss is **minus the log of the probability you gave to the right answer**. Giving the right answer probability 1 costs $-\log 1 = 0$. Giving it 0.5 costs $-\log 0.5 \approx 0.69$ (natural log). Giving it a tiny probability costs a **huge** amount, so **confident mistakes are punished very heavily**. The model's log loss is the average over all instances.

Metrics from the confusion matrix [W5 p20; F1 from the review file]

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \qquad \text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \qquad TPR = \text{Recall} \qquad FPR = \frac{FP}{FP + TN}$$

IN PLAIN WORDS

In plain words. Look at **which column or row you divide by**. *Precision* divides by everything you **called** positive: "when I say yes, how often am I right?" *Recall* divides by everything that **is** positive: "of all the real yeses, how many did I catch?" *F1* is the harmonic mean, which is high only when **both** are high. *Accuracy* can mislead on imbalanced data: with 99% negatives, always saying "no" is 99% accurate yet has recall 0. (FPR, F1 and the accuracy-on-imbalanced-data point are standard; F1 is from the review file.)

WORKED EXAMPLE / MODEL ANSWER

Layout to draw in the exam (rows = actual, columns = predicted):

	Predicted +	**Predicted −**
Actual +	TP	FN
Actual −	FP	TN

Precision reads **down** the "Predicted +" column; recall reads **across** the "Actual +" row. A fully worked numeric example is in p. **83**.

Mechanisms: why and how

- **Why we need a test set separate from validation:** validation data is *used* to choose hyperparameters and features, so it becomes optimistic. Only untouched test data gives an honest final estimate [W5 p9].
- **Why bias falls and variance rises with complexity:** a flexible model can bend toward the true pattern (less bias), but it also bends toward the noise of each particular training set, so its predictions vary more from sample to sample (more variance) [W5 p6].
- **Why log loss rather than accuracy for probabilistic classifiers:** it uses the *probability*, rewarding confident correct predictions and punishing confident wrong ones, while accuracy only sees the thresholded label [W5 p15, p18].
- **Why there is a precision–recall tradeoff:** raising the threshold for "positive" produces fewer positive predictions. The ones that remain are more certain (precision ↑), but more true positives are missed (recall ↓) [W5 p21].
- **How an ROC curve is built:** sweep the probability cutoff from 1 down to 0. Each cutoff gives one (FPR, TPR) point, and connecting them gives the curve. The diagonal is random guessing (AUC 0.5); the top-left corner is perfect (AUC 1) [W5 p22].
- **Why k-fold CV is better than one split:** every instance is used for validation exactly once and for training $k-1$ times, so the estimate depends less on one lucky or unlucky split [W5 p23].
- **Why bootstrapping works without population data:** resampling the sample with replacement imitates drawing new samples from the population. About one third of the instances are left out each time, and they form a free validation set (the out-of-bag data) [W5 p26–p27]. *(The $\approx 1/3$ figure is standard, not from the deck.)*

Comparisons

A	B	Key difference
Bias	Variance	systematic error from wrong assumptions, falls with complexity vs. sensitivity to the particular training set, rises with complexity [W5 p4–p6]
Validation set	Test set	tune hyperparameters, select features vs. final, untouched evaluation and comparison [W5 p9]
MSE	MAE	squared, outlier-sensitive, smooth math vs. absolute, robust to outliers, used less [W5 p13–p14]
MSE	RMSE	squared y units vs. same units as y [W5 p13]
Absolute loss	Log loss	$\lvert y-\hat{p}\rvert$, linear penalty vs. $-\log$(prob. of truth), explodes for confident errors [W5 p16, p18]

A	B	Key difference
Precision	**Recall**	TP ÷ **predicted** positives vs. TP ÷ **actual** positives. Precision matters when false alarms are costly (spam filter); recall matters when misses are costly (cancer screening) [W5 p20–p21]
Accuracy	F1	all correct ÷ all (misleading with class imbalance) vs. harmonic mean of precision and recall
ROC curve	AUC	the whole TPR-vs-FPR curve vs. one number summarizing it (bigger = better) [W5 p22]
Large k in k-fold	Small k	small validation folds, more variance, more compute vs. more bias, less compute [W5 p24]
Exhaustive CV	Non-exhaustive CV	every possible split vs. a subset of the splits [W5 p25]
k-fold CV	Bootstrapping	partition without replacement, each point validated once vs. sample **with** replacement, validate on out-of-bag points [W5 p23–p27]
AIC / BIC	Adjusted R^2	choose the **lowest** vs. the **highest** [W5 p29]

W6 — Supervised Learning: kNN, Naive Bayes, SVM

*Deck W6_Supervised_Learning.pdf, 21 slides. Géron Ch 5 (SVM). kNN is mentioned only briefly in Géron Ch 3, and **Naive Bayes is not covered in Géron**, so this section is self-contained. **Outside reading (IBM / DeepLearning.AI): p. 94.***

Core definitions

Supervised learning [W6 p2–p5]. Uses **labeled** instances (known outcomes). The outcome is the *target / dependent* variable; the inputs are *features / independent* variables. Goal: predict unlabeled instances. **Regression** predicts numbers (house prices: linear regression, decision trees, random forests, kNN). **Classification** predicts categories (purchase, retention, spam: logistic regression, decision trees, random forests, naive Bayes, SVM, neural networks). Models aim to **predict** and/or **explain**: scientists favor interpretability, image recognition favors predictive power.

k-nearest neighbors (kNN)

Idea [W6 p6]. Predict using nearby instances. Choose k, find the k nearest instances, and predict from their outputs. "**Birds of a feather flock together**": similar inputs give similar outputs. A **metric** is a way of measuring the distance between instances; the most common is **Euclidean**.

Regression with kNN [W6 p8]. (1) Select k. (2) Compute the distance from the new instance to all labeled instances. (3) The k closest are the nearest neighbors. (4) Predict the **average** of their outputs. *For classification, take the **majority vote** (review file).*

Deck example [W6 p7]. Query point (6,3), $k = 2$. Distances: to (2,4) $= 4.12$, to (9,7) $= \sqrt{3^2 + 4^2} = 5$, to (1,0) $= 5.83$. The 2 nearest are **(2,4) and (9,7)**.

Naive Bayes

What it is [W6 p9–p11]. A supervised classifier that estimates likelihoods from **how often categories occur in each class**. It is particularly suited to **text** (authorship identification, spam). Classification methods estimate $P(\text{Class} \mid \text{Data})$. Bayes' rule reverses the conditioning, and the words are **assumed independent**.

Process [W6 p12]. (1) Note which categories (words) are present and absent in the instance. (2) Class likelihood = the product of the category-match proportions within the class $\times$ the class's share of the training set. (3) Pick the class with the **highest likelihood**.

Pros and cons [W6 p10]. *Advantages*: works well with many features; fast to calculate; handles rare events, categorical, and missing data well. *Disadvantages*: needs a large amount of training data; continuous data must be preprocessed; **assumes features are independent and equally important**.

Why "naive" [W6 p14] . It **assumes the words of a message are independent of each other** (conditional independence given the class). This contradicts how language works, where word order matters: "I love bacon" is more likely than other orderings. Despite that, naive Bayes remains effective.

Laplace smoothing [W6 p13] . Without it, naive Bayes assigns **zero likelihood** whenever a category never appears with a class in training (e.g. staff never tweeting "love"). Because the likelihood is a *product*, one zero wipes out the whole class. **Laplace smoothing adds a fictional occurrence to every category's count**, even unobserved ones, so every category contributes and **no likelihood is zero**.

Support vector machines (SVM)

What it is [W6 p15–p16]. A supervised algorithm for classification and regression. It finds the **hyperplane** that separates the classes while **maximizing the margin**. The **margin** is the space between two hyperplanes parallel to the dividing one, touching the closest instances. The **support vectors** are the instances closest to the hyperplane, and **they alone determine its position** (W6 p17: "the points on the margin's edge are the support vectors that determine the dividing line").

Kernel trick [W6 p15, p18] . If a hyperplane cannot separate the classes in the original features, **map the data to a higher-dimensional space where it becomes linearly separable**. The kernel is a specialized mapping that makes it efficient to find the optimal hyperplane in that space. *Kernels* [W6 p18]:

Kernel	Mapping
Linear	no special mapping; the hyperplane uses the input features directly
Polynomial	maps to a space with every product of features up to the degree. Two features x, y (degree 2) map to the 6-D space $x^2, xy, y^2, x, y, 1$
Radial basis function (RBF)	each new feature relates to the distance from an instance: **closer = larger value, decaying exponentially** with distance

Soft margin and hinge penalty [W6 p19–p20] . For **non-separable** data (Chinstrap and Adelie penguins overlap), SVM allows instances inside the margin or on the wrong side, **with a penalty**. The common penalty is the **hinge function: zero on one side of a threshold, increasing linearly on the other**. The penalty is computed from **how far the instance is into the margin**. **Correctly classified instances outside the margin have no penalty.** The best hyperplane balances **margin width against total penalty**, and this balance is tuned through the hinge function's steepness. Support vectors now include the **margin and misclassified** instances.

Formulas

Euclidean distance [W6 p6] (Manhattan from W10 p44 and the review file)

$$dist(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_p - y_p)^2} \qquad d_{Manhattan}(x, y) = \sum_{j=1}^{p} |x_j - y_j|$$

IN PLAIN WORDS

In plain words. Euclidean distance is Pythagoras: subtract feature by feature, square, add, take the root. That is the straight-line distance. Manhattan distance is the city-block walk: just add the absolute differences. Example: from (6,3) to (9,7), Euclidean $= \sqrt{9+16} = 5$ and Manhattan $= 3+4 = 7$.

Distance-weighted kNN (review file): $w_i = \dfrac{1}{d(x', x_i)^2}$. Closer neighbors get bigger votes, which also helps break ties.

Naive Bayes [W6 p11–p12]

$$P(VIP \mid Message) = \frac{P(Message \mid VIP)\,P(VIP)}{P(Message)} \qquad P(Message \mid VIP) \approx P(Word_1 \mid VIP)\,P$$

General form: $\hat{c} = \arg\max_c \; P(c) \prod_j P(x_j \mid c)$. The denominator $P(\text{Data})$ is the same for every class, so it can be dropped when comparing.

IN PLAIN WORDS

In plain words. Score each class = (how common the class is) × (for each word: how often that word appears in this class's messages, or does *not* appear, if absent). The biggest score wins. The "≈" is exactly the **naive independence assumption**: the joint probability of all the words is replaced by a product of one-word probabilities.

Laplace smoothing (formula from the review file; the deck states the idea in words)

$$P(x_j \mid C) = \frac{\text{count}(x_j, C) + 1}{\text{count}(C) + V} \qquad V = \text{number of categories}$$

IN PLAIN WORDS

In plain words. Pretend you saw every category once more than you did. A word never seen with a class gets $\frac{0+1}{\text{count}+V} > 0$ instead of 0, so the product never collapses to 0. Adding V to the denominator keeps the probabilities summing to 1.

WORKED EXAMPLE / MODEL ANSWER

Deck worked example [W6 p12]: classify the new post "Tomorrow. Bacon!!!!" The training counts are 200 VIP posts and 87 Staff posts (287 total).

Term	VIP	Staff
Sandwich	36	23
Tomorrow	12	31
Bacon	43	11

The post contains *Tomorrow* and *Bacon* but **not** *Sandwich*, so we use "not Sandwich" = total − count.

$$\text{VIP: } \frac{200-36}{200}\cdot\frac{12}{200}\cdot\frac{43}{200}\cdot\frac{200}{200+87}\approx 0.0074 \qquad \text{Staff: } \frac{87-23}{87}\cdot\frac{31}{87}\cdot\frac{11}{87}\cdot\frac{87}{287}\approx 0.0100$$

$0.0100 > 0.0074$, so we **predict Staff**. Normalized posterior: $P(\text{Staff} \mid \text{post}) = \frac{0.0100}{0.0100+0.0074} \approx 0.57$.

SVM geometry [W6 p16] (formulas from the review file; the deck describes them in words)

hyperplane: $w \cdot x + b = 0$ margin $= \dfrac{2}{\|w\|}$ hinge loss: $\max\big(0,\ 1 - y_i(w \cdot x_i + b)\big),\ y_i \in \{-1,$

IN PLAIN WORDS

In plain words. $y_i(w \cdot x_i + b)$ is a "correctness score": positive means the right side, ≥ 1 means the right side **and** outside the margin. The hinge loss is then 0 (the **flat part**). If the score is below 1 (inside the margin) or negative (wrong side), the loss rises linearly (the **rising part**). A *smaller* $\|w\|$ means a *wider* margin, which is why SVM keeps w small. The tradeoff parameter C (review file) weighs "wide margin" against "few penalties": large C means penalties hurt more, giving a narrower margin and fewer violations.

Mechanisms: why and how

- **Why kNN needs feature scaling:** distance adds up raw differences, so a feature measured in large units (income in dollars) swamps one in small units (age) [review file]. Decision trees do not need scaling, because they compare one feature to a threshold at a time [W8 p13].
- **Why k matters in kNN:** a small k is noisy and follows single points (overfitting). A large k lets distant points outvote close ones. For 2 classes, use an **odd** k to avoid ties [review file].
- **Why kNN is a "lazy learner":** there is no training step, it just stores the data. Prediction is expensive because it computes distances to all training points [review file].
- **Why naive Bayes is fast and works with many features:** each $P(x_j \mid c)$ is a simple count ratio, and the independence assumption avoids estimating huge joint tables [W6 p10].
- **How Laplace smoothing prevents zero likelihood:** add one pseudo-count to every category, so no factor in the product is ever 0 [W6 p13].
- **Why only the support vectors matter:** the optimal hyperplane is fixed by the points that touch the margin. Moving or removing any other point (one far from the boundary) does not change it [W6 p16–p17].

- **Why the kernel trick helps:** data that are tangled in low dimensions often become separable by a flat plane after adding features such as products or distances. The kernel does this efficiently without hand-building those features [W6 p18]. *(Formal statement: the kernel computes inner products in the high-dimensional space without computing the mapping explicitly. This is standard; it is in the review file, not the deck.)*
- **Why hinge loss gives SVM its shape:** points safely on the correct side cost nothing, so they do not influence the solution. Only margin violators and misclassified points push the hyperplane [W6 p19–p20].

Comparisons

A	B	Key difference
kNN regression	kNN classification	**average** of the neighbors' outputs vs. **majority vote** [W6 p8; review file]
Small k	Large k	flexible, noisy, overfits vs. smooth, may underfit or be outvoted by far points
Euclidean	Manhattan	$\sqrt{\sum(x_j - y_j)^2}$ (L2) vs. $\sum\|x_j - y_j\|$ (L1)
Naive Bayes	Logistic regression	**generative**: models $P(x \mid c)P(c)$ vs. **discriminative**: models $P(c \mid x)$ directly [W11 p24]
Hard-margin SVM	Soft-margin SVM	separable data, no points allowed in the margin vs. overlapping data, violations allowed with a hinge penalty [W6 p16, p19]
Linear kernel	Polynomial / RBF kernel	original features vs. a mapping to a higher-dimensional space [W6 p18]
Support vectors	Other points	on or inside the margin, or misclassified, they define the hyperplane vs. no effect on the hyperplane [W6 p20]
Without Laplace smoothing	With Laplace smoothing	unseen category $\Rightarrow$ likelihood $0 \Rightarrow$ class impossible vs. every category > 0 [W6 p13]

W7 — Unsupervised Learning: Clustering, DBSCAN, Factor Analysis, PCA

Deck W7_Unsupervised_Learning.pdf, 43 slides. Géron Ch 9 (K-Means, DBSCAN) and Ch 8 (PCA). ***Outside reading (IBM / DeepLearning.AI): p. 95.***

Core definitions

Unsupervised learning [W7 p2–p3]. Identifies patterns **without labeled outputs**. Its outputs are **clusters, outliers, and latent variables**, from structured data (names, addresses) or unstructured data (images, audio, emails).

Applications [W7 p4–p6]. *Clustering*: customer segmentation (mall data with age, income, and spending score gives 5 clusters; Gap Inc.'s brands each target a segment). *Outlier detection*: density-based methods flag unusual house prices (Tesla's 2020 Austin move: 76 homes formed 1 cluster plus 4 outliers). *Latent variables*: combine correlated features into one hidden variable (economists measuring "well-being").

Centroid clustering [W7 p7]. Clusters are sets of similar instances. A **centroid** represents the cluster center, and similarity is the distance to the centroid.

K-Means

Setup [W7 p8–p9]. Input: observations $x_i \in \mathbb{R}^d$ and the number of clusters k. Output: cluster assignments and centroids $\mu_j \in \mathbb{R}^d$. Each point gets a binary **1-of-k** vector z_i: $z_{ij} = 1$ if point i is in cluster j, all other entries 0.

Algorithm [W7 p11–p12] . (1) Initialize k centroids. (2) Repeat until convergence: (3) **assignment step**: compute z_{nk} by assigning each point to the **nearest centroid**; (4) **update step**: recompute each μ_k as the **mean of its points**.

Convergence [W7 p17, p25] . The plot of J vs. iteration falls (about $890 \to 515 \to 215 \to 80$) and then stays **flat**. K-Means **often terminates at a local optimum**.

Choosing k [W7 p19–p23] . Idea 1: cross-validation. Idea 2: a domain expert judges the clustering. Idea 3: the **knee / elbow**. Plot J against k: $k = 1$ gives 873.0, $k = 2$ gives 173.1, $k = 3$ gives 133.6. "**The abrupt change at k = 2 is highly suggestive of two clusters**."

What K-Means needs [W7 p18]. Selected features, representative data, a clustering algorithm, and a specified number of clusters.

K-modes / k-prototype [W7 p24]. For **categorical** data, replace the means with **modes** and use a new dissimilarity measure. For mixed categorical and numeric data, use the **k-prototype** method.

Strengths and weaknesses [W7 p25] . *Strengths*: relatively efficient, $O(tkn)$ (n objects, k clusters, t iterations, normally $k, t \ll n$); often terminates at a local optimum. *Weaknesses*: needs the mean to be defined (a problem for categorical data); k **must be specified in advance**; **cannot handle noisy data and outliers**; **not suitable for non-convex cluster shapes**.

Hard vs. soft assignment [W7 p26]. K-Means makes **hard** guesses. A probabilistic version would let z_{nk} be the *probability* that point n belongs to cluster k (a soft assignment, which leads to mixture models).

Image segmentation [W7 p27] uses K-Means on pixel colors. **Mini-batch K-Means** [W7 p28] is a scalable version that updates using **small random batches of fixed size**.

Hierarchical clustering

Types [W7 p29]. Builds a **hierarchy** from single-instance clusters to one cluster with everything. **Agglomerative** starts with individual clusters and **merges** them iteratively; it is computationally simpler and is the one emphasized. **Divisive** starts with everything in one cluster and **splits**. Both perform similarly, but the clusters can differ.

Linkage (cluster similarity) [W7 p30]. **Single**: distance between the **most similar** instances of the two clusters. **Complete**: distance between the **most different** instances. **Centroid**: distance between the **centroids**.

Dendrogram [W7 p31–p32] . The tree output, showing the grouping order and distances. A **clade** is a branch (a vertical line). A **link** connects clades, and **its height is the distance between the clusters**. A **leaf** is the terminal end, a single instance. To choose the number of clusters, draw a threshold line: **the number of clades it intersects = the number of clusters** (the deck's example cuts 3 clades, giving 3 clusters). Do not decide from the dendrogram alone; also use quantitative methods such as the elbow.

DBSCAN (density-based)

Why [W7 p33–p34]. K-Means struggles with non-spherical shapes and outliers. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups points in **dense, connected regions**, handles arbitrary shapes, and **detects outliers** (data outside any cluster).

Terminology [W7 p35] . **ε-neighborhood**: a sphere of radius ε around a point. **Core point**: its ε-neighborhood contains at least the minimum number of points (min_samples). **Border point**: inside a core point's ε-neighborhood but not a core point itself. **Outlier**: neither a core nor a border point.

Algorithm [W7 p36]. Given ε and min_samples: (1) count the points in each ε-neighborhood and mark the core points; (2) connect core points that are within each other's ε-neighborhood into clusters; (3) assign the points within an ε-neighborhood to those clusters; (4) points outside every neighborhood are **outliers**. (The example uses $\varepsilon = 2$, min_samples $= 3$.)

Pros and cons [W7 p33]. *Advantages*: works well with outliers; finds arbitrary shapes; **no preset number of clusters**. *Disadvantages*: **fails with varying densities**; non-deterministic for some border points; does not work well with **high-dimensional** data.

Factor analysis and PCA

Factor analysis [W7 p37–p39]. Explains the data's variance using **fewer latent variables (factors)**. It groups correlated features (correlation > 0.70 are combined) and reveals hidden variables. The rock dataset has 48 samples with area, perimeter, shape, and permeability. It studies **variance, not the**

mean. Variance splits into **common** variance (shared) and **unique** variance (from error or specific causes). **Communality** h_i^2 (between 0 and 1) = the sum of the squared factor loadings.

Factor model [W7 p38]: $x_i - \mu_i = l_{i,1}F_1 + l_{i,2}F_2 + \cdots + l_{i,n}F_n + \epsilon_i$. That is a sequence of multiple linear regressions. The **factor loading** $l_{i,j}$ is the influence of factor F_j on feature x_i, and it can be positive or negative.

PCA [W7 p40] . Factor analysis has two steps, **extraction and rotation**. **PCA is a type of factor analysis that identifies orthogonal components.** It seeks the components with **maximum variability**: the first component has the most, and each later one contributes less.

Eigenvalues and eigenvectors in PCA [W7 p41] . **An eigenvalue is a factor's explained variability**, computed by summing the squared loadings across the features. **The eigenvector holds the coefficients of the input features**: a factor is a combination of the inputs.

Scree plot [W7 p42] . PCA aims to use **fewer factors**. You *can* keep as many factors as features, but some have little variability and can be dropped. A **scree plot of the eigenvalues (sorted, descending)** helps select them: **keep the factors to the left of the leveling-off point**. In the deck's plot (eigenvalues $\approx 2.6, 0.95, 0.27, 0.02$), the leveling-off point is at factor 3, so **keep factors 1–2**.

Formulas

K-Means objective [W7 p10]

$$J = \sum_{n=1}^{N}\sum_{k=1}^{K} z_{nk}\,\|x_n - \mu_k\|^2$$

> **IN PLAIN WORDS**
>
> **In plain words.** For every point, take the squared distance to the center of the cluster it is assigned to, and add these up over all points. z_{nk} is a $0/1$ switch, so only the assigned cluster counts. Small J means tight clusters. This is the **within-cluster sum of squares (WCSS)** from the review file.

The two alternating steps [W7 p11]

$$\text{Assign: } z_{nk} = \begin{cases} 1 & k = \arg\min_j \|x_n - \mu_j\|^2 \\ 0 & \text{otherwise} \end{cases} \qquad \text{Update: } \mu_k = \frac{\sum_{n=1}^{N} z_{nk}\, x_n}{\sum_{n=1}^{N} z_{nk}}$$

> **IN PLAIN WORDS**
>
> **In plain words.** *Assign*: with the centers fixed, put each point in its nearest center's cluster. Each point can be handled independently. *Update*: with the assignments fixed, set the derivative of J to zero; the best center is the **average** of the points in the cluster (sum of the points ÷ number of points). One full step by hand is worked in p. **83**.

PCA quantities [W7 p41]

$$\text{Percent variability} = \frac{\text{eigenvalue}}{\#\text{features}} \qquad \text{Loadings} = \text{Eigenvector} \times \sqrt{\text{Eigenvalue}}$$

IN PLAIN WORDS

In plain words. With standardized features, the eigenvalues add up to the number of features, so eigenvalue ÷ #features = the **share of total variance** a component explains. In the deck's scree example with 4 features: $2.6/4 = 65\%$ and $0.95/4 \approx 24\%$, so the first two components keep about 89%.

NOT IN DECK — STANDARD DEFINITION

PCA as eigendecomposition (not in deck — standard definition). Center the data, compute the covariance matrix $\Sigma = \frac{1}{n-1} X^T X$, and eigendecompose it: $\Sigma v = \lambda v$. The **principal vectors (directions)** are the eigenvectors v, which are orthogonal because Σ is symmetric (W2: $A = Q \Lambda Q^T$). The **principal components** are the projected data Xv (the new coordinates). The **eigenvalue** λ is the variance of the data along v. Sorting the eigenvalues in decreasing order gives PC1, PC2, ... *Further reading:* IBM `principal-component-analysis`; DeepLearning.AI *Linear Algebra for ML and Data Science* (eigenvectors).

Mechanisms: why and how

- **Why K-Means always converges (a quiz topic):** each step can only **lower or keep** J. The assignment step picks the nearest center for each point (it cannot increase that point's distance), and the update step uses the mean, which minimizes squared distance [W7 p11]. There are only **finitely many** ways to assign N points to k clusters, and J never goes up, so the algorithm cannot cycle forever. It **must stop**, but at a **local** optimum that depends on the initialization [W7 p17, p25].
- **Why K-Means can give a bad answer:** the result depends on the initial centroids (local optimum), outliers drag the means, and the squared-distance-to-center logic assumes compact, convex blobs [W7 p25, p34].
- **Why the elbow works:** adding clusters always lowers J (with $k = N$, $J = 0$). Before the true number, each new cluster gives a big drop; after it, only small gains. The bend marks the natural k [W7 p23].
- **Why DBSCAN finds outliers and K-Means does not:** K-Means must put every point in some cluster. DBSCAN leaves points in sparse regions unassigned [W7 p34–p36].
- **Why PCA components are uncorrelated:** they are orthogonal eigenvectors of a symmetric matrix [W7 p40].
- **How a scree plot is used:** sort the eigenvalues in decreasing order, find where the curve stops dropping steeply (the leveling-off point), and keep the components **before** it. After that point, each extra component explains very little variance, which is mostly noise [W7 p42].
- **Why scale before K-Means or PCA:** both depend on distances and variances, so large-unit features would dominate [review file].

Comparisons

A	B	Key difference
Supervised	Unsupervised	labels known, predict them vs. no labels, find structure [W7 p2]
Clustering	Dimensionality reduction (PCA)	groups **instances** (rows) vs. combines **features** (columns) into fewer components
K-Means	Hierarchical	must fix k, efficient $O(tkn)$ vs. no k needed (cut the dendrogram), expensive on large data [W7 p25, p29]
K-Means	DBSCAN	spherical clusters, k preset, every point assigned vs. arbitrary shapes, no k, finds outliers, struggles with varying density and high dimensions [W7 p33–p34]
K-Means	K-modes	means and numeric data vs. modes and categorical data [W7 p24]
Agglomerative	Divisive	bottom-up merging vs. top-down splitting [W7 p29]
Single linkage	Complete linkage	closest pair vs. farthest pair; centroid linkage uses the centers [W7 p30]
Core point	Border point	$\geq$ min_samples in its ε-ball vs. inside a core point's ball but not core; an outlier is neither [W7 p35]
Hard assignment	Soft assignment	$z_{nk} \in \{0, 1\}$ vs. $z_{nk} =$ probability [W7 p26]
Eigenvector	Eigenvalue	direction / feature coefficients of a component vs. the amount of variance it explains [W7 p41]
Factor analysis	PCA	latent factors with common and unique variance, extraction + rotation vs. orthogonal components of maximum variance [W7 p39–p40]
K-Means	Mini-batch K-Means	all the data each iteration vs. small random batches, scalable [W7 p28]

W8 — Decision Trees and Random Forests

Deck W8_Decision_Trees.pdf, 23 slides. Géron Ch 6 (trees), Ch 7 (random forests). ***Outside reading (IBM / DeepLearning.AI): p. 95.***

Core definitions

Decision tree anatomy [W8 p2–p3] . A series of questions leading to a decision, drawn as an upside-down tree. **Decision node**: tests a feature. **Root node**: the first decision node. **Branch**: a connection between nodes. **Layer**: all nodes at the same distance from the root. **Leaf node**: tests no feature; this is where the decision is made. **Depth**: the number of layers. In a **binary** tree every node has two branches. The deck's example is "accept a job offer?": the root asks "another offer?", then "is the current offer better?".

Classification vs. regression trees [W8 p4–p5]. *Classification trees* categorize (fraud, show an ad?), and each leaf is a **class**. *Regression trees* estimate numbers (house prices): they split the feature space into regions and predict the **mean of each region**. They excel at **non-linear** relationships (car price vs. age, shot success vs. distance). Use cross-validation to compare them.

Regression tree vs. linear regression [W8 p6–p7]

Linear regression	Regression tree
Fits linear data better	More flexible with non-linear data
Missing values are discarded	Missing values can become a separate class
Heavily affected by outliers	Not affected by outliers
Hard to interpret graphically with many features	Easy to interpret even with many features
Statistical significance of each feature is easy to measure	Feature importance is not straightforward

CART [W8 p8] . *Classification and Regression Trees* builds trees by **recursively splitting** on numeric thresholds. Each split aims to minimize the **RSS/MSE** (regression) or the **Gini index** (classification).

CART for regression, step by step [W8 p9–p10]. (1) Choose a feature x. (2) Split the data into two regions at a threshold. (3) Compute the **RSS** in each region. (4) Try all thresholds. (5) Repeat for all features. (6) Split on the feature and threshold with the **lowest RSS**, and stop according to predefined criteria. The deck's tree splits square footage ≤ 1394 at the root (price 172.2, MSE 4255.5), and its leaves predict prices of 108.0, 153.4, 200.3, and 360.7. The overall $R^2 = 0.59$.

Depth and accuracy [W8 p11]. R^2 (the coefficient of determination) measures a regression tree's accuracy (higher is better). Deeper trees may give **marginal** R^2 gains at the cost of interpretability, so balance the two.

Overfitting control [W8 p12, p19] . Greedy RSS/Gini minimization can **overfit**. **Stopping criteria** prevent this: a minimum number of instances per leaf, or a maximum number of terminal nodes.

Classification trees [W8 p13–p15]. Divide the feature space into regions and classify by the **majority class** in the region. *Advantages*: easy to interpret, **no feature scaling needed**, fast classification. *Disadvantages*: **does not extrapolate well**, does not work well with small datasets, takes longer to

train. They handle **more than two classes** naturally (unlike plain logistic regression), but trees with many categories are hard to interpret, and rare categories may never appear as leaves.

Split quality [W8 p16] . Measured by the **Gini index, entropy, and information gain**. The **Gini index** (used in the original CART) measures misclassification: 0 means a **pure** split (one class). The **overall Gini** is the **weighted average** of the regions' Gini values, weighted by instance counts.

CART for classification, step by step [W8 p18–p19]. Same loop as for regression, but compute the Gini index from the class proportions in R_1 and R_2, and pick the feature and threshold with the **lowest** Gini.

Random forest classifier [W8 p20–p21] . Many decision trees. Each tree is trained on a **bootstrap sample** (bagging) **and** considers only a **random subset of features** ($m < M$) when splitting. This **double randomization** reduces the correlation among the trees and improves generalization. A new instance goes through all trees, and the **most frequent class** is the prediction (W8 p21: Sira, Sira, Seker $\rightarrow$ Sira).

Random forest regressor [W8 p22]. Predicts the **average** of the trees' outputs and is evaluated with MSE or RMSE.

Formulas

Gini index [W8 p16–p17]

$$\text{Gini} = \sum_{i=1}^{k} p_i(1-p_i) = 1 - \sum_{i=1}^{k} p_i^2 \qquad \text{Overall Gini} = \sum_{\text{regions } r} \frac{n_r}{n}\,\text{Gini}_r$$

IN PLAIN WORDS

In plain words. p_i = the share of class i in the node. Square the shares, add them, and subtract from 1. If one class has everything, $1 - 1^2 = 0$ (pure). An even 50/50 mix gives $1 - 0.25 - 0.25 = 0.5$, the most impure for 2 classes. For the whole split, average the regions' Gini values, weighting each by its share of the points.

WORKED EXAMPLE / MODEL ANSWER

Deck example [W8 p17]. Region A_1 has 3 Sira and 0 Cali, so Gini $= 0$. Region A_2 has 1 Sira and 3 Cali: Gini $= 1 - (\frac{1}{4})^2 - (\frac{3}{4})^2 = 1 - \frac{1}{16} - \frac{9}{16} = \frac{6}{16} = 0.375$. Overall: $\frac{3}{7}(0) + \frac{4}{7}(0.375) = 0.214$.

NOT IN DECK — STANDARD DEFINITION

Entropy and information gain (not in deck — standard definition; W8 p16 names them but gives no formula)

$$H(S) = -\sum_{i=1}^{k} p_i \log_2 p_i \qquad\qquad IG = H(\text{parent}) - \sum_{\text{children } c} \frac{n_c}{n} H(c)$$

In plain words. Entropy is the "surprise" of a node in bits: 0 for a pure node and 1 for a 50/50 two-class node. Information gain = parent impurity − the weighted average child impurity. Choose the split with the **largest** gain, which is the same idea as choosing the **lowest** weighted Gini. *Further reading:* IBM `decision-trees`; DeepLearning.AI MLS C2 W4 (information gain). A worked example on the deck's Sira/Cali data is in p. **84**.

RSS of a regression region [W8 p8–p9]: $RSS = \sum_{\text{regions } R} \sum_{i \in R} (y_i - \bar{y}_R)^2$, where the prediction $\bar{y}_R$ is the region's mean.

Mechanisms: why and how

- **Why trees need no feature scaling:** each split compares *one* feature to a threshold, so rescaling a feature just moves the threshold and the ordering of the points stays the same [W8 p13].
- **Why trees overfit:** greedy splitting can keep going until each leaf is pure, memorizing noise. Stopping criteria (minimum instances per leaf, maximum leaves) limit this [W8 p12, p19].
- **Why trees do not extrapolate:** a leaf predicts a constant (a mean or majority class), so beyond the training range the prediction stays flat [W8 p13].
- **Why trees are robust to outliers:** an outlier only affects its own region's mean, not a global line [W8 p6].
- **Why random forests beat a single tree:** averaging many decorrelated trees cancels out their individual errors, so variance falls. The randomness in the rows (bootstrap) and in the features ($m < M$) keeps the trees from all making the same mistakes [W8 p20].
- **How CART picks a split:** it tries every feature and every threshold and keeps the one with the lowest weighted RSS (regression) or Gini (classification) [W8 p9, p18].

Comparisons

A	B	Key difference
Classification tree	Regression tree	leaf = majority class, Gini vs. leaf = region mean, RSS/MSE, R^2 [W8 p4–p5, p8]
Gini index	Entropy	$1 - \sum p_i^2$ (max 0.5 for 2 classes) vs. $-\sum p_i \log_2 p_i$ (max 1 bit); they usually choose similar splits
Low Gini / high information gain	High Gini / low gain	purer children: a good split vs. mixed children: a poor split

A	B	Key difference
Decision tree	Random forest	one tree, interpretable, high variance vs. many bootstrapped trees with random features, lower variance, less interpretable [W8 p20]
Random forest	Plain bagging of trees	bootstrap **and** random feature subset at each split vs. bootstrap only (the trees are more correlated) [W8 p20; W9]
Regression tree	Linear regression	non-linear, robust to outliers vs. linear, outlier-sensitive, gives significance tests [W8 p6]

W9 — Ensemble Learning: Boosting, Bagging, Stacking

Deck W9_Ensemble_Learning.pdf, 21 slides. Géron Ch 7. ***Outside reading (IBM / DeepLearning.AI): p. 95.***

Core definitions

Ensemble modeling [W9 p2–p3]. Combines the predictions of multiple **base models** into a more robust and accurate composite model. It addresses the complexity–accuracy balance: complex models overfit, simple ones err. The hepatitis C example on p3 shows two simple one-feature models (albumin, cholesterol); where they disagree, the ensemble resolves it.

Sequential vs. parallel [W9 p4–p5] . **Sequential** (boosting): base models are fit **one after another**, updating the data and predictions each time. It takes longer but **focuses on the regions where earlier models were inaccurate**. **Parallel** (bagging, stacking): base models are fit **independently**, which saves time and can run on multiple cores or nodes, and they are combined afterwards. **Stacking** uses *different kinds* of base models plus a **meta model**.

Combining predictions [W9 p6]. **Voting** for classification (4 of 5 base models say DELAY, so 80%, so the ensemble predicts DELAY). **Averaging** for regression ($\frac{3+10+90+15+25}{5} = \frac{143}{5} = 28.6$ minutes. Note: the slide prints $\frac{158}{5} = 31.6$, but its own numbers sum to 143).

Boosting [W9 p7–p8] . Sequential. After each base model, **update the instance weights to emphasize the poorly predicted instances**, so the next model focuses on the previous model's weaknesses. Deck example (5 hepatitis C patients): model 1 ("ALB $< 32 \Rightarrow$ hepatitis") gets 4 right and 1 wrong. The correct instances 1, 3, 4, 5 get **lower** weight and instance 2 gets **higher** weight (it effectively appears 4 times). Model 2 ("cholesterol $< 4 \Rightarrow$ hepatitis") gets all of them right, so boosting stops.

AdaBoost [W9 p9–p11] . *Adaptive boosting*: a sequential technique without a fixed number of base models. It continues until the maximum number of base models is reached or the **error no longer changes**, and in practice it often stops early. Steps: (1) equal weights $w_i^{(1)} = \frac{1}{n}$; (2) fit a base model and predict; (3) compute the error; (4) **misclassified instances get higher weight** ($w_i^{(2)} \geq w_i^{(1)}$) and **correctly classified ones lower weight**; (5) repeat; (6) stop at the error threshold or the maximum number of models.

Bagging [W9 p12–p15] . **B**ootstrap **agg**regat**ing**. It fits base models **in parallel, without updating weights**, and aggregates them **after** all are fit. Algorithm: (1) draw **bootstrap samples** (with replacement); (2) fit a base model to each; (3) combine by **averaging** (numeric) or **majority vote** (categorical). The bootstrap introduces variability between the models and **improves stability and accuracy**. Deck example: patient ALB $= 39$, CHOL $= 4$; 4 of the 6 base models predict $y = 1$, so the ensemble predicts $y = 1$.

Out-of-bag (OOB) instances [W9 p16] . Each bootstrap sample leaves some instances out. These **OOB** instances evaluate the base models and estimate the ensemble's prediction error, like a free validation set. Deck example: dataset $\{1,\ldots,10\}$. Bootstrap sample 1 = 9 9 2 6 9 8 2 1 7 2 has 6 unique values, and the slide lists its OOB set as {3, 4, 7, 10} (#OOB $= 4$). *Sample 2 = 2 2 9 6 4 7 9 10 9 1 has 7*

unique values, OOB = {3, 5, 8}. Sample 3 = 8 2 2 5 3 9 7 1 6 10 has 9 unique values, OOB = {4}. Counting sample 1 by hand, it also lacks 5, and 7 *is* drawn. The slide's set looks slightly off. The rule is what matters: **OOB = the instances never drawn**.

Stacking [W9 p17–p20] . Combines **diverse** base models through a **meta model**. The base models train on the **entire dataset with all features**: diversity comes from the different algorithms, not from bootstrapping or random features. A **new dataset** (inputs + base-model predictions) trains the meta model: **logistic regression for classification, linear regression for regression**. The meta model **smooths the final decision boundary**, and one base model's weakness is compensated by another's strength. The base models must all suit the same task (all classifiers, such as SVM, or all regressors, such as linear regression). Example: k-NN regression, linear regression, and quadratic regression are stacked; decision tree, SVM, and logistic regression are stacked.

Formulas

AdaBoost weight update [W9 p11]

$$\epsilon_1 = \frac{\#\text{misclassified}}{n} \text{ (weighted error)} \qquad \text{misclassified: } w \times \sqrt{\frac{1-\epsilon_1}{\epsilon_1}} \qquad \text{correct: } w \times \frac{1}{\sqrt{\frac{1-\epsilon_1}{\epsilon_1}}}$$

then **rescale** so the weights sum to 1.

WORKED EXAMPLE / MODEL ANSWER

Deck numbers [W9 p11]. There are 5 instances and only instance 2 is wrong, so $\epsilon_1 = \frac{1}{5} = 0.2$. Factor $= \sqrt{\frac{1-0.2}{0.2}} = \sqrt{4} = 2$. Instance 2: its relative weight becomes 2; the others become $\frac{1}{2}$. New weights $\frac{1}{2}, 2, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}$ sum to 4. Scaled: $\frac{1}{8}, \frac{4}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{8}$. **The misclassified point now carries half of the total weight.**

IN PLAIN WORDS

In plain words. The better the model (ϵ small), the bigger the factor $\sqrt{(1-\epsilon)/\epsilon}$, so its mistakes are boosted more strongly. If $\epsilon = 0.5$ (a coin flip), the factor is $\sqrt{1} = 1$ and nothing changes.

Combining [W9 p6, p14]: classification $\hat{y} = \text{mode}(\hat{y}^{(1)}, \ldots, \hat{y}^{(B)})$; regression $\hat{y} = \frac{1}{B}\sum_{b=1}^{B} \hat{y}^{(b)}$.

Mechanisms: why and how

- **Why ensembles work:** independent errors partly cancel out when you vote or average, so the combination is more stable than any single model [W9 p2].
- **Why bagging reduces variance:** each model sees a slightly different bootstrap sample, so their overfitting quirks differ, and averaging smooths those quirks out [W9 p14].
- **Why boosting reduces bias:** each new model concentrates on the examples the current ensemble gets wrong, gradually fitting the hard regions [W9 p7]. *(The labels "bagging → variance, boosting → bias" are standard; the deck gives the mechanisms.)*
- **Why boosting cannot be parallelized:** model $t+1$ needs the weights produced by model t [W9 p4].

- **Why OOB error is "free" validation:** OOB points were never used to train that base model, so predicting them is an honest test [W9 p16].
- **Why stacking uses a meta model instead of a simple vote:** the meta model *learns* how much to trust each base model in each region, which smooths the decision boundary [W9 p18].

Comparisons

	Boosting	**Bagging**	**Stacking**
Fitting	sequential	parallel	parallel
Data for each base model	reweighted data (emphasizes errors)	bootstrap samples	the entire dataset, all features
Base models	usually the same type (weak learners)	usually the same type (e.g. trees)	**different** algorithms
Combine by	weighted sequence	vote / average	**meta model** (logistic or linear regression)
Updates weights?	**yes**, after each model	no	no
Speed	slower	fast (parallel)	moderate
Main effect	lowers bias (focuses on errors)	lowers variance (stability)	exploits complementary strengths
Deck pages	W9 p7–p11	W9 p12–p16	W9 p17–p20

A	**B**	**Key difference**
Bagging	Random forest	bootstrap only vs. bootstrap + random feature subset per split [W8 p20]
Voting	Averaging	classification vs. regression [W9 p6]
OOB error	Cross-validation	free by-product of bootstrapping vs. explicit fold splitting [W9 p16; W5 p23]

W10 — Recommender Systems

Deck W10_Recom_Systems.pdf, 65 slides. ***Not covered in Géron****, so this section is fully self-contained.* ***Outside reading (IBM / DeepLearning.AI): p. 95.***

Core definitions

Recommendation system [W10 p2]. An algorithm that suggests **relevant items** to users: things to buy, watch, or consume. Goals: **find items useful or entertaining to the user**, and **keep the user engaged on the platform** (YouTube, Amazon, TikTok, Spotify).

History [W10 p3]. **Grundy**: interviewed users to form groups, then recommended the same book to each group. **Tapestry (Xerox)**: users commented on documents collaboratively and got thematically relevant ones recommended (the start of collaborative filtering). **Salton**: vector representations of items, retrieved with a similarity statistic against past choices (the start of content-based filtering). **CineMatch (Netflix)**: a successful **hybrid** that later combined 170 child algorithms.

The four types [W10 p4–p11]

Type	Idea (deck wording)	Good when	Typical tool
Content-based	"Recommend items similar to what the user chose / liked in the past"	the user's own preferences largely determine purchases (food shopping, customized searches)	item feature vectors + similarity (Salton)
Collaborative filtering	"People who agreed in the past are likely to agree in the future": if A and B had similar taste, B's preferences predict A's	lots of user–item interaction data	user–item rating matrix; matrix factorization; kNN; naive Bayes
Knowledge-based	"Use domain knowledge to retrieve items that match user needs" ("if user needs X recommend Y"); query and infer the user's needs	**sparse interactions** and **complex item features** (buying cars, renting apartments)	rules, questionnaires
Hybrid	combines the designs above; **most common in practice**	real platforms	ensembles (CineMatch)

Collaborative filtering gets its item descriptions from users, either **explicitly** (labels, as in Tapestry) or **implicitly** (items are similar if the same people buy them) [W10 p8].

Vectorized representation [W10 p5]. Describe an item or user with a vector, one entry per chosen feature: **binary** for qualitative features, **real** for quantitative ones. A convenient way to capture multidimensionality.

Content-based recipe [W10 p7]. Item vector: 1 if it has the feature, else 0. **User preference vector = average of the items the user viewed in the past.** Recommend the items most similar to it (dot product).

Matrix factorization [W10 p9] . Put the ratings in a matrix M with M_{ij} = user i's rating of item j (many entries are missing). Find a d-length vector for each user and each item so that $\langle user_i, item_j \rangle \approx M_{ij}$. *Intuition*: the user vector holds preferences, the item vector holds features, and their dot product measures the match. The predicted ratings fill in the blanks.

Problems [W10 p13–p16]

Problem	What goes wrong	Deck's potential solutions
Serendipity	cat video → more cat videos → . . . repetitive, never explores new interests	look at how *similar users shifted* interests; domain knowledge ("if user needs X, they will likely need Y later")
Cold start	a new user, or naturally sparse interactions (buying cars), gives no informative history	**query the user** (import contacts at signup; ask what size of car or how many seats)
Feature extraction	manual labeling is infeasible with too many items, or items such as images and video that a few labels cannot describe	**neural methods that learn features** from the data
Scalability	millions or billions of users and items; pairwise metrics are infeasible	efficient methods
Large datasets	lots of possibly unstructured data	
Sparse ratings	confident recommendations for items with little information	
Lack of user profiles	bootstrapping CF without a developed user ecosystem	

Deck question [**W10 p12**]: a Coinbase recommender for cryptocurrencies, based on risk appetite, retirement date, preferred industries, **and** coins the user has had success with in the past. *Suggested answer (reasoning, not on the slide):* **hybrid**. The needs-based factors (risk, retirement horizon, preferred domains) call for **knowledge-based** recommendation, and "coins they had success with" adds a **content-based / collaborative** history signal. If only one type is allowed, choose knowledge-based, because the request is driven by stated needs and constraints.

Bag of words (BoW) [W10 p22] . Represent a text as a vector as long as the vocabulary, where each entry counts a word's occurrences. *Problems:* **all words are considered equally important**, and **word order is ignored**.

TF-IDF [W10 p23–p27] . *Intuition: rare words are more important when they appear.* The inverse document frequency **lowers the weight of terms that occur in almost all documents** and raises it for rare ones. Each document becomes a TF-IDF vector, and cosine similarity retrieves the top matches. Assumption: the distribution of terms represents the distribution of topics. **IDF keeps stop words ("the", "is") from dominating.**

Learned word embeddings [W10 p28–p30] . "A learned representation for text where words that have the same meaning have a similar representation," induced by training on an auxiliary task. **Embedding matrix** E: $N \times d$ (N = vocabulary size, d = dimension, e.g. 10,000 words × 300

features). A **one-hot vector times E retrieves that word's row** (the deck's example picks row 4 = [10 12 19]).

Word2Vec [W10 p31–p36] . **Skip-gram**: given the **center word, predict the distribution of the context words**. It models $P(w \in \text{context} \mid \text{center})$. Take the dot product of the center embedding with every candidate context word's vector, apply a softmax, and optimize to match the corpus. In matrix form, multiply by E^T: "this is just an output layer + classification head" (a network with 10,000 inputs, a 300-neuron linear hidden layer, and a 10,000-way softmax). **CBOW (continuous bag of words)**: given the **context words, predict the center word**. It models $P(\text{center} = w \mid \text{context})$: the **sum** of the context embeddings should be close to the center embedding, again with a softmax and E^T.

GloVe [W10 p37–p38]. Learn word vectors such that **their dot product equals the log of the words' co-occurrence probability**. Build the $N \times N$ co-occurrence matrix and factor it into EE^T, using gradient descent. Deck table: $P(k \mid ice)$ is large for k = solid (1.9×10^{-4}); $P(k \mid steam)$ is large for k = gas (7.8×10^{-4}); both are similar for water and fashion. **Ratios** of co-occurrence probabilities reveal meaning.

Using embeddings for recommendation [W10 p39]. Run an item's text description through an NLP model (RNN, LSTM, Transformer) to get an **item feature vector** automatically. This solves the feature-extraction problem.

Collaborative filtering formal setup [W10 p47–p50] . Users $U = \{1, \dots, u\}$, items $I = \{1, \dots, m\}$, training set T = the observed ratings r_{ui}. **Goal: predict the unknown pairs, i.e. fill in the user–item matrix R.** Big idea: Emi likes Apple, Banana, Pear; Andre likes Apple, Banana, so Andre will probably like Pear. To predict for user u: find N users similar to u rating-wise, and predict from their ratings. The two flavors are **user-based** and **item-based**.

CF pipeline [W10 p51]. *Data preparation*: **remove the bias term** (e.g. subtract each user's mean rating) and **shrinkage**. *Classifier model*: naive Bayes or kNN. *Measure success*: **RMSE**.

CF as classification/regression [W10 p52–p54]. Treat CF as **one classification or regression problem per item i**. Each user is a rating vector over all the *other* items (with missing values), and the rating $r_{ui} \in \{1, 2, 3, 4, 5\}$ is the class (stars). Train a separate predictor (kNN or naive Bayes) for each item.

Naive Bayes for CF [W10 p55–p57] . Assumptions: ratings are a discrete set {1,…,5}, and **user u's rating r_{ui} is independent of u's other ratings** r_{uj} (the "naive" part). Train on the users who already rated item i: estimate the **prior** $P(r_i = v)$ and the **likelihood** $P(r_j = w \mid r_i = v)$, then predict the **posterior**.

kNN for CF [W10 p58–p62] . "Commonly used method for CF!!" Classical kNN measures *distance*; CF kNN measures **similarity** and takes the K most similar users. **The vectors need to be complete. Solution: only look at the items rated by both users.**

Formulas

Dot product and cosine similarity [W10 p6, p18–p19]

$$\langle v, u \rangle = \sum_{i=1}^{d} v_i u_i = |v||u| \cos\theta \qquad \cos\theta = \frac{\langle v, u \rangle}{|v||u|}$$

IN PLAIN WORDS

In plain words. With **binary** vectors, the dot product just **counts the matching features** [W10 p18]. Example [W10 p17]: a user wants educational + casual + science, so App 1 scores 2, App 2 scores 1, App 3 scores 0, and we recommend App 1. *Problem*: an item with **all** features always wins. **Cosine** fixes this by dividing by the lengths, so it **penalizes "un-specific" items**, and irrelevant features hurt the score. It measures only the **angle** (direction), from -1 to 1 (0 to 1 for non-negative vectors).

WORKED EXAMPLE / MODEL ANSWER

Hand example. User $u = (1, 1, 0)$. Item A $= (1, 1, 0)$: dot $= 2$, cos $= \frac{2}{\sqrt{2}\sqrt{2}} = 1$. Item B $= (1, 1, 1)$: dot $= 2$, cos $= \frac{2}{\sqrt{2}\sqrt{3}} \approx 0.82$. The dot product ties them; cosine prefers the **specific** item A.

TF, IDF, TF-IDF [W10 p23–p26]

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \qquad \text{tf}(t, d) = 0.5 + 0.5 \cdot \frac{f_{t,d}}{\max\{f_{t',d} : t' \in d\}}$$

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|} \qquad \text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D)$$

IN PLAIN WORDS

In plain words. TF = how often the term appears in *this* document ÷ the document's length (its relative importance in the document). The variation divides by the most frequent term's count and adds 0.5 for smoothing, so that a small change in count does not cause a large change in tf and doubling the frequency does not double the importance. **IDF** = log(total documents ÷ documents containing the term). A word in **every** document has $\log(N/N) = \log 1 = 0$, so it gets weight 0. A word in only 1 of 100 documents gets $\log 100$, a large weight. *Why log?* "Information-theoretic reasons": a word's specificity grows **sublinearly** with its inverse occurrence count.

WORKED EXAMPLE / MODEL ANSWER

Hand example. $N = 4$ documents. "data" appears in all 4, so idf $= \log(4/4) = 0$. "recommender" appears in 1, so idf $= \log 4$. In a 10-word document containing "recommender" twice: tf $= 0.2$, so tfidf $= 0.2 \log 4$ (≈ 0.28 with the natural log, 0.4 with $\log_2$).

Similarity and distance measures [W10 p41–p46, p61]

Measure	Formula	Treats vectors as	Notes
Euclidean	$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$	points	sensitive to magnitude: finds words similar in meaning **and frequency** [W10 p43]

Measure	Formula	Treats vectors as	Notes
Manhattan	$d(x,y)=\sum_i \lvert x_i-y_i\rvert$	points	city-block distance [W10 p44]
Cosine	$\cos\theta=\frac{\langle v,u\rangle}{\lvert v\rvert\lvert u\rvert}$	directions	**ignores magnitude**: words similar in meaning [W10 p43]
Dice	$DSC=\frac{2\lvert X\cap Y\rvert}{\lvert X\rvert+\lvert Y\rvert}$; binary: $\frac{2\,a\cdot b}{\lVert a\rVert^2+\lVert b\rVert^2}$	sets	[W10 p45]
Jaccard	$J(A,B)=\frac{\lvert A\cap B\rvert}{\lvert A\cup B\rvert}$; binary: $\frac{A\cdot B}{\lVert A\rVert^2+\lVert B\rVert^2-A\cdot B}$	sets	intersection over union [W10 p46]
Pearson (CF)	$\mathrm{sim}(x,y)=\frac{\sum_{s\in S_{xy}}(r_{xs}-\bar r_x)(r_{ys}-\bar r_y)}{\sqrt{\sum_{s\in S_{xy}}(r_{xs}-\bar r_x)^2}\sqrt{\sum_{s\in S_{xy}}(r_{ys}-\bar r_y)^2}}$	mean-centered ratings	S_{xy} = the items **both** users rated; removes each user's rating bias [W10 p61]

WORKED EXAMPLE / MODEL ANSWER

Sets example. $X=\{\text{Apple, Banana, Pear}\}$ (Emi), $Y=\{\text{Apple, Banana}\}$ (Andre). Intersection $=2$, union $=3$. Jaccard $=\frac{2}{3}\approx 0.67$. Dice $=\frac{2\cdot 2}{3+2}=0.8$. (Dice is always $\geq$ Jaccard.)

Why the dot product fails for GloVe word similarity [W10 p42]: an infrequent word may be most similar to another infrequent word, but the dot product (dominated by magnitude) never retrieves it. **Solution: Euclidean distance or cosine similarity.**

Naive Bayes CF formulas [W10 p57]

$$P(r_i=v)=\frac{\sum_{u=1}^{U}\mathbf{1}(r_{ui}=v)}{\sum_{w=1}^{V}\sum_{u=1}^{U}\mathbf{1}(r_{ui}=w)} \qquad P(r_j=w\mid r_i=v)=\frac{\sum_{u=1}^{U}\mathbf{1}(r_{ui}=v,\,r_{uj}=w)}{\sum_{z=1}^{V}\sum_{u=1}^{U}\mathbf{1}(r_{ui}=v,\,r_{uj}=z)}$$

$$P(r_{ui}=v\mid r_{u1},\dots,r_{uM})=\frac{P(r_{ui}=v)\prod_{j\neq i}P(r_{uj}\mid r_{ui}=v)}{\sum_{w=1}^{V}P(r_{ui}=w)\prod_{j\neq i}P(r_{uj}\mid r_{ui}=w)}$$

IN PLAIN WORDS

In plain words. Prior: among the users who rated item i, what fraction gave it v stars? **Likelihood**: among the users who gave item i v stars, what fraction gave item j w stars? **Posterior**: for each possible star value v, multiply prior $\times$ all likelihoods, then divide by the sum over all v so they add up to 1. Predict the v with the largest posterior. The $\mathbf{1}(\cdot)$ just counts the users who match.

Measuring success [W10 p64]

$$RMSE=\sqrt{\frac{1}{|Q|}\sum_{(u,i)\in Q}(r_{ui}-\hat r_{ui})^2} \qquad MAE=\frac{1}{|Q|}\sum_{(u,i)\in Q}|r_{ui}-\hat r_{ui}|$$

Q = the set of test user–item pairs. These are the same RMSE and MAE as in W5, applied to the missing ratings.

W10

Mechanisms: why and how

- **Why content-based systems lack serendipity:** they recommend only what resembles the past, so a user's world shrinks into a loop [W10 p13].
- **Why collaborative filtering has a cold-start problem:** it needs rating history. A new user or item has none, so there is no one to compare with [W10 p14].
- **Why knowledge-based systems suit cars and apartments:** purchases are rare (sparse interactions) and the items are complex, so explicit needs and constraints work better than history [W10 p10].
- **Why cosine beats the raw dot product:** dividing by the lengths stops "has-every-feature" items from always winning [W10 p18–p19].
- **Why IDF uses a log:** importance should grow slowly (sublinearly) with rarity, and the log also makes very common words weigh exactly 0 [W10 p26].
- **Why embeddings beat bag of words:** BoW treats all words as equal and unrelated ("good" and "great" are as different as "good" and "car"). Embeddings place similar words close together [W10 p22, p29].
- **Why Pearson similarity mean-centers:** some users rate everything high, so subtracting each user's mean compares *taste*, not generosity (the "remove bias" step) [W10 p51, p61].
- **Why CF kNN uses only co-rated items:** the similarity formulas need complete vectors, and ratings are missing for most items [W10 p62].

Comparisons

A	B	Key difference
Content-based	Collaborative filtering	similarity between **items** (features) vs. similarity between **users'** behavior; CF needs no item features but suffers from cold start
Collaborative	Knowledge-based	learns from interaction history vs. uses domain rules and a query of user needs (sparse, complex items)
User-based CF	Item-based CF	find users similar to u vs. find items similar to i [W10 p49]
Dot product	Cosine similarity	rewards magnitude (all-feature items win) vs. angle only, penalizes un-specific items [W10 p18–p19]
Cosine	Euclidean	ignores magnitude (similar meaning) vs. sensitive to magnitude (similar meaning *and* frequency) [W10 p43]
Jaccard	Dice	$\frac{\lvert A\cap B\rvert}{\lvert A\cup B\rvert}$ vs. $\frac{2\lvert A\cap B\rvert}{\lvert A\rvert+\lvert B\rvert}$ [W10 p45–p46]
Bag of words	TF-IDF	raw counts, all words equal vs. counts weighted by rarity across documents [W10 p22–p23]
TF	IDF	importance *within* one document vs. rarity *across* the corpus [W10 p24, p26]

A	B	Key difference
Skip-gram	CBOW	center → predict context, $P(\text{context} \mid \text{center})$ vs. context → predict center, $P(\text{center} \mid \text{context})$ [W10 p31–p36]
Word2Vec	GloVe	local context-window prediction vs. factorizing the global co-occurrence matrix [W10 p37–p38; W12 p17]
Matrix factorization	kNN CF	learns latent user/item vectors whose dot product ≈ rating vs. averages the ratings of similar users [W10 p9, p59]

W11 — Deep Learning

Deck W11_Deep_Learning.pdf, ***new 30-slide version*** *(Canvas, Nov 10, 2026). Géron Ch 10–11, 14, 15, 17.*
Outside reading (IBM / DeepLearning.AI): p. 96.

Core definitions

Biological neuron [W11 p2]. A nerve cell with a **cell body** (the nucleus: genetic info, function, energy), an **axon** (a long, thin part that transmits electrical signals), and **dendrites** (receive chemicals and decide on electrical signals).

Artificial neuron [W11 p3] . Takes inputs, computes a **weighted sum**, and compares it to a threshold with a **threshold function** (e.g. the unit step): **output 1 if sum $\geq$ threshold, else 0**. Deck example, "go to the beach?": $x_1 =$ sunny $(0/1)$ with $w_1 = 5$, $x_2 =$ miles to the beach with $w_2 = -0.2$. For a sunny day 6 miles away: $5(1) + (-0.2)(6) = 3.8 \geq 0$, so the output is 1 (go).

Artificial neural network (ANN) [W11 p4]. Layers of neurons: the **input layer** receives the features, the **hidden layer(s)** apply weights and produce numeric outputs, and the **output layer** produces the class or value.

Single-layer perceptron [W11 p5–p6] . A network for **binary classification**. Components: inputs, weights, weighted sum, activation function, output. "Single-layer" means **inputs connect directly to the output** (no hidden layers). **Bias** improves accuracy: it is a constant input 1 with weight b. Geometrically [W11 p6], without bias the boundary $w_1x_1 + w_2x_2 = 0$ **must pass through the origin**; with bias, $w_1x_1 + w_2x_2 + b = 0$ **can shift** and separate the classes.

Activation function [W11 p7, p9] . Gives the perceptron's output. The most common for a perceptron is the **unit step**. Others: **sigmoid** (0 to 1, value 0.5 at 0), **tanh** (-1 to 1, 0 at 0), **ReLU** $\max\{0, z\}$, and **leaky ReLU** $\max\{0.1z, z\}$.

Training basics [W11 p7]. Training minimizes the **average training error** by **iteratively updating the weights**. An **epoch** is one round of weight updates over **all** the training data. **Learning rate**: lower rates need **more epochs**; higher rates need fewer but **may give suboptimal weights**.

Why non-linear activations [W11 p8] . A perceptron handles **linear decision boundaries** only. For complex data, use **multi-layer networks with non-linear activations**. *(Standard addition: stacking only linear layers is still one linear function, so without non-linearity, depth adds nothing.)*

Which activation to use [W11 p10]. It depends on the task (regression or classification) and the network type: **unit step for single-layer perceptrons, ReLU for multilayer networks, tanh for recurrent networks such as LSTM** (sequences).

Multilayer perceptron (MLP) [W11 p11–p12] . A single-layer perceptron is a **feed-forward** network: there are **no cyclic connections**. MLPs have hidden layers and suit **regression and classification**; single-layer perceptrons are mainly for classification. Training has two steps. **Forward propagation** feeds the inputs through the network and computes the error with a loss function.

Example [W11 p12]: 5 beds and 2,896 sq ft give a prediction of $285,600, with loss $L = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$. **Backpropagation** computes the partial derivative of the loss with respect to **each weight** and updates the weights to reduce the error.

Loss function [W11 p13]. Measures the network's error and guides the weight updates. **Hinge or log loss for classification; mean squared or mean absolute error for regression.** A **loss curve** tracks the error during training and reveals under- or overfitting. Ideally it shows an **overall decreasing trend with occasional fluctuations**.

Loss curves [W11 p14] . **Underfitted** (MLP with 1 hidden layer of 1 neuron): training and validation loss are both high and decrease slowly, in parallel. **Overfitted** (2 hidden layers of 3 and 2 neurons, learning rate 0.1): validation loss **turns upward** while training loss stays low. **Optimal** (same architecture, learning rate 0.001): both drop quickly and level off together.

Handwritten digits (MNIST) [W11 p15]. A 28×28-pixel image is flattened into a 784-value vector and fed through hidden layers to 10 output neurons (digits 0–9). The output probabilities are, for example, 0.999 for "0", 0.000 for "1", and 0.001 for "9".

Training cycle [W11 p16] . Training data → network → **forward propagation** (compute $\hat{y}$) → loss $L(y, \hat{y})$ → **backpropagation** (use L to update the weights) → repeat until the **minimum loss OR the maximum number of iterations**.

Dropout [W11 p17] . "Improving training performance": during training, randomly switch off (drop) some neurons and their connections. *(Purpose, standard: the network cannot rely on any single neuron, which reduces overfitting.)*

Gradient descent [W11 p18] . Picture the loss as a bowl over a weight. If the slope $\frac{\partial L}{\partial w_i} < 0$, the update **increases** w_i; if > 0, it **decreases** w_i; at $= 0$, L is minimized.

Gradient descent variations [W11 p19] . **Momentum** adds a third term: the previous weight change $\Delta w_i = w_i - w_{i-1}$, weighted by α. **AdaGrad** lets the **learning rate ν change for each connection**. **Adam** (adaptive moment estimation) uses the sum of the gradients (**first moment**) and the sum of the squared gradients (**second moment**), with parameters β_1, β_2. "**Adam generally outperforms other optimization algorithms.**"

Chain rule [W11 p20–p21] . The gradient depends on the loss function **and** the activation functions. Backpropagation computes gradients with the chain rule, multiplying the local derivatives along the path from the loss back to the weight.

Vanishing vs. exploding gradients [W11 p22] . On a scale of $|\partial L/\partial w_i|$ from 0 to ∞: **vanishing** means the gradients approach 0, so the early layers stop learning. With the **sigmoid**, the derivative is small (maximum 0.25) and almost 0 for large $|z|$. The slide's remedy: **batch normalization** keeps z in the steep middle region (and a linear activation has a constant derivative). **Exploding** means the gradients grow huge, which makes training unstable. The slide's picture shows a very deep, densely connected network being **simplified into a smaller one**. *(Standard remedies: ReLU, careful initialization, batch norm, gradient clipping, residual connections, LSTM gates.)*

Batch, mini-batch, stochastic GD [W11 p23] . **(Full-batch) gradient descent**: forward and backprop on **all n samples**, then one update; **slow for large n**. **Mini-batch GD**: split the data into b batches and update after each batch, so there are b **iterations per epoch; faster**. **Stochastic GD**

(SGD): update after **every single sample**, so there are n **iterations per epoch; slower** per epoch and noisier.

Discriminative vs. generative ML [W11 p24] . **Discriminative models** make predictions or classifications using **decision boundaries estimated from the training data**. Example: **support vector models**. **Generative models generate new data** from **probability distributions estimated from the training data**. Examples: **some unsupervised models, naive Bayes classifiers, and large language models**. The deck's figure: discriminative = a line between the classes; generative = a distribution blob around each class.

Architectures [W11 p25–p29]

Architecture	Pipeline (deck)	Advantages	Disadvantages
CNN [p25]	original input → **convolution → pooling → flattening** → MLP	works well for image classification and object detection; generalizes well to new data; captures high-level features (colors, objects) in the convolutional layers	does not work well for sequential data; requires a large training set; convolution layers are hard to interpret
RNN [p26]	hidden layers with a **loop**; unrolled: $x_{t-1} \to h_{t-1} \to \hat{y}_{t-1}$, $x_t \to h_t$, … with weights **shared across time**	models sequences with dependence (text, time series); takes input sequences of any length; repeated weights mean fewer parameters	susceptible to **vanishing and exploding gradients** (slow or poor training); may memorize past data instead of learning patterns; processes one input at a time, so it is computationally expensive
Autoencoder [p27]	original inputs → **encoder → latent space (bottleneck) → decoder** → re-created inputs	useful for **unsupervised** learning; works well with large or noisy datasets	computationally expensive vs. other unsupervised algorithms; the **latent space is not interpretable**
Transformer [p28]	encoder block (attention + MLP) and decoder block (attention, attention, MLP). Translation example: "I like the color blue." → "Ich mag die Farbe ___" → "blau"; attention weights such as [0.2, 0.1, 0.05, 0.25, 0.4]	**more efficient training than RNNs and autoencoders**; processes multiple parts of a sequence **in parallel**; many pre-trained models are available	sensitive to the quality and amount of training data; computationally intensive; **prone to hallucination** (inaccurate output)
GAN [p29]	**generator network** creates instances; the **discriminator network** judges real vs. generated (e.g. "Prediction: Generated")	generates new images or data based on the training data; works with **unlabeled** data	computationally expensive (**two networks**); prone to overfitting the training data

Formulas

Neuron / unit step [W11 p7]

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b \qquad u(z) = \begin{cases} 1, & z \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Perceptron learning rule [W11 p7]

$$\text{weight} = \text{weight} + \text{learning_rate} \times \text{error} \times \text{input}, \qquad \text{error} = \text{actual_class} - \text{predicted_class}$$

Equivalently, in the minus form used everywhere else: $w_j \leftarrow w_j - \eta\,(\hat{y} - y)\,x_j$, and the bias: $b \leftarrow b - \eta(\hat{y} - y)$.

IN PLAIN WORDS

In plain words. If the prediction is right, the error is 0 and **nothing changes**. If you predicted 0 but the truth is 1 (error $= +1$), **add** $\eta\times$ input to each weight, which raises z next time. If you predicted 1 but the truth is 0 (error $= -1$), **subtract**. Inputs that were 0 do not change their weight. A full numeric step is in p. **85**.

Gradient-descent / backprop update (MINUS sign is standard) [W11 p18; correcting W11 p11]

$$w_i^* = w_i - \nu \frac{\partial L}{\partial w_i}$$

DECK ERROR / INCONSISTENCY

W11 p11 prints weight $=$ weight $+ r\frac{\partial L}{\partial w}$ with a **plus**, which is likely a slide typo. The slide's own p18 uses a minus. Write the minus form, and note the discrepancy if asked.

Momentum [W11 p19]: $w_{i+1} = w_i - \nu\frac{\partial L}{\partial w_i} + \alpha\Delta w_i$, with $\Delta w_i = w_i - w_{i-1}$.

IN PLAIN WORDS

In plain words. Keep part of the last step's movement, like a ball rolling downhill that builds speed. This speeds things up along consistent directions and damps zig-zagging.

Activation functions and derivatives [W11 p9, p20]

Activation	$f(z)$	$f'(z) = \partial f/\partial z$
Sigmoid	$\frac{1}{1+\exp(-z)}$	$\frac{\exp(-z)}{(1+\exp(-z))^2}$, which equals $f(z)(1 - f(z))$; **max 0.25 at** $z = 0$
Linear	az	a
ReLU	z if $z \geq 0$; 0 if $z < 0$	1 if $z > 0$; 0 if $z < 0$; undefined at $z = 0$
Leaky ReLU	z if $z \geq 0$; cz if $z < 0$	1 if $z > 0$; c if $z < 0$; undefined at $z = 0$
tanh	$\frac{\exp(2z)-1}{\exp(2z)+1}$	$\frac{4\exp(2z)}{(\exp(2z)+1)^2}$, which equals $1 - \tanh^2(z)$

(The identities $f(1 - f)$ and $1 - \tanh^2$ are standard algebra, not from the deck.)

Chain rule [W11 p21]

$$h(x) = f(g(x)) \;\Rightarrow\; h'(x) = f'(g(x))\,g'(x)$$

Deck example: the output is $\hat{y}_i = w_0 + w_1 z_1 + w_2 z_2 + w_3 z_3$ and the loss is $L(\hat{y}_i) = (y_i - \hat{y}_i)^2$. Take $f(\hat{y}) = (y_i - \hat{y}_i)^2$, so $f'(\hat{y}_i) = -2(y_i - \hat{y}_i)$. Take $g(w_1) = \hat{y}_i$, so $g'(w_1) = z_1$. Then

$$\frac{\partial L}{\partial w_1} = f'(g(w_1))\, g'(w_1) = -2(y_i - \hat{y}_i)\, z_1$$

IN PLAIN WORDS

In plain words. To find how the loss reacts to a weight, multiply "how the loss reacts to the output" by "how the output reacts to the weight". If the network under-predicts ($y > \hat{y}$), the gradient for w_1 is negative (when $z_1 > 0$). The minus-sign update then *increases* w_1, which raises $\hat{y}$. Exactly right.

Mechanisms: why and how

- **Why bias matters:** without b, the decision boundary must go through the origin; b lets it shift [W11 p6].
- **Why we need non-linear activations:** with only linear pieces, the whole network is one linear model and can only draw straight boundaries [W11 p8].
- **How backprop trains:** forward pass → loss → chain rule to get $\partial L/\partial w$ for every weight → step **against** the gradient → repeat over epochs [W11 p11, p16, p21].
- **Why gradients vanish with sigmoid:** backprop multiplies one derivative per layer, and each sigmoid derivative is ≤ 0.25. After 10 layers the factor is at most $0.25^{10} \approx 10^{-6}$, so the early layers barely learn. Exploding is the mirror image: factors > 1 multiply into huge values [W11 p22]. See the figure in the gallery.
- **Why RNNs especially suffer:** the same weights are multiplied through every time step, so long sequences mean long products [W11 p26; W12 p27].
- **Why mini-batch is the usual compromise:** it is more stable than single-sample SGD and much faster per update than full-batch GD [W11 p23].
- **Why the learning rate changes the loss curve:** in the deck, the same network overfit at learning rate 0.1 and was optimal at 0.001 [W11 p14].
- **How to read a loss curve:** both curves high and parallel = underfit (make the model bigger or train longer). Validation rising while training falls = overfit (stop early, use dropout, regularize, get more data). Both low and flat together = good [W11 p13–p14, p17].
- **Why a GAN needs two networks:** the generator learns to fool the discriminator, and the discriminator learns to catch it. This competition improves the generated data [W11 p29].

Comparisons

A	B	Key difference
Single-layer perceptron	Multilayer perceptron	input → output directly, linear boundary, classification vs. hidden layers, non-linear, regression or classification [W11 p5, p11]
Forward propagation	Backpropagation	compute $\hat{y}$ and the loss vs. compute $\partial L/\partial w$ and update the weights [W11 p11]

A	B	Key difference
Mini-batch GD	**Stochastic GD**	update per batch of n_b samples, b iterations per epoch, faster vs. update per single sample, n iterations per epoch, slower and noisier; full-batch GD: one update per epoch, slow for large n [W11 p23]
Vanishing gradients	**Exploding gradients**	$\lvert\partial L/\partial w\rvert \to 0$, early layers freeze (sigmoid; fix: batch norm, ReLU) vs. $\to \infty$, unstable (fix: simplify the network, clipping) [W11 p22]
Discriminative	**Generative**	learns the **decision boundary** / $P(y \mid x)$ (SVM, logistic regression) vs. learns the **data distribution**, can generate new data (naive Bayes, some unsupervised models, LLMs, GANs) [W11 p24]
Sigmoid	ReLU	output (0,1), derivative ≤ 0.25 (vanishing) vs. output $[0, \infty)$, derivative 1 for $z > 0$ (the default for MLPs) [W11 p9–p10, p20]
Sigmoid	tanh	range (0,1), centered at 0.5 vs. range $(-1,1)$, centered at 0 (used in LSTM) [W11 p9–p10]
Momentum	Adam	adds a fraction of the previous step vs. adapts each step from the first and second moments of the gradients; generally best [W11 p19]
CNN	RNN	grid/image data, convolution + pooling vs. sequential data, recurrent loop with shared weights [W11 p25–p26]
RNN	Transformer	one token at a time, vanishing gradients vs. the whole sequence in parallel with attention [W11 p26, p28; W12 p29]
Autoencoder	GAN	compress $\to$ reconstruct (latent space) vs. generator vs. discriminator game [W11 p27, p29]
Underfit loss curve	Overfit loss curve	both losses high vs. validation loss rises while training loss falls [W11 p14]
Epoch	Iteration	one pass over all the data vs. one weight update (per batch or sample) [W11 p7, p23]

W12 — Intro to NLP (incl. LDA = Latent Dirichlet Allocation)

Deck W12_Intro_to_NLP.pdf, ***re-posted 36-slide version*** *(Nov 5, 2026). Géron Ch 16 covers RNN/attention/Transformers; the* ***basics and LDA are not in Géron****, so this section is self-contained.* ***Outside reading (IBM / DeepLearning.AI): p. 96.***

Core definitions

Natural language [W12 p2]. (Ralph Grishman) A primary, natural mode of human communication; the representation of most recorded human knowledge; very rich and flexible compared with formal representations.

NLP [W12 p3–p4]. (Christopher Manning) The goal is for computers to process or "understand" natural language to perform useful tasks: making appointments, buying things, question answering. Fully understanding and representing meaning is a difficult goal. Example tasks: machine translation, dictation, document retrieval, question answering, information extraction, personal assistants, chatbots.

Why NLP is hard [W12 p5–p6] . It is complex to represent, learn, and use linguistic, situational, world, and visual knowledge. **Human languages are ambiguous** (unlike programming languages). Interpretation depends on the real world, common sense, and context. The deck's examples: **non-standard English** (tweets: "SOO PROUD of what youve accomplished! U taught us 2…"), **segmentation issues** ("the New York-New Haven Railroad"), **idioms** (dark horse, get cold feet, lose face, throw in the towel), **neologisms** (unfriend, Retweet, bromance), **world knowledge** ("Mary and Sue are sisters" vs. "…are mothers"), and **tricky entity names** ("Where is *A Bug's Life* playing", "*Let It Be* was recorded", "a mutation on the *for* gene").

ELIZA [W12 p7]. Weizenbaum (1966). An early chatbot built from **simple pattern rules**, such as "I need X" → "What would it mean to you if you got X?"

Regular expressions [W12 p8] . Pattern matching on text. Example task: find every "the". The pattern `the` **misses capitalized** cases. `[tT]he` **incorrectly matches** "other" and "theology". `[^a-zA-Z][tT]he[^a-zA-Z]` requires a **non-letter on both sides**, so it matches only the standalone word.

Tokenization [W12 p9] . **Segmenting the words in running text.** It is **the first step for most NLP applications.**

Stemming vs. lemmatization [W12 p10] . **Stemming**: keep a list of prefixes and suffixes and **cut them off** (playing, plays, played → play). **Lemmatization**: **use linguistics to get the root word** (am, are, is → be; car, cars, car's, cars' → car). It is **more complex but more useful**. Example normalization: "the boy's cars are different colors" → "the boy car be differ color".

Topic modeling with LDA [W12 p11] . **Latent Dirichlet Allocation** finds the distributions of **latent topics** in a set of documents.

- A **topic is a probability distribution over words.**
- **Each document gets a distribution over topics** (a document is a *mixture* of topics).

- Topics are **latent** (hidden) because they are **not directly observed**. They are used to explain the **words, which are directly observed**.
- Documents and topics follow **Dirichlet distributions**, because these are "**distributions of distributions**".

LDA example [W12 p12] . Five documents: (1) Cats purr and have fur. (2) My cat meows and purrs. (3) Cats do not like the smell of oranges. (4) Oranges are citrus fruits. (5) Oranges are high in vitamin C. With $k = 2$: **Topic 1** puts high probability on *cat* ($\approx$0.30) and *purr* ($\approx$0.25); **Topic 2** on *citrus* and *vitamin* ($\approx$0.30 each); *orange* is split ($\approx$0.15 / 0.20). Documents 1–2 are $\approx 90\%$ Topic 1, Document 3 $\approx$ 70/30 (it mentions both cats and oranges), and Documents 4–5 $\approx 90\%$ Topic 2. *(Bar heights read from the slide.)*

LDA algorithm [W12 p13]

1. **Pick** k, the number of topics.
2. **Compile a vocabulary** from the documents.
3. **Randomly assign each word to a topic.**
4. Until all document distributions converge:
    a. For each document d:
        i. For each word w in d and each topic t:
            1. compute $P(t \mid d)$ = the **percent of the other words (not w) in d that are in t**;
            2. compute $P(w \mid t)$ = the **percent of the other documents (not d) that have word w** [in topic t];
        ii. repeat step i until document d's distribution converges.
    b. **Update** the probability of word w belonging to topic t: $P(w \mid t) = P(t \mid d) \cdot P(w \mid t)$.

Hyperparameters α and β [W12 p14] . They define the distributions that the document and topic densities come from. **α (or θ) = document–topic density: higher means documents are made of more topics. β = topic–word density: higher means topics are made of more words.**

Uses of LDA [W12 p15]. Find the major themes (topics) in a corpus; the topic distribution of each document (to visualize the words' topics); the important words of each topic. Use each document's **LDA vector** to **cluster documents** or as **features for supervised methods**, or weight bag-of-words vectors by how likely each word is to belong to a topic.

Word embeddings [W12 p16–p18] . Numeric vectors that represent word meaning in a continuous space. They capture semantic and syntactic relationships and enable text classification, sentiment analysis, and similar tasks. They go **beyond one-hot vectors**: words with shared context land in similar regions. *Classic (static)*: **Word2Vec**, where **CBOW predicts the target word from the context** and **skip-gram predicts the context from the target** ("king" $\approx$ "queen"); and **GloVe**, which uses **global co-occurrence statistics** and captures linear substructures ("Paris" $-$ "France" $\approx$ "Tokyo" $-$ "Japan"). **Limitation of static embeddings:** one vector per word ignores context ("bank" = a financial institution or a riverbank). **Transformers (BERT, GPT)** produce **context-dependent** vectors, so the "bank" embedding changes with the sentence.

BERT vs. GPT vs. BART [W12 p19]

	BERT	GPT	BART
Model type	**Encoder only**	**Decoder only**	**Encoder–decoder**
Direction	Bidirectional	**Unidirectional (left-to-right)**	Bidirectional
Pre-training	**Masked language modeling (MLM)**	**Autoregressive (causal) language modeling**	**Span corruption** (masking entire spans of words)
Fine-tuning	a task-specific layer on top of pre-trained BERT	task-specific prompts (few-shot / one-shot) and adapting parameters	versatile, many NLP tasks
Use cases	sentiment analysis, named entity recognition, word classification	text generation, completion, creative writing	translation, summarization, question answering
Origin	Google AI	OpenAI	Facebook AI

Embedding applications [W12 p20]. Similarity and analogies ("king" − "man" + "woman" ≈ "queen"), sentiment analysis, machine translation (align "book" ≈ "livre"), named entity recognition. **Challenges** [W12 p21]: **bias** in embeddings (they encode societal biases), **resource demands** (especially Transformers); next steps are efficient training, bias reduction, and multimodal (text + image) embeddings.

Language model [W12 p22] . **Predicts the next word or phrase from the previous words** by assigning **probabilities to word sequences**. "I am going to the" → "store" (high), "banana" (low). Used in speech recognition, text prediction, and chatbots.

N-gram models and the Markov assumption [W12 p23] . Predict using a **fixed window of previous words**. The deck's bigram example: "I am" → "going". **Markov assumption: a word depends only on a few previous words.** It works well but **struggles with long-range dependencies**, needs large datasets to cover rare phrases, and predicts rare words poorly ("The cat sat on the ___":"mat" works).

Neural language models [W12 p24]. RNNs and LSTMs remember past words better. **Transformers process all words at once** and use **attention** to focus on the important words. In "The dog that chased the cat was fast", the model knows "was" refers to "dog", not "cat". **LLMs** are deep transformers trained on multilingual text (GPT-4, LLaMA, DeepSeek-V2). **Challenges** [W12 p25]: bias; high compute (GPT-3 has **175 billion parameters**). Future directions: smaller, efficient models, ethical AI, and multimodal AI.

Neural networks for NLP [W12 p26]. Traditional NLP was rule-based and statistical. Neural networks learn patterns from data and process text as **sequences of numbers (embeddings)**. The key models are **RNN, LSTM, and Transformer**.

RNN [W12 p27] . Processes text **sequentially, remembering past words** by passing information through **hidden states**. **Limitation: struggles with long sentences because of vanishing gradients.** It is improved by **LSTMs and GRUs**. Example: sentiment of "I really enjoyed the movie", processed word by word.

LSTM [W12 p28] . Solves the vanishing-gradient issue with **gates**. The **forget gate** removes unimportant information. The **input gate** adds new important information. The **output gate** decides what to send forward. It is better at long-term dependencies (it links "The actor was amazing" to "I loved the performance").

Transformers [W12 p29] . Process **entire sentences at once**. **Self-attention** captures the relationships between words **no matter their distance**. They **eliminate sequential dependency, which allows parallel processing**, and they are the backbone of **BERT, GPT, and T5**. Example: translating long sentences without losing meaning.

Where we are now [W12 p30]. Chatbots (GPT, Bard), machine translation (Google Translate), sentiment analysis (customer reviews). The challenges are bias and computational cost.

Ethics [W12 p31–p35] . NLP affects hiring, finance, and law. **Bias**: models learn gender, racial, and cultural biases from the data ("doctor" ↔ "he", "nurse" ↔ "she"), which can lead to unfair decisions in hiring, law enforcement, and recommendations. **Privacy**: models process emails, medical records, and personal conversations; the risks are **data leaks** and **unintentional retention** of private data (chatbots storing user data without consent). **Misinformation and manipulation** (p34 repeats the privacy text). **Solutions**: diverse, carefully curated training data (bias mitigation); encryption and user control over data (privacy); fact-checking and model transparency (misinformation); regulations and ethics standards.

Formulas

LDA update [W12 p13]

$$P(t \mid d) = \frac{\#\text{ other words in } d \text{ assigned to } t}{\#\text{ other words in } d} \qquad P(w \mid t) \leftarrow P(t \mid d) \cdot P(w \mid t)$$

IN PLAIN WORDS

In plain words. To decide which topic a word belongs to, ask two questions. (1) **Does this document like topic t?** That is the share of its other words already in t. (2) **Does topic t like this word?** That is how often w appears in t elsewhere. Multiply the two and (re)assign the word in proportion to the product. Repeat until the assignments stop changing. Example: "orange" in Document 3 sits with cat words (Topic 1 is high for that document) but is a citrus word globally (Topic 2 likes it). The product balances these, giving the 70/30 mix.

NOT IN DECK — STANDARD DEFINITION

Generative story of LDA (not in deck — standard definition). For each document, draw a topic mixture $\theta_d \sim \text{Dirichlet}(\alpha)$. For each topic, draw a word distribution $\phi_t \sim \text{Dirichlet}(\beta)$. For each word position: draw a topic $z \sim \theta_d$, then a word $w \sim \phi_z$. Inference runs this backwards from the observed words (the deck's iterative algorithm is a simplified version of Gibbs sampling). *Further reading:* IBM `latent-dirichlet-allocation`, `topic-modeling`.

Language-model probability (bigram / Markov; formula: not in deck — standard definition)

$$P(w_1, \ldots, w_n) \approx \prod_{i=1}^{n} P(w_i \mid w_{i-1}) \qquad P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

IN PLAIN WORDS

In plain words. The chance of a sentence ≈ the product of "how often this word follows the previous one". Example: if "am" appears 10 times in the corpus and is followed by "going" 4 of those times, then $P(\text{going} \mid \text{am}) = \frac{4}{10} = 0.4$. *Further reading:* DeepLearning.AI NLP C2 M3 (n-gram language models).

Mechanisms: why and how

- **Why tokenization comes first:** every later step (counting, stemming, embedding) works on tokens [W12 p9].
- **Why lemmatization is more useful than stemming:** it returns real dictionary words and handles irregular forms (is → be). Stemming blindly chops suffixes [W12 p10].
- **Why LDA topics are "latent":** nobody labels the topics. They are inferred because they explain which words co-occur in the documents [W12 p11].
- **Why the Dirichlet distribution:** LDA needs a distribution over *probability vectors* (topic mixtures and word mixtures), and the Dirichlet is "a distribution of distributions" [W12 p11].
- **How α and β change the result:** a larger α spreads each document over more topics; a larger β spreads each topic over more words [W12 p14].
- **Why n-grams fail on long-range dependency:** the Markov window only sees the last few words [W12 p23].
- **Why RNNs forget and LSTMs remember:** gradients through many time steps vanish, and the LSTM's gates create a protected path that keeps important information [W12 p27–p28].
- **Why Transformers are faster to train than RNNs:** there is no step-by-step recurrence, so all positions are computed in parallel, and attention links any two words directly [W12 p29].
- **Why static embeddings are not enough:** a word has one vector regardless of meaning; contextual models (BERT, GPT) recompute the vector for each sentence [W12 p18].

Comparisons

A	B	Key difference
Stemming	**Lemmatization**	chop prefixes and suffixes from a list, fast, may give non-words vs. linguistic root (lemma), more complex, more useful [W12 p10]
Tokenization	Stemming/lemmatization	split the text into tokens vs. normalize each token to a root [W12 p9–p10]
LDA topic	LDA document	a distribution over **words** vs. a distribution over **topics** [W12 p11]
α	β	document–topic density vs. topic–word density [W12 p14]
LDA (Latent Dirichlet Allocation)	Linear Discriminant Analysis	unsupervised topic model for text (**this course's LDA**) vs. a supervised classifier / dimensionality reduction, not covered in these decks

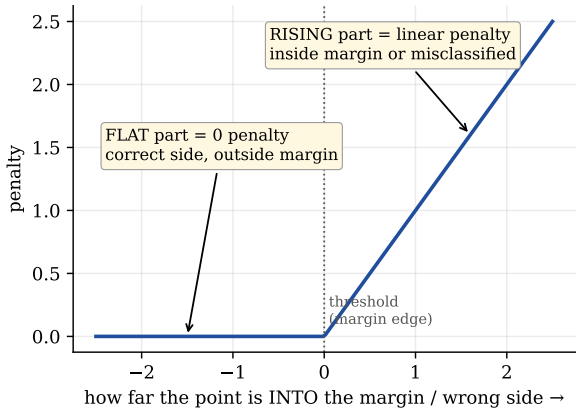
A	B	Key difference
Static embeddings (Word2Vec, GloVe)	Contextual embeddings (BERT, GPT)	one vector per word vs. a vector that depends on the sentence [W12 p17–p18]
CBOW	Skip-gram	context → target vs. target → context [W12 p17]
BERT	GPT	encoder, bidirectional, masked LM, understanding tasks vs. decoder, left-to-right, autoregressive, generation [W12 p19]
N-gram LM	Neural LM	fixed window, Markov, poor on rare words and long range vs. RNN/LSTM/Transformer, longer memory [W12 p23–p24]
RNN	LSTM	plain hidden state, vanishing gradients vs. forget/input/output gates, long-term memory [W12 p27–p28]
RNN/LSTM	Transformer	sequential vs. parallel self-attention, any distance [W12 p29]
Rule-based NLP (ELIZA, regex)	Statistical/neural NLP	hand-written patterns vs. patterns learned from data [W12 p7–p8, p26]

Figure Gallery: read any plot in 20 seconds

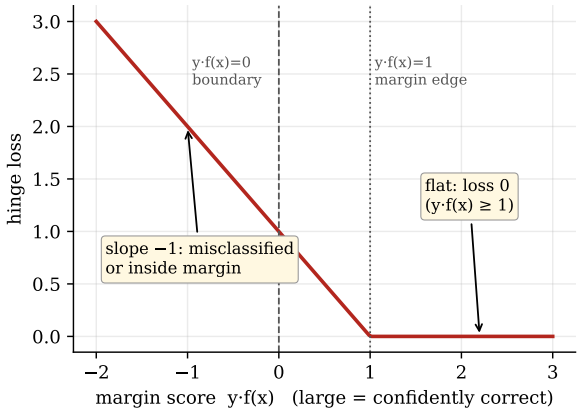
Each figure is re-drawn from a deck slide (or marked as a standard illustration) and labeled part by part. Under each one, ***Say this*** *is a ready-to-write exam answer.*

Hinge loss (flat part vs. rising part)

(a) Deck view (W6 p20): penalty vs. distance into margin



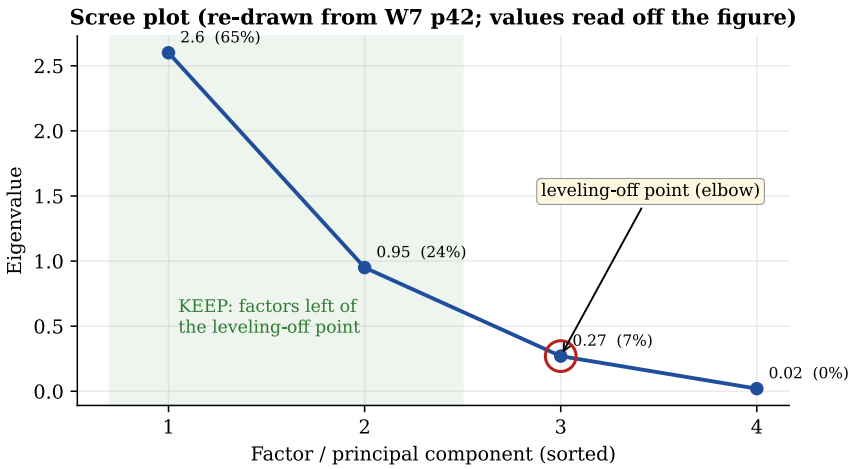
(b) Standard formula: max(0, 1 − y·f(x))



EXAM ANSWER

Say this. "The **flat part** is zero penalty: the point is on the correct side **and outside the margin**, so it does not affect the hyperplane. The **rising part** is a **linear penalty** that grows with how far the point is inside the margin or on the wrong side. These points (margin and misclassified) are the support vectors. SVM balances margin width against the total hinge penalty" [W6 p19–p20]. Formula (review file): $\max(0,\ 1 - y_i(w \cdot x_i + b))$. The kink is at $y\,f(x) = 1$, the margin edge.

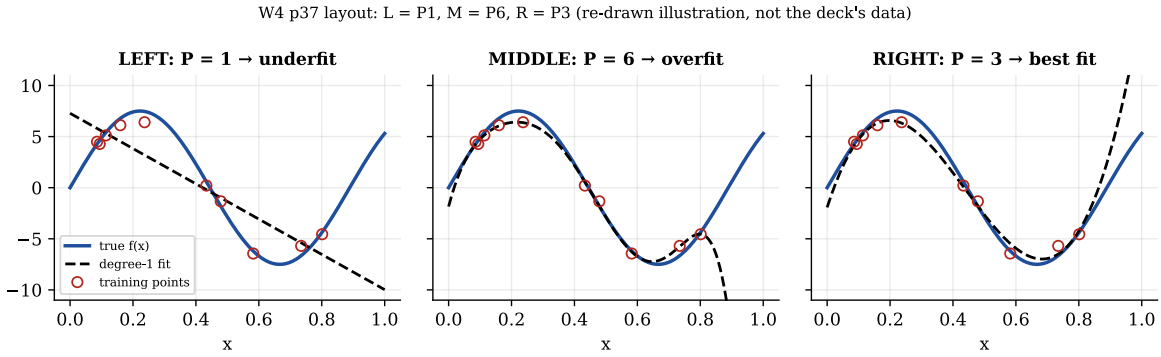
Scree plot (elbow / leveling-off point)



EXAM ANSWER

Say this. "A scree plot shows the **eigenvalues** (variance explained) of the principal components, sorted in decreasing order. It is used **after computing PCA to decide how many components to keep**: keep the components **to the left of the leveling-off (elbow) point**. Beyond it, each extra component adds little variance, which is mostly noise" [W7 p42]. Here: keep 2. Percent of variance = eigenvalue ÷ #features [W7 p41].

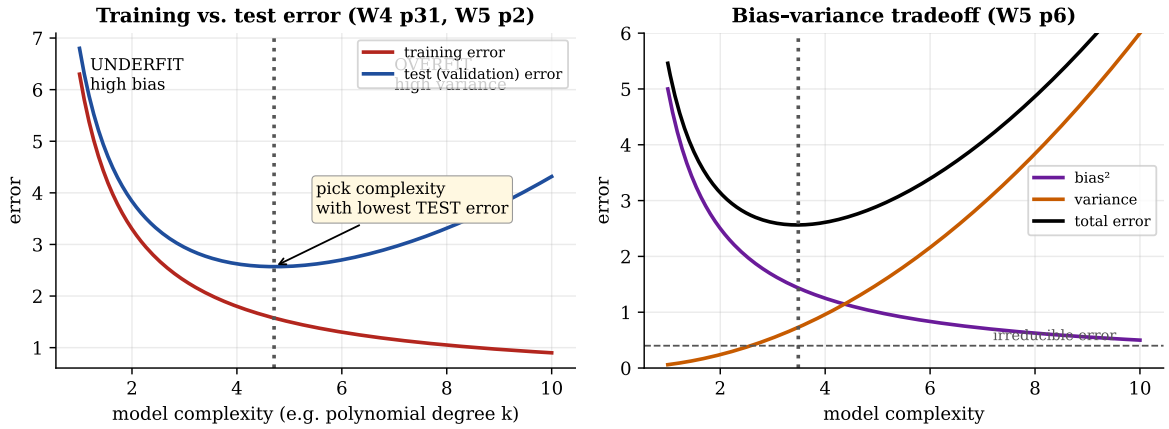
Which fits better: k = 1, 3, or 6? (W4 p37 layout)



EXAM ANSWER

Say this. "**k = 3 (the right panel)** fits better. k = 1 (left) is a straight line that **underfits**: large training error, it misses the curve (high bias). k = 6 (middle) passes almost exactly through the training points but swings wildly between them, so it **overfits**: near-zero training error, high test error (high variance), and small data changes would alter it drastically. k = 3 has some training error but **closely resembles the true function**, so it **generalizes best**" [W4 p34–p37].

Training vs. test error and the bias–variance tradeoff

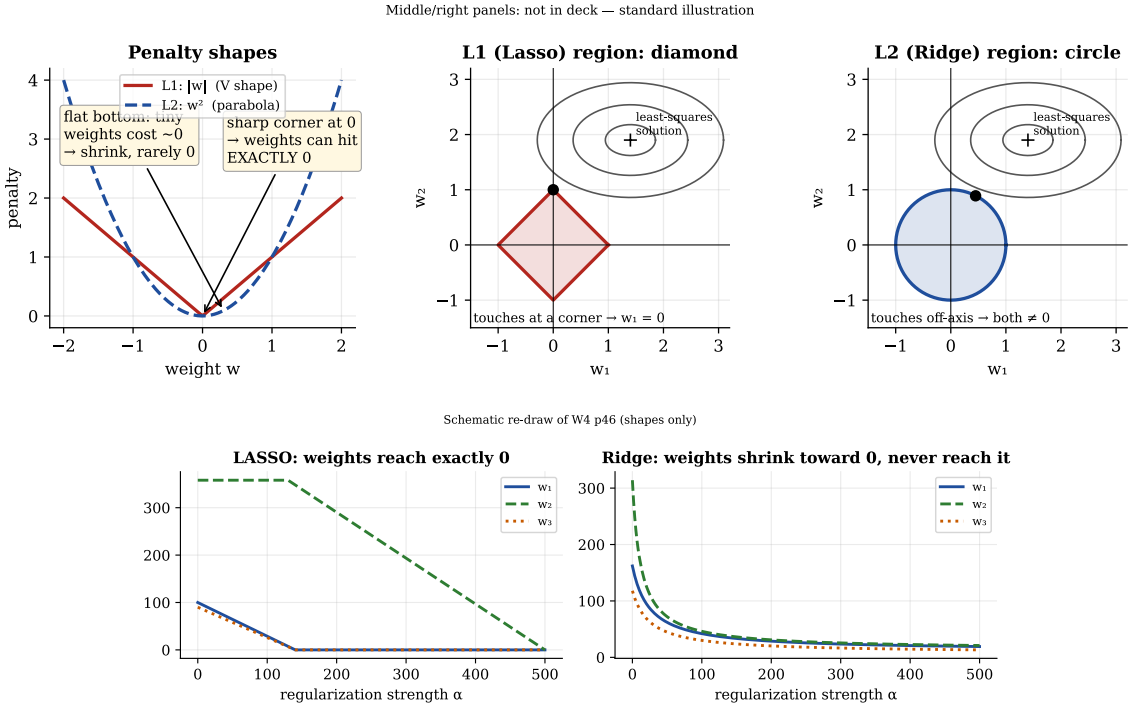


EXAM ANSWER

Say this. “As complexity grows, training error keeps falling, but test error is U-shaped. On the left both are high (**underfitting, high bias**). On the right training error is low but test error rises (**overfitting, high variance**). Choose the complexity at the **minimum of the test/validation error**” [W4 p31; W5 p6]. *The deck’s W4 p31 sketch draws only the rising test-error branch; the full U-shape is standard.*

Figs

L1 (V) vs. L2 (parabola) penalty

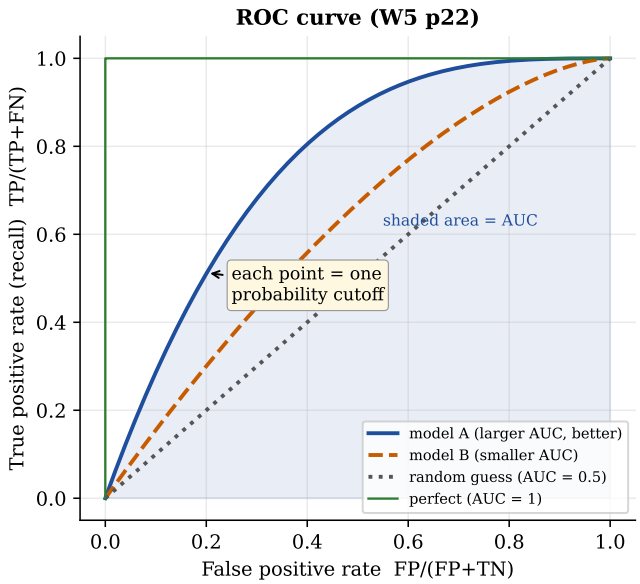


EXAM ANSWER

Say this. "**L1 (Lasso), penalty** $\alpha \sum |w_j|$, is **V-shaped** with a sharp corner at 0. Its slope stays constant near 0, so the optimizer can push weights **exactly to zero**, which gives **feature selection and a sparse model**. **L2 (Ridge), penalty** $\alpha \sum w_j^2$, is a **parabola**, flat at 0. Tiny weights cost almost nothing, so weights **shrink smoothly toward 0 but rarely reach it**. As α grows, Lasso coefficients hit 0 one by one while Ridge coefficients decay gradually" [W4 p43–p48]. The diamond/circle picture is standard (not in deck): the RSS contours touch the L1 diamond at a corner, where one weight is 0.

Figs

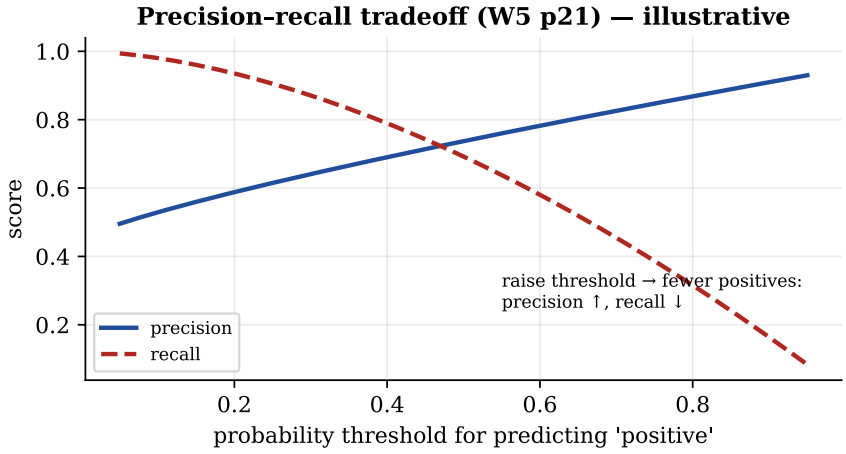
ROC curve and AUC



EXAM ANSWER

Say this. "ROC plots the **true positive rate (recall)** against the **false positive rate** as the probability cutoff varies. Every point is one threshold. The diagonal is random guessing (AUC 0.5); the top-left corner is perfect (AUC 1). **The curve closer to the top-left, i.e. the larger AUC, is the better classifier**" [W5 p22].

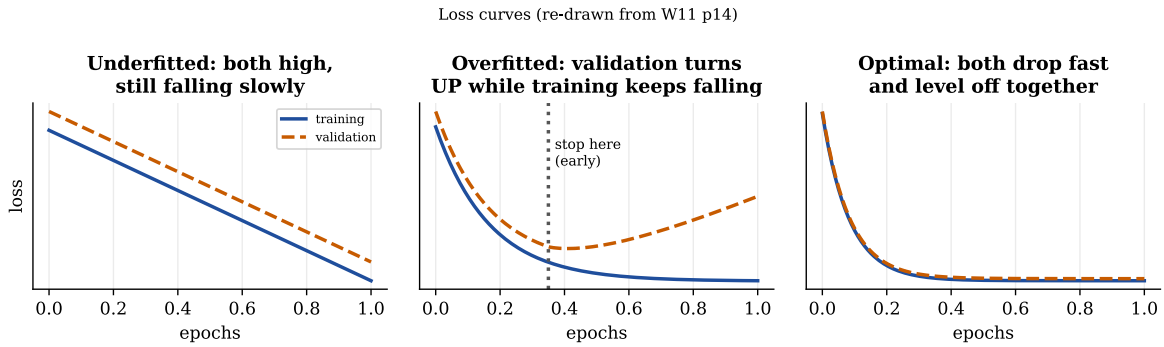
Precision–recall vs. threshold



EXAM ANSWER

Say this. "Raising the threshold produces fewer positive predictions. Those that remain are more reliable (**precision** ↑), but more real positives are missed (**recall** ↓). This is the tradeoff" [W5 p21].

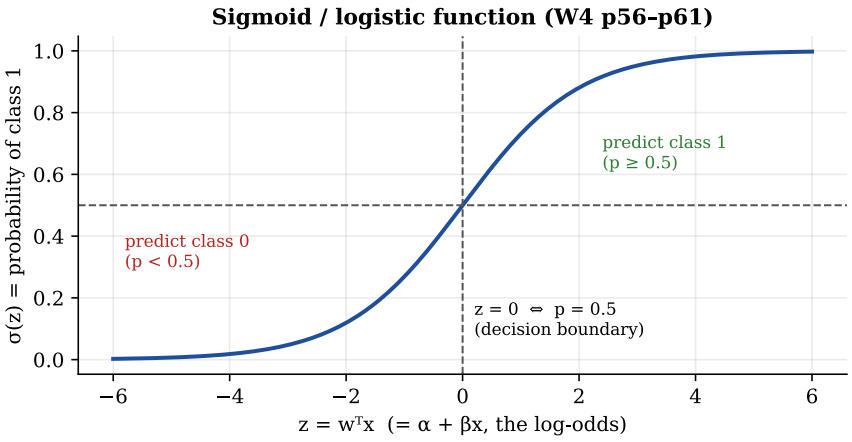
Loss curves during training (underfit / overfit / optimal)



EXAM ANSWER

Say this. "**Underfit:** training and validation loss are both high and still slowly decreasing, because the model is too small (1 hidden layer, 1 neuron). **Overfit:** validation loss turns **upward** while training loss keeps falling; the model memorizes the training data (here with learning rate 0.1). **Optimal:** both drop and level off together (learning rate 0.001). A good curve shows an overall decreasing trend with occasional fluctuations" [W11 p13–p14].

Sigmoid (logistic) curve

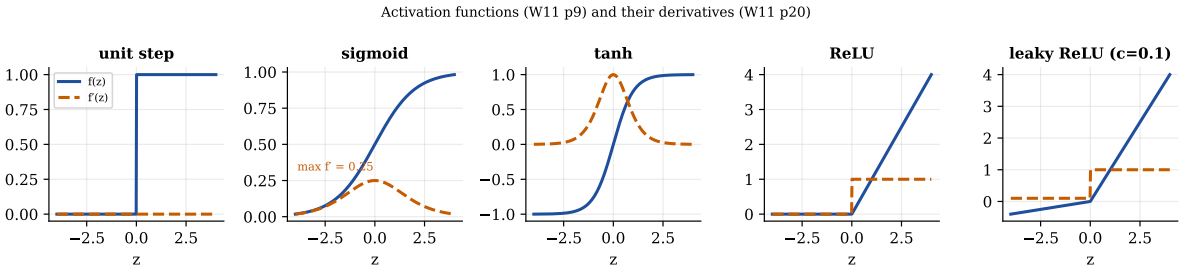


EXAM ANSWER

Say this. "The sigmoid maps any score $z = w^T x$ to a probability in (0,1). It is S-shaped and monotonic, with $\sigma(0) = 0.5$, the decision boundary at threshold 0.5. Its inverse, the logit $\ln \frac{p}{1-p}$, is linear in x" [W4 p56–p61].

Figs

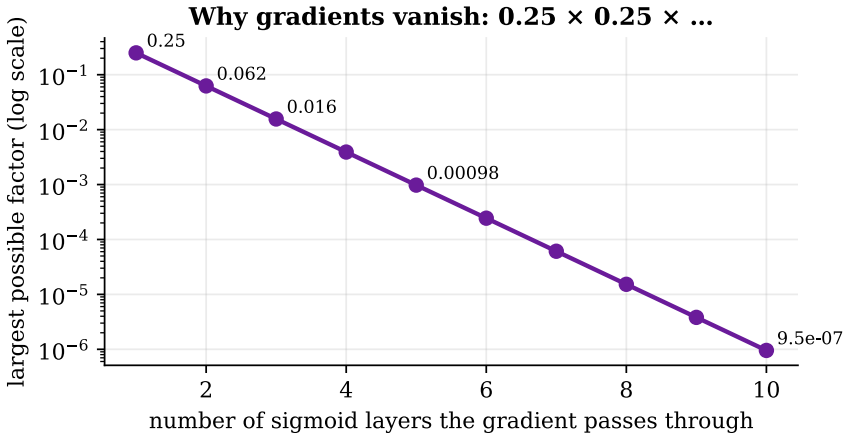
Activation functions and their derivatives



EXAM ANSWER

Say this. "The unit step is for single-layer perceptrons, ReLU for multilayer networks, and tanh for recurrent networks [W11 p10]. The sigmoid's derivative peaks at only **0.25** and is nearly 0 for large $|z|$, which causes **vanishing gradients**. ReLU's derivative is 1 for $z > 0$" [W11 p20, p22].

Why gradients vanish

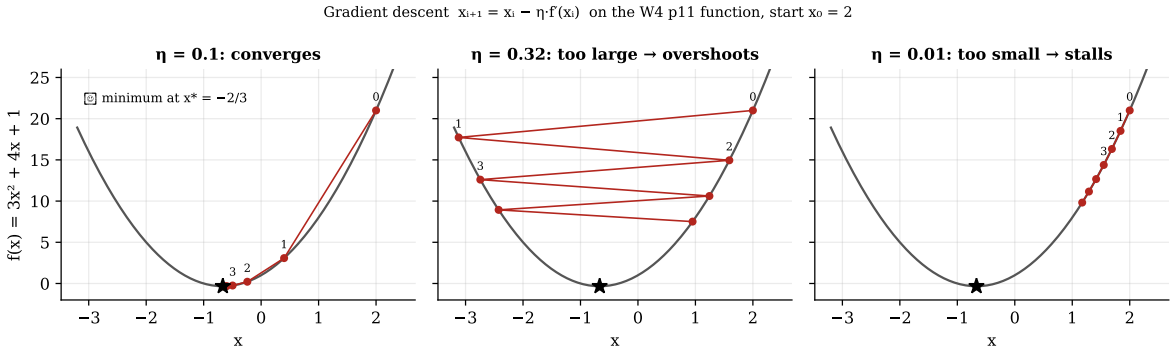


EXAM ANSWER

Say this. "Backpropagation multiplies one activation derivative per layer (chain rule). With sigmoid, each factor is ≤ 0.25, so after n layers the gradient is shrunk by up to 0.25^n; early layers barely update. If the factors are > 1, they multiply into **exploding** gradients" [W11 p21–p22].

Figs

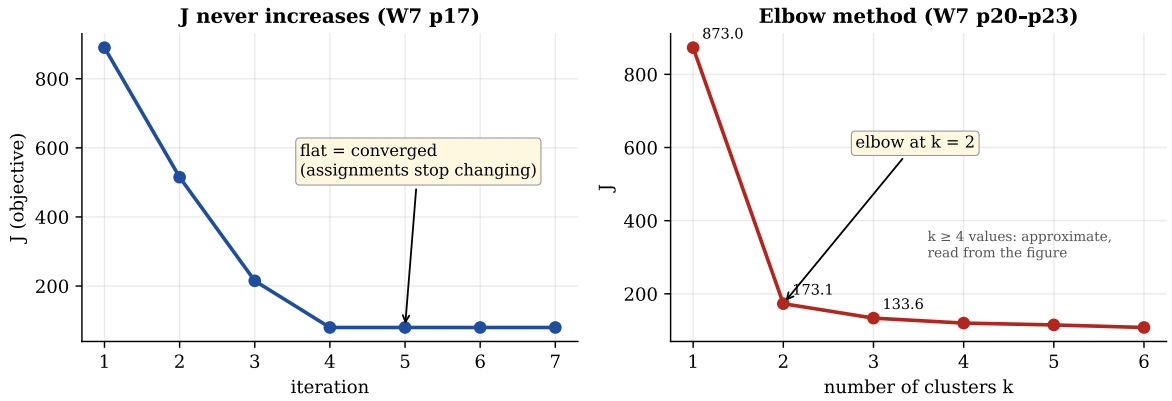
Gradient descent and the learning rate



EXAM ANSWER

Say this. "Each step moves **against** the slope: $x_{i+1} = x_i - \eta f'(x_i)$. A well-chosen η converges to the minimum ($x^* = -2/3$ for the deck's $f(x) = 3x^2 + 4x + 1$). **Too large: it overshoots** and oscillates across the valley. **Too small: it stalls**, with tiny steps and a long time to converge" [W4 p11–p18].

K-Means: objective per iteration and elbow over k

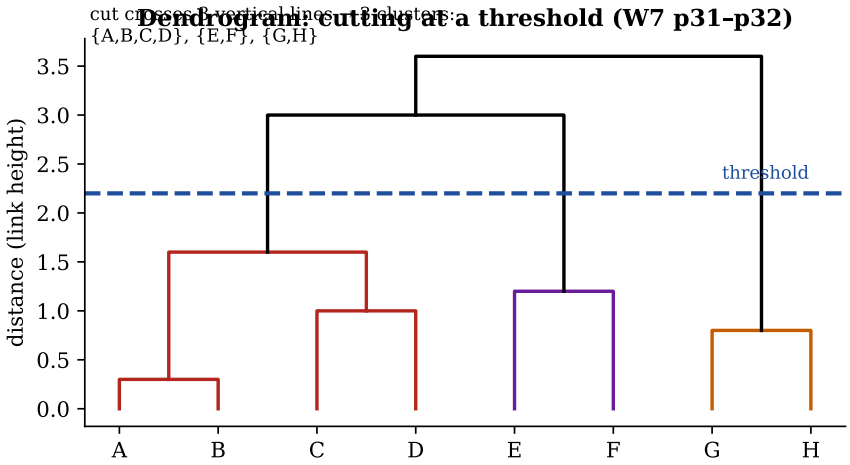


EXAM ANSWER

Say this. "**Left:** J only decreases or stays flat across iterations, and once it is flat the assignments stop changing: K-Means has **converged** (to a local optimum). **Right:** the elbow method plots J against k. The abrupt drop from $k = 1$ (873.0) to $k = 2$ (173.1), followed by small gains, suggests **2 clusters**" [W7 p17, p20–p23].

Figs

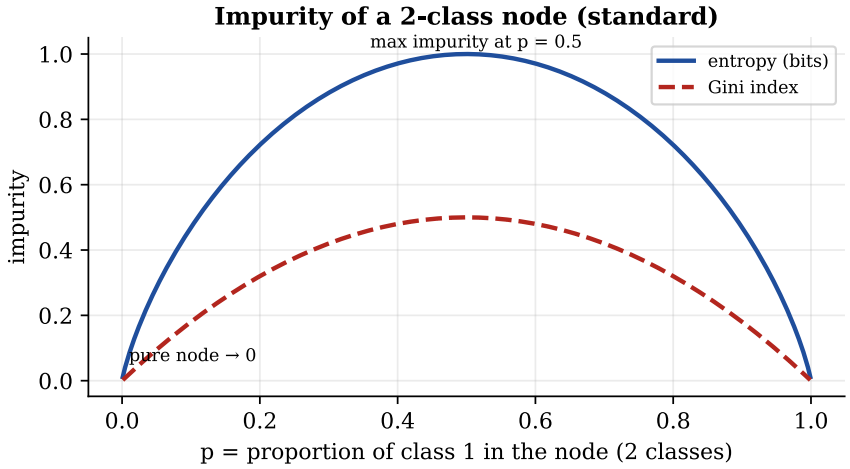
Dendrogram cut



EXAM ANSWER

Say this. "Link height = the distance at which clusters merge. A horizontal threshold line cuts the dendrogram, and **the number of vertical lines (clades) it intersects = the number of clusters**. Confirm with a quantitative method such as the elbow" [W7 p31–p32].

Impurity: entropy vs. Gini

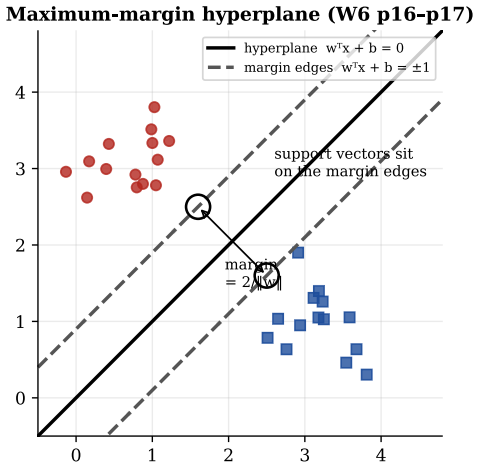


EXAM ANSWER

Say this. "Both are 0 for a pure node and largest for an even mix (2 classes: entropy 1 bit, Gini 0.5). CART chooses the split with the lowest weighted Gini, which is equivalent in spirit to the highest information gain" [W8 p16]. (Entropy curve: standard.)

Figs

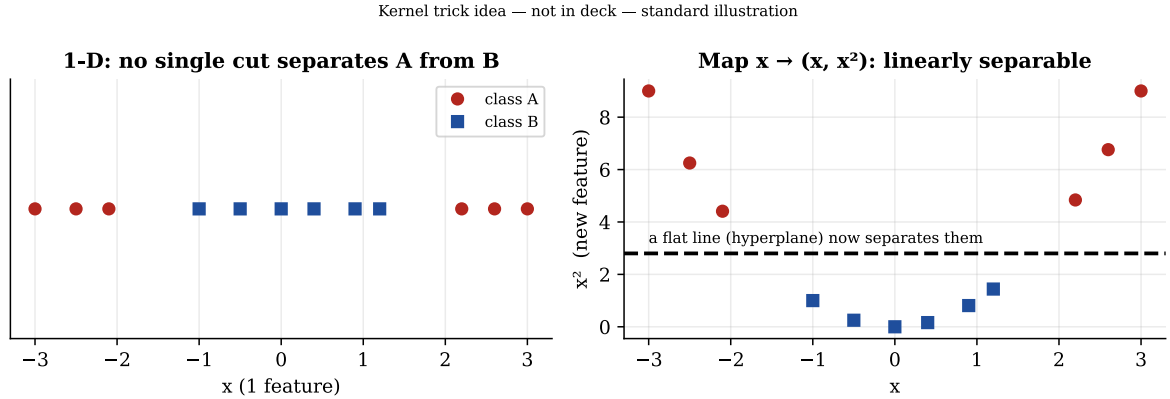
SVM margin and support vectors



EXAM ANSWER

Say this. "SVM picks the hyperplane with the **widest margin**. The circled points on the margin edges are the **support vectors**; they alone determine the hyperplane, and moving any other point changes nothing" [W6 p16–p17].

Kernel trick

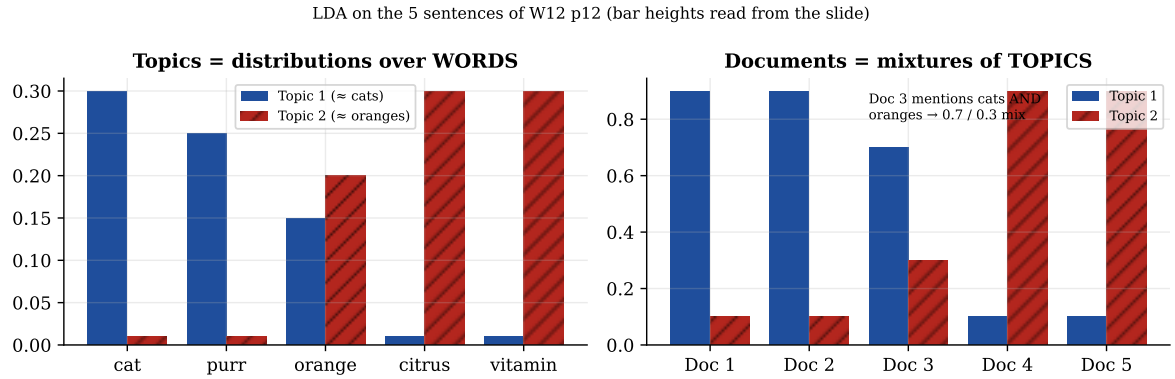


EXAM ANSWER

Say this. "When no hyperplane separates the classes in the original space, the kernel trick **maps the data to a higher-dimensional space where a linear boundary exists** (here $x \mapsto (x, x^2)$). The kernel performs this mapping efficiently (polynomial, RBF)" [W6 p15, p18]. (This particular picture is a standard illustration.)

Figs

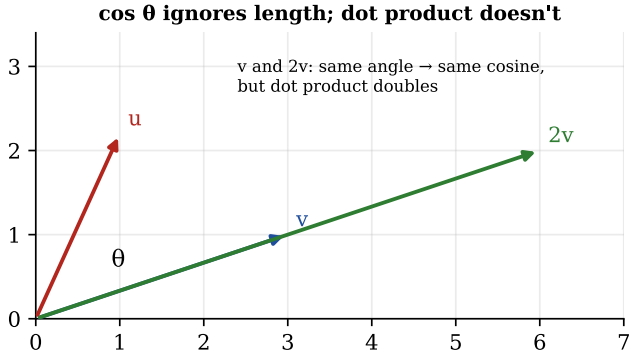
LDA: topics over words, documents over topics



EXAM ANSWER

Say this. "In LDA a **topic is a probability distribution over words** (Topic 1 ≈ cat/purr, Topic 2 ≈ citrus/vitamin) and **each document is a mixture of topics** (Doc 3 is about 70% Topic 1 and 30% Topic 2). Topics are latent: they are inferred from the observed words" [W12 p11–p12].

Cosine similarity vs. dot product



EXAM ANSWER

Say this. "The dot product grows with vector length, so an item with every feature always wins. Cosine divides by the lengths and keeps only the **angle**, penalizing un-specific items" [W10 p18–p19].

Quiz Types Seen in Class and Model Solutions

*These are the professor's surprise-quiz topics. Expect **variants** of each. Every entry gives a model answer to copy the structure from, the key mechanism, and the twists to expect.*

Q1. Why is overfitting stated with n+1 points instead of n?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "A polynomial of degree n has $n+1$ coefficients $(w_0, \ldots, w_n)$. With $n+1$ data points we get $n+1$ equations in $n+1$ unknowns, so we can solve for a curve that passes **exactly through every point**: training error is 0. That happens **regardless of the true pattern, even for pure noise**, so the perfect fit says nothing about generalization; the model is memorizing. With only n points the system is underdetermined (infinitely many exact fits). With $n+1$ points the fit is **unique and exact**, which is why the deck states it that way: *'a degree-N polynomial exactly fits N+1 data points'*" [W4 p31].

Key mechanism: #parameters $\geq$ #data points $\Rightarrow$ zero training error $\Rightarrow$ overfitting ("less training data relative to the model parameters") [W4 p31]. **Variants:** "What happens to the test error?" (it explodes). "How do you fix it?" (more data, cross-validation, regularization [W4 p38]). "Two points and a line?" (a degree-1 line fits 2 points exactly).

Q2. From the three curves L / M / R (W4): which k fits better, 1, 3, or 6?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "**k = 3.** The left panel (k = 1) is a straight line, too simple, with a large training error, so it **underfits** (high bias). The middle panel (k = 6) goes almost exactly through the training points but oscillates, so it **overfits** (low training error, high test error, high variance; a small change in the data changes it drastically). The right panel (k = 3) has some training error but **closely follows the target function**, so it will have the **lowest test error** and generalizes best" [W4 p34–p37].

Key mechanism: judge by test (generalization) error, not training error. **Variants:** the panels shuffled (identify them by shape: straight = 1, wiggly with extreme ends = 6, smooth curve = 3). "Which has the lowest *training* error?" (k = 6, a twist!). "Which has the largest weights?" (k = 6 [W4 p39]). An error-vs-complexity plot asking for the best degree (the minimum of the validation curve; W5 p28 picks degree 3).

Q3. Does K-Means converge? Why?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "**Yes, it always converges, but to a local optimum.** K-Means minimizes $J = \sum_n \sum_k z_{nk} \|x_n - \mu_k\|^2$ by alternating two steps, and **neither step can increase** J. (1) The *assignment step* moves each point to its **nearest** centroid, so each point's squared distance can only drop or stay. (2) The *update step* sets each centroid to the **mean** of its points, the value that minimizes the sum of squared distances. So J is **monotonically non-increasing**. There are only **finitely many** ways to assign N points to k clusters, so it cannot keep improving forever, and it stops when the assignments no longer change. The result depends on the initial centroids, so it may be only a **local** optimum" [W7 p11–p12, p17, p25].

Variants: "Is the solution globally optimal?" (no, local; re-run with different initializations). "What does the J-vs-iteration plot look like?" (decreasing, then flat [W7 p17]). "Why does it fail on non-convex shapes or outliers?" (squared distance to a mean assumes compact blobs; use DBSCAN [W7 p25, p34]).

Q4. PCA: what are the principal components / principal vectors, and how do they relate to eigenvectors and eigenvalues?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "PCA finds new **orthogonal** axes in the directions of **maximum variance**. The first component has the most variance and each later one less [W7 p40]. The **principal vectors (directions) are the eigenvectors** of the data's covariance matrix: each eigenvector holds the **coefficients of the input features** that make up that component [W7 p41]. The **eigenvalue** is the **amount of variance explained** by that component (percent explained = eigenvalue ÷ #features [W7 p41]). The **principal components** are the data projected onto those eigenvectors. Sort by eigenvalue, then keep the top few (scree plot) to reduce dimension while keeping most of the information." *(Covariance-matrix wording: standard; the deck says eigenvalue = explained variability, eigenvector = feature coefficients.)*

Variants: "Why are the components uncorrelated?" (they are orthogonal eigenvectors of a symmetric matrix [W2 p46]). "What is the loading?" (eigenvector $\times \sqrt{\text{eigenvalue}}$ [W7 p41]). "PCA vs. clustering?" (it reduces **features**, not groups of instances). "Drawback?" (the new features lose their original meaning, so they are less interpretable [review file]).

Quiz

Q5. What is the role of the kernel trick in SVM?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "When the classes **cannot be separated by a hyperplane** in the original feature space, the kernel trick **maps the data into a higher-dimensional space where they become linearly separable**, so SVM can still find a maximum-margin *linear* boundary there. That boundary corresponds to a **non-linear** boundary in the original space. The kernel is a specialized mapping that does this efficiently, without building all the new features by hand (polynomial: all products up to degree d; RBF: similarity that decays exponentially with distance)" [W6 p15, p18].

Variants: "Give an example" (1-D points with class B in the middle become separable using x^2; see the gallery). "Name the kernels" (linear, polynomial, RBF [W6 p18]). "2 features, polynomial degree 2: how many dimensions?" (6: $x^2, xy, y^2, x, y, 1$ [W6 p18]).

Q6. What does "naive" mean in naive Bayes?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "It assumes the features (words) are **conditionally independent given the class**, so the likelihood of a whole message factors into a product of single-word likelihoods: $P(\text{Message} \mid \text{Class}) \approx \prod_j P(\text{Word}_j \mid \text{Class})$. It is 'naive' because real language violates this: **word order and context matter** ('I love bacon' is more likely than other orderings). Despite the unrealistic assumption, it classifies well and is fast" [W6 p11, p14].

Variants: "Is naive Bayes generative or discriminative?" (**generative** [W11 p24]). "Why is it good for text?" (many features, fast counting [W6 p9–p10]). "In recommender CF, what is the naive assumption?" (a user's rating of item i is independent of that user's other ratings [W10 p56]).

Q7. How do you prevent zero likelihood in a classification model?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "Use **Laplace smoothing**. In naive Bayes, if a category (word) never appeared with a class in training, its estimated probability is 0, and because the class likelihood is a **product**, that single 0 makes the whole likelihood 0. The class is then ruled out no matter how strong the other evidence is. Laplace smoothing **adds a fictional occurrence (+1) to every category's count**, including unseen ones: $P(x_j \mid C) = \dfrac{\text{count}(x_j, C) + 1}{\text{count}(C) + V}$. Every probability is then positive and every category contributes" [W6 p13; formula from the review file].

Variants: "Give an example" (staff never tweeted 'love' [W6 p13]). "What is V?" (the number of categories, which keeps the probabilities summing to 1). "Side effect?" (slightly shrinks the frequent-category probabilities toward uniform).

Q8. What is a scree plot for, and when do you use it?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "A scree plot graphs the **eigenvalues** (variance explained) of the components/factors, **sorted from largest to smallest**, against the component number. Use it **after running PCA / factor analysis, to decide how many components to keep**. Keep the components **to the left of the leveling-off (elbow) point**. After that point the curve flattens, so each additional component explains very little extra variance, which is mostly noise, and it can be discarded. (A complementary rule: keep enough components to reach about 90–95% cumulative variance [review file])" [W7 p42].

Variants: given a plot, "how many components?" (count the points before the elbow; the deck's example gives 2). "How is it similar to the elbow method in K-Means?" (same idea, a point of diminishing returns, but the y-axis there is J against k [W7 p23]).

Q9. LDA in NLP (Latent Dirichlet Allocation)

WORKED EXAMPLE / MODEL ANSWER

Model answer. "LDA is an unsupervised **topic model**. It assumes each **document is a mixture (probability distribution) of topics**, and each **topic is a probability distribution over words**. Topics are **latent**: never observed, only inferred from the observed words. The document–topic and topic–word distributions follow **Dirichlet** distributions ('distributions of distributions'), with hyperparameters $\boldsymbol{\alpha}$ **= document–topic density** (higher means more topics per document) and $\boldsymbol{\beta}$ **= topic–word density** (higher means more words per topic). *Algorithm:* pick k, build the vocabulary, randomly assign words to topics, then repeatedly re-estimate $P(t \mid d)$ (the share of a document's other words in topic t) and $P(w \mid t)$ (how often w appears in t elsewhere), updating $P(w \mid t) = P(t \mid d)P(w \mid t)$ until the distributions converge. The result: themes of the corpus, each document's topic vector (usable for clustering or as features), and the top words per topic" [W12 p11–p15].

Variants: "In the 5-sentence example, why is Document 3 mixed?" (it has cat words and orange words [W12 p12]). "Effect of a larger α?" "Is LDA supervised?" (no). "Don't confuse it with Linear Discriminant Analysis."

Q10. Figure: hinge loss. What are the flat part and the rising part?

WORKED EXAMPLE / MODEL ANSWER

Model answer. "The figure is the **hinge function** used as the SVM penalty. **Flat part (penalty = 0):** the instance is correctly classified and lies **outside the margin**, so it costs nothing and does not influence the hyperplane. **Rising part (linear):** the instance is **inside the margin or misclassified**, and the penalty grows **linearly with its distance into the margin**. The kink is at the margin edge, $y\,f(x) = 1$. SVM chooses the hyperplane that balances a wide margin against the total penalty, tuned through the hinge's steepness" [W6 p19–p20].

Quiz

Variants: "Compute the hinge loss for $y = +1$, $f(x) = 0.4$" ($\max(0, 1 - 0.4) = 0.6$). "$y = +1$, $f(x) = 2$" (0). "$y = -1$, $f(x) = 0.5$" ($\max(0, 1 + 0.5) = 1.5$, misclassified). "Which loss do neural nets use for classification?" (hinge **or** log loss [W11 p13]).

Q11. Figure: L1 vs. L2 penalty shapes

WORKED EXAMPLE / MODEL ANSWER

Model answer. "The V-shaped curve is the L1 (Lasso) penalty $\alpha \sum |w_j|$, and the parabola is the L2 (Ridge) penalty $\alpha \sum w_j^2$. The V has a **sharp corner at 0** with a constant slope, so even small weights pay a fixed price, and the optimum often sets weights **exactly to 0**: **feature selection and a sparse model**. The parabola is **flat near 0**, so small weights cost almost nothing; Ridge **shrinks all weights smoothly but rarely to exactly 0**. In both, a larger α means stronger shrinkage: less variance, more bias. Elastic Net mixes them" [W4 p43–p48].

Variants: "Which would you use with many irrelevant features?" (Lasso). "With correlated features?" (Ridge, or Elastic Net). "Why standardize first?" [W4 p41]. "What is $\alpha = 0$?" (ordinary least squares).

Likely next variants (same styles, not yet asked)

Likely question	One-line answer	Where
Why not use linear regression for a 0/1 target?	Non-normal errors, non-constant variance, and p must stay in [0,1]; use logistic regression	p. 20
Precision vs. recall: which matters for X?	False alarms costly → precision; misses costly → recall	p. 29
Bagging vs. boosting?	Parallel bootstrap + vote (variance ↓) vs. sequential reweighting of errors (bias ↓)	p. 47
Generative vs. discriminative?	Learns the data distribution (naive Bayes, LLMs) vs. the decision boundary (SVM)	p. 59
Stemming vs. lemmatization?	Chop affixes vs. linguistic root	p. 65
Mini-batch vs. stochastic GD?	b updates per epoch vs. n updates per epoch	p. 55
Vanishing gradients: what and why?	Gradients → 0 from multiplying small derivatives (sigmoid ≤ 0.25); fix with ReLU, batch norm, LSTM	p. 59
Cold start / serendipity?	No history → query the user / a filter bubble → explore similar users' shifts	p. 48
Why does a random forest decorrelate its trees?	Bootstrap rows + random feature subsets ($m < M$)	p. 41
Why does DBSCAN find outliers?	Points that are neither core nor border are left unassigned	p. 37

Worked Numeric Examples (by hand, no calculator)

Confusion matrix → metrics

*The decks give the formulas but no numeric confusion matrix [W5 p19–p20], so the numbers below are **illustrative**.*

	Predicted +	**Predicted −**	**Row total**
Actual +	TP = 40	FN = 10	50
Actual −	FP = 20	TN = 130	150
Column total	60	140	200

WORKED EXAMPLE / MODEL ANSWER

1. **Accuracy** $= \dfrac{TP + TN}{\text{all}} = \dfrac{40 + 130}{200} = \dfrac{170}{200} = 0.85$.
2. **Precision** $= \dfrac{TP}{TP + FP} = \dfrac{40}{60} = \frac{2}{3} \approx 0.667$ (the "Predicted +" column).
3. **Recall** $= \dfrac{TP}{TP + FN} = \dfrac{40}{50} = 0.8$ (the "Actual +" row).
4. **F1** $= \dfrac{2PR}{P + R} = \dfrac{2 \cdot \frac{2}{3} \cdot \frac{4}{5}}{\frac{2}{3} + \frac{4}{5}} = \dfrac{16/15}{22/15} = \dfrac{16}{22} \approx 0.727$.
5. **FPR** $= \dfrac{FP}{FP + TN} = \dfrac{20}{150} \approx 0.133$, the x-coordinate of this threshold's point on the ROC curve; its y-coordinate is TPR = recall = 0.8.

Trap check: a model that always predicts "−" gets accuracy $\dfrac{150}{200} = 0.75$ but recall 0. Accuracy alone misleads on imbalanced data.

One K-Means iteration (assign + update)

Illustrative points (the deck shows the steps only as figures [W7 p13–p16]). Points: $A = (1, 1)$, $B = (2, 1)$, $C = (4, 3)$, $D = (5, 4)$; $k = 2$; initial centroids $\mu_1 = (1, 1)$, $\mu_2 = (5, 4)$.

Calc

WORKED EXAMPLE / MODEL ANSWER

Assignment step (squared distances; no square roots needed to compare):

Point	to $\mu_1 = (1,1)$	to $\mu_2 = (5,4)$	Assigned
A (1,1)	0	$16 + 9 = 25$	cluster 1
B (2,1)	$1 + 0 = 1$	$9 + 9 = 18$	cluster 1
C (4,3)	$9 + 4 = 13$	$1 + 1 = 2$	cluster 2
D (5,4)	$16 + 9 = 25$	0	cluster 2

J before the update $= 0 + 1 + 2 + 0 = 3$.

Update step (mean of each cluster): $\mu_1 = (\frac{1+2}{2}, \frac{1+1}{2}) = (1.5,\ 1)$, $\mu_2 = (\frac{4+5}{2}, \frac{3+4}{2}) = (4.5,\ 3.5)$.

J after $= 0.25 + 0.25 + (0.25 + 0.25) + (0.25 + 0.25) = 1.5 < 3$. **$J$ decreased.**

Convergence check: re-assign with the new centroids. A, B are still closest to μ_1 and C, D to μ_2. **No assignment changed, so the algorithm has converged.**

Entropy and information gain for a split (deck data)

Data from the deck's Gini example [W8 p17]: 7 beans, 4 Sira and 3 Cali. The split sends 3 Sira to A_1 and 1 Sira + 3 Cali to A_2. The entropy formulas are standard (not in deck).

WORKED EXAMPLE / MODEL ANSWER

Parent: $p = (\frac{4}{7}, \frac{3}{7})$. $H = -\frac{4}{7}\log_2\frac{4}{7} - \frac{3}{7}\log_2\frac{3}{7} = \frac{4}{7}(0.807) + \frac{3}{7}(1.222) \approx 0.461 + 0.524 = \mathbf{0.985}$ bits. *(From the helper table:* $\log_2\frac{4}{7} = 2 - 2.807 = -0.807$ *and* $\log_2\frac{3}{7} = 1.585 - 2.807 = -1.222$.*)*

Child A_1 (3 Sira, 0 Cali): pure, so $H = 0$. **Child A_2** (1 Sira, 3 Cali): $H = -\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4} = \frac{1}{4}(2) + \frac{3}{4}(0.415) = 0.5 + 0.311 = \mathbf{0.811}$.

Weighted child entropy $= \frac{3}{7}(0) + \frac{4}{7}(0.811) \approx 0.463$. **Information gain** $= 0.985 - 0.463 =$ **0.52** bits.

Same split with Gini (deck): parent Gini $= 1 - (\frac{4}{7})^2 - (\frac{3}{7})^2 = 1 - \frac{16}{49} - \frac{9}{49} = \frac{24}{49} \approx 0.490$; children 0 and 0.375; weighted $\frac{4}{7}(0.375) = 0.214$ [W8 p17]. Gini decrease $= 0.490 - 0.214 = 0.276$. **Both criteria say this is a good split.**

Calc

A short Bayes probability (deck numbers)

WORKED EXAMPLE / MODEL ANSWER

(a) Conditional probability with the W2 dice [W2 p17]. Two dice. $P(\text{at least one 6}) = \dfrac{11}{36}$ and $P(\text{both 6}) = \dfrac{1}{36}$. Then $P(\text{both 6} \mid \text{at least one 6}) = \dfrac{P(\text{both} \cap \text{at least one})}{P(\text{at least one})} = \dfrac{1/36}{11/36} = \dfrac{1}{11}$. Bayes view: likelihood $P(\geq 1 \text{ six} \mid \text{both}) = 1$, prior $\frac{1}{36}$, marginal $\dfrac{11}{36}$, so the posterior is $\dfrac{1 \cdot \frac{1}{36}}{\frac{11}{36}} = \frac{1}{11}$.

(b) Naive Bayes posterior [W6 p12]. Unnormalized scores: VIP ≈ 0.0074, Staff ≈ 0.0100 (prior $\times$ likelihoods). Posterior $P(\text{Staff} \mid \text{"Tomorrow. Bacon!!!!"}) = \dfrac{0.0100}{0.0100 + 0.0074} = \dfrac{100}{174} \approx 0.57$. The denominator is the **marginal** $P(\text{message})$, the sum over the classes (law of total probability). Note the prior: $P(\text{VIP}) = \dfrac{200}{287} \approx 0.70$, yet Staff still wins because "Tomorrow" is much more typical of Staff ($\dfrac{31}{87}$ vs. $\dfrac{12}{200}$).

One perceptron update and one gradient-descent step

WORKED EXAMPLE / MODEL ANSWER

Perceptron (deck's beach neuron [W11 p3]; the update is illustrative). Weights $w_1 = 5$ (sunny), $w_2 = -0.2$ (miles), threshold 0, learning rate $\eta = 0.1$. Input: sunny ($x_1 = 1$), 6 miles ($x_2 = 6$). $z = 5(1) + (-0.2)(6) = 3.8 \geq 0$, so the prediction is **1** (go). Suppose the true class is **0**: error = actual − predicted $= 0 - 1 = -1$. Update (W11 p7 rule, weight + η·error·input): $w_1 = 5 + 0.1(-1)(1) = 4.9$ and $w_2 = -0.2 + 0.1(-1)(6) = -0.8$. New $z = 4.9 - 4.8 = 0.1$, still 1 but closer to 0. One more identical update: $w_1 = 4.8$, $w_2 = -1.4$, $z = 4.8 - 8.4 = -3.6 < 0$, so the prediction is **0**, now correct. *(The minus form gives the same numbers: $w \leftarrow w - \eta(\hat{y} - y)x$ with $\hat{y} - y = +1$.)*

Gradient descent on the deck's function [W4 p11–p15]. $f(x) = 3x^2 + 4x + 1$, $f'(x) = 6x + 4$, $\eta = 0.1$, $x_0 = 0$. $x_1 = 0 - 0.1 \cdot f'(0) = 0 - 0.1 \cdot 4 = -0.4$. $x_2 = -0.4 - 0.1 \cdot (6(-0.4) + 4) = -0.4 - 0.1 \cdot 1.6 = -0.56$. It approaches $x^* = -\frac{2}{3} \approx -0.667$, and the steps shrink as the slope flattens.

Backprop gradient via the chain rule [W11 p21]. $\hat{y} = w_0 + w_1 z_1 + w_2 z_2 + w_3 z_3$ and $L = (y - \hat{y})^2$. With $y = 10$, $\hat{y} = 8$, $z_1 = 2$: $\dfrac{\partial L}{\partial w_1} = -2(y - \hat{y})z_1 = -2(2)(2) = -8$. With $\nu = 0.1$: $w_1 \leftarrow w_1 - 0.1(-8) = w_1 + 0.8$. w_1 goes **up**, which raises $\hat{y}$ toward 10.

Calc

AdaBoost re-weighting (deck numbers)

WORKED EXAMPLE / MODEL ANSWER

5 instances with weights $\frac{1}{5}$ each; base model 1 misclassifies only instance 2, so $\epsilon_1 = \frac{1}{5} = 0.2$ [W9 p11]. Factor $\sqrt{\frac{1-\epsilon}{\epsilon}} = \sqrt{\frac{0.8}{0.2}} = \sqrt{4} = 2$. The misclassified instance's relative weight becomes 2 and the correct ones $\frac{1}{2}$. Sum $= 2 + 4 \cdot \frac{1}{2} = 4$. Normalize: $\frac{2}{4} = \frac{4}{8}$ for instance 2 and $\frac{1/2}{4} = \frac{1}{8}$ for the others.

Calc

Master Formula Sheet (all weeks, one place)

Format: formula · plain words · source. "Std" = not in deck (standard). "Rev" = review file.

W2 — Probability, statistics, linear algebra

Name	Formula	Plain words	Src
Arithmetic mean	$\bar{x} = \frac{1}{n}\sum x_i$	add, divide by n	W2 p8
Harmonic mean	$\bar{x} = n\big(\sum \frac{1}{x_i}\big)^{-1}$	average of rates	W2 p8
Geometric mean	$\bar{x} = \big(\prod x_i\big)^{1/n}$	average growth	W2 p8
Variance (pop / sample)	$\frac{1}{n}\sum(x_i-\bar{x})^2$ / $\frac{1}{n-1}\sum(x_i-\bar{x})^2$	average squared spread	W2 p8
Std. dev., precision	$\sigma = \sqrt{\sigma^2}$, precision $= 1/\sigma^2$	spread in the data's units	W2 p20
Skewness (pop)	$\gamma = \sqrt{n}\frac{\sum(x_i-\bar{x})^3}{(\sum(x_i-\bar{x})^2)^{3/2}}$	$+$ = right tail	W2 p8
Kurtosis (pop)	$\kappa = n\frac{\sum(x_i-\bar{x})^4}{(\sum(x_i-\bar{x})^2)^2}$	tail heaviness	W2 p8
Expectation	$E[X] = \sum x_i p(x_i)$; $\int x f(x)dx$	probability-weighted average	W2 p12
Variance of an RV	$V[X] = \sum(x_i-\mu)^2 p(x_i)$		W2 p20
Linearity	$E[X+Y] = E[X]+E[Y]$, $E[cX] = cE[X]$	always holds	W2 p13
Union	$P(A\cup B) = P(A) + P(B) - P(A\cap B)$	don't double count	W2 p16
Complement	$P(\bar{E}) = 1 - P(E)$		W2 p17
Conditional	$P(A \mid B) = \frac{P(A\cap B)}{P(B)}$	shrink the world to B	W2 p21
Total probability	$P(B) = \sum_i P(B \mid S_i)P(S_i)$	split into cases	W2 p22
Independence	$P(A\cap B) = P(A)P(B)$		W2 p23
Bayes	$P(A \mid B) = \frac{P(B\|A)P(A)}{P(B)}$	posterior = likelihood × prior ÷ marginal	W2 p25
Bernoulli	$P(y) = p^y(1-p)^{1-y}$; $E = p$, $Var = pq$	one 0/1 trial	W2 p26, Rev
Binomial	$b(k;n,p) = \binom{n}{k}p^k q^{n-k}$; $E = np$, $Var = npq$	# successes in n trials	W2 p27
Gaussian	$\frac{1}{\sqrt{2\pi\sigma^2}}\exp\big(-\frac{(x-\mu)^2}{2\sigma^2}\big)$	bell curve	W2 p28
z-score	$z = \frac{x-\mu}{\sigma}$	# std devs from the mean	W2 p30
Matrix product	$C_{ij} = \sum_k A_{ik}B_{kj}$	row · column	W2 p37
Transpose of a product	$(AB)^T = B^T A^T$	order flips	W2 p36
Inverse solve	$x = A^{-1}b$	needs square, full rank	W2 p41
L^p norm	$\|x\|_p = (\sum\|x_i\|^p)^{1/p}$; $L^\infty = \max\|x_i\|$	vector size	W2 p44
Orthogonal	$A^TA = I$, $A^{-1} = A^T$		W2 p45

Form

Name	Formula	Plain words	Src
Eigen	$Av = \lambda v$; $A = V\text{diag}(\lambda)V^{-1}$; symmetric: $Q\Lambda Q^T$	stretch along v	W2 p46
SVD, pseudoinverse	$A = UDV^T$; $A^\dagger = VD^\dagger U^T$	any shape	W2 p48–p50
Trace	$\text{Tr}(A) = \sum A_{ii}$; $\text{Tr}(ABC) = \text{Tr}(CAB)$		W2 p51

W4 — Regression

Name	Formula	Plain words	Src
Linear model	$\hat{y} = w_0 + w_1 x$; $\hat{y} = w^T x$ (with $x_0 = 1$)	line / plane	W4 p4, p23
SSE	$E(w) = \sum_i (w_0 + w_1 x_i - y_i)^2$	total squared miss	W4 p7
1-D closed form	$w_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{\text{Cov}(x,y)}{\text{Var}(x)}$, $w_0 = \bar{y} - w_1 \bar{x}$		W4 p21
Normal equation	$w = (X^T X)^{-1} X^T y$		W4 p27
Gradient	$\nabla f = [\partial f/\partial x_1, \dots]^T$	points uphill	W4 p14
GD update	$x_{i+1} = x_i - \eta \nabla f(x_i)$	step downhill (**minus**)	W4 p15
GD for linear regression	$J = \frac{1}{2N}\sum (w^T x_i - y_i)^2$; $\partial_{w_0} J = \frac{1}{N}\sum (w^T x_i - y_i)$; $\partial_{w_1} J = \frac{1}{N}\sum (w^T x_i - y_i) x_i$	avg. error ($\times$ input)	W4 p24
Ridge	$RSS + \alpha \sum_j w_j^2$	L2: shrink	W4 p43
Lasso	$RSS + \alpha \sum_j \lvert w_j \rvert$	L1: shrink + zeros	W4 p45
Elastic Net	$RSS + \alpha\lambda \lVert w \rVert_1 + \alpha(1-\lambda)\lVert w \rVert_2^2$	λ=0 Ridge, λ=1 Lasso	W4 p47
Logistic	$p = \frac{1}{1+e^{-(\alpha+\beta x)}}$; $\sigma(z) = \frac{1}{1+e^{-z}}$	squash to (0,1)	W4 p58, p60
Odds, logit	$\frac{p}{1-p} = e^{\alpha+\beta x}$; $\ln \frac{p}{1-p} = \alpha + \beta x$	log-odds are linear	W4 p58–p60
MLE (logistic)	$\arg\min \sum \log(1 + e^{-y_i(w^T x_i + b)})$		W4 p64
MAP (logistic)	$\arg\min \sum \log(1 + e^{-y_i w^T x_i}) + \lambda w^T w$	MLE + L2	W4 p63

W5 — Evaluation

Name	Formula	Plain words	Src
MSE / RMSE / MAE	$\frac{1}{n}\sum (y_i - \hat{y}_i)^2$ / $\sqrt{MSE}$ / $\frac{1}{n}\sum \lvert y_i - \hat{y}_i \rvert$	squared / root / absolute miss	W5 p13–p14
Absolute loss	$L_{abs} = \lvert y - \hat{p} \rvert$		W5 p16
Log loss	$-(y \log \hat{p} + (1-y)\log(1-\hat{p}))$	−log prob. of the truth	W5 p18

Form

Name	Formula	Plain words	Src
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	all correct / all	W5 p20
Precision	$\frac{TP}{TP+FP}$	÷ predicted +	W5 p20
Recall = TPR	$\frac{TP}{TP+FN}$	÷ actual +	W5 p20
F1	$\frac{2PR}{P+R}$	harmonic mean	Rev
FPR	$\frac{FP}{FP+TN}$	ROC x-axis	Std

W6 — kNN, naive Bayes, SVM

Name	Formula	Plain words	Src
Euclidean	$\sqrt{\sum_j (x_j - y_j)^2}$	straight line	W6 p6
Manhattan	$\sum_j \lvert x_j - y_j \rvert$	city blocks	W10 p44
Distance weight	$w_i = 1/d(x', x_i)^2$	closer = bigger vote	Rev
Naive Bayes	$\hat{c} = \arg\max_c P(c) \prod_j P(x_j \mid c)$	prior × word likelihoods	W6 p11–p12
Laplace smoothing	$\frac{\text{count}(x_j, C)+1}{\text{count}(C)+V}$	+1 pseudo-count	W6 p13, Rev
Hyperplane, margin	$w \cdot x + b = 0$; margin $= \frac{2}{\lVert w \rVert}$		W6 p16, Rev
Hinge loss	$\max(0, 1 - y_i(w \cdot x_i + b))$	0 if safe, linear otherwise	W6 p19, Rev

W7 — Unsupervised

Name	Formula	Plain words	Src
K-Means objective	$J = \sum_n \sum_k z_{nk} \lVert x_n - \mu_k \rVert^2$	WCSS	W7 p10
Assignment	$z_{nk} = 1$ for the nearest μ_k	nearest center	W7 p11
Update	$\mu_k = \frac{\sum_n z_{nk} x_n}{\sum_n z_{nk}}$	cluster mean	W7 p11
Complexity	$O(tkn)$		W7 p25
Factor model	$x_i - \mu_i = \sum_j l_{i,j} F_j + \epsilon_i$		W7 p38
Communality	$h_i^2 = \sum_j l_{i,j}^2$	common variance	W7 p39
% variability	eigenvalue / #features		W7 p41
Loadings	eigenvector × $\sqrt{\text{eigenvalue}}$		W7 p41

W8 — Trees

Name	Formula	Plain words	Src
Gini	$\sum p_i(1-p_i) = 1 - \sum p_i^2$	0 = pure	W8 p16
Overall Gini	$\sum_r \frac{n_r}{n} \text{Gini}_r$	weighted average	W8 p16–p17
Entropy	$H = -\sum p_i \log_2 p_i$	0 pure, 1 = 50/50	Std
Information gain	$H(\text{parent}) - \sum_c \frac{n_c}{n} H(c)$	impurity drop	Std
Regression-tree RSS	$\sum_R \sum_{i \in R} (y_i - \bar{y}_R)^2$		W8 p9

Form

W9 — Ensembles

Name	Formula	Plain words	Src
Initial weights	$w_i^{(1)} = \frac{1}{n}$		W9 p10
AdaBoost factor	misclassified $\times\sqrt{\frac{1-\epsilon}{\epsilon}}$; correct $\div$ it; then normalize	boost the mistakes	W9 p11
Vote / average	mode of the predictions / $\frac{1}{B}\sum \hat{y}^{(b)}$		W9 p6

W10 — Recommender systems

Name	Formula	Plain words	Src
Dot product	$\langle v, u\rangle = \sum v_i u_i = \lvert v\rvert\lvert u\rvert \cos\theta$	# matching features (binary)	W10 p6, p18
Cosine	$\frac{\langle v,u\rangle}{\lvert v\rvert\lvert u\rvert}$	angle only	W10 p19
TF	$\frac{f_{t,d}}{\sum_{t'} f_{t',d}}$; or $0.5 + 0.5\frac{f_{t,d}}{\max f}$	frequency in the doc	W10 p24–p25
IDF	$\log \frac{N}{\lvert\{d : t \in d\}\rvert}$	rarity across docs	W10 p26
TF-IDF	$\text{tf} \cdot \text{idf}$		W10 p23
Dice	$\frac{2\lvert X \cap Y\rvert}{\lvert X\rvert + \lvert Y\rvert}$		W10 p45
Jaccard	$\frac{\lvert A \cap B\rvert}{\lvert A \cup B\rvert}$; binary: $\frac{A \cdot B}{\lVert A\rVert^2 + \lVert B\rVert^2 - A \cdot B}$	overlap / union	W10 p46
Pearson	$\frac{\sum_{S_{xy}}(r_{xs}-\bar{r}_x)(r_{ys}-\bar{r}_y)}{\sqrt{\sum(r_{xs}-\bar{r}_x)^2}\sqrt{\sum(r_{ys}-\bar{r}_y)^2}}$	co-rated items only	W10 p61
Matrix factorization	$\langle user_i, item_j\rangle \approx M_{ij}$		W10 p9
NB-CF posterior	$\frac{P(r_{ui}=v)\prod_{j\neq i} P(r_{uj}\lvert r_{ui}=v)}{\sum_w P(r_{ui}=w)\prod_{j\neq i} P(r_{uj}\lvert r_{ui}=w)}$		W10 p57
RMSE / MAE	$\sqrt{\frac{1}{\lvert Q\rvert}\sum(r_{ui}-\hat{r}_{ui})^2}$ / $\frac{1}{\lvert Q\rvert}\sum\lvert r_{ui}-\hat{r}_{ui}\rvert$		W10 p64

W11 — Deep learning

Name	Formula	Plain words	Src
Weighted sum	$z = \sum_j w_j x_j + b$		W11 p7
Unit step	$u(z) = 1$ if $z \geq 0$, else 0		W11 p7
Perceptron rule	$w \leftarrow w + \eta(y - \hat{y})x \equiv w - \eta(\hat{y} - y)x$	fix only mistakes	W11 p7
Backprop / GD	$w_i \leftarrow w_i - \nu\frac{\partial L}{\partial w_i}$ (deck p11 prints $+$)	downhill	W11 p18
Momentum	$w_{i+1} = w_i - \nu\frac{\partial L}{\partial w_i} + \alpha\Delta w_i$	keep inertia	W11 p19
MSE loss	$L = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$		W11 p12
Sigmoid, derivative	$\frac{1}{1+e^{-z}}$; $\frac{e^{-z}}{(1+e^{-z})^2}$ (max 0.25)		W11 p20
tanh, derivative	$\frac{e^{2z}-1}{e^{2z}+1}$; $\frac{4e^{2z}}{(e^{2z}+1)^2}$		W11 p20

Form

Name	Formula	Plain words	Src
ReLU, derivative	$\max(0, z)$; 1 ($z > 0$), 0 ($z < 0$)		W11 p9, p20
Leaky ReLU	z or cz; derivative 1 or c		W11 p20
Chain rule	$h'(x) = f'(g(x))g'(x)$; $\frac{\partial L}{\partial w_1} = -2(y_i - \hat{y}_i)z_1$	multiply local slopes	W11 p21

W12 — NLP

Name	Formula	Plain words	Src
LDA update	$P(w \mid t) = P(t \mid d) \cdot P(w \mid t)$	doc likes topic × topic likes word	W12 p13
$P(t \mid d)$	share of the other words in d assigned to t		W12 p13
Bigram LM	$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$	Markov	Std
Regex (whole word “the”)	`[^a-zA-Z][tT]he[^a-zA-Z]`		W12 p8

Form

Learn More: IBM and DeepLearning.AI (outside the deck)

NOT IN DECK — STANDARD DEFINITION

How to use this appendix. The exam is graded against the **professor's decks**, so answer from the deck sections first. Use these outside sources only to *understand* a topic you find hard, **IBM first (shorter, easier), then DeepLearning.AI / Andrew Ng (deeper)**. Every link below was opened and checked on Nov 10, 2026: IBM pages by their final URL and page title, and Coursera courses by their public syllabus pages. **Course videos require a Coursera login, so their content was not checked here; only the course and week titles are listed.** The two DeepLearning.AI "AI Notes" are free articles.

The sources

Source	Level	What it is	Where
IBM Think topic pages	Easy	Free short articles, "What is . . .?"	`ibm.com/think/topics/<topic>`
IBM — Machine Learning with Python: Build Real-World ML Models	Easy–medium	Course 1 of the IBM AI Engineering Professional Certificate. Modules: Linear and Logistic Regression · Building Supervised Learning Models (trees, SVM, KNN, random forests, XGBoost, bias–variance) · Building Unsupervised Learning Models (K-Means, DBSCAN, PCA) · Evaluating and Validating ML Models (metrics, cross-validation, regularization, data leakage)	`coursera.org/learn/machine-learning-with-python`
IBM — Introduction to Deep Learning & Neural Networks with Keras	Easy–medium	Course 2 of the same certificate. Modules: Basics of Deep Learning (gradient descent, backpropagation, vanishing gradient, activation functions) · Deep Learning Models (CNN, RNN, transformers, autoencoders)	`coursera.org/learn/introduction-to-deep-learning-with-keras`

More

Source	Level	What it is	Where
DeepLearning.AI — Machine Learning Specialization (Andrew Ng, with Stanford Online)	Medium	C1 *Supervised ML: Regression and Classification* · C2 *Advanced Learning Algorithms* · C3 *Unsupervised Learning, Recommenders, Reinforcement Learning*	`coursera.org/learn/machine-learning`, `…/advanced-learning-algorithms`, `…/unsupervised-learning-recommenders-reinforcement-learning`
DeepLearning.AI — Deep Learning Specialization (Andrew Ng)	Medium–hard	*Neural Networks and Deep Learning* · *Improving Deep Neural Networks* · *Convolutional Neural Networks* · *Sequence Models*	`coursera.org/learn/neural-networks-deep-learning`, `…/deep-neural-network`, `…/convolutional-neural-networks`, `…/nlp-sequence-models`
DeepLearning.AI — NLP Specialization	Medium	*NLP with Classification and Vector Spaces* (naive Bayes, Laplacian smoothing, cosine similarity, KNN) · *NLP with Probabilistic Models* (n-gram language models, CBOW)	`coursera.org/learn/classification-vector-spaces-in-nlp`, `…/probabilistic-models-in-nlp`
DeepLearning.AI — Mathematics for ML and Data Science (Luis Serrano)	Easy–medium	*Linear Algebra* (eigenvalues/eigenvectors, inverses, rank) · *Calculus* (derivatives, gradients, gradient descent) · *Probability & Statistics* (distributions, MLE, MAP, hypothesis testing)	`coursera.org/specializations/mathematics-for-machine-learning-and-data-science`
DeepLearning.AI AI Notes (free articles)	Medium	*Initializing neural networks* (vanishing/exploding gradients) · *Parameter optimization in neural networks* (GD vs. SGD, mini-batch, momentum, RMSprop, Adam)	`deeplearning.ai/ai-notes/initialization/`, `deeplearning.ai/ai-notes/optimization/`

Topic map by week

*Columns: **Easier (IBM) → Deeper (DeepLearning.AI)**. IBM Think pages are written as their slug: `overfitting` means `ibm.com/think/topics/overfitting`. "C2 W4" = course 2, week 4.*

W2 — Math Review

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Probability, Bayes, distributions, MLE/MAP, p-value	(no IBM Think page found)	Math for ML: *Probability & Statistics for ML & DS*

More

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Vectors, matrices, inverse, rank, eigenvalues/eigenvectors	(no IBM Think page found)	Math for ML: *Linear Algebra for ML and Data Science*
Derivatives, gradients	—	Math for ML: *Calculus for ML and Data Science*

W4 — Regression

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Linear regression, cost function	`linear-regression`	MLS C1 W1 "Introduction to Machine Learning" (linear regression, cost function)
Gradient descent, learning rate	`gradient-descent`	MLS C1 W1 (gradient descent and learning rate); C1 W2 (choosing the learning rate, checking convergence)
Multiple features, feature scaling, polynomial regression	IBM ML with Python, "Linear and Logistic Regression"	MLS C1 W2 "Regression with multiple input variables"
Overfitting / underfitting	`overfitting`, `underfitting`	MLS C1 W3 "Classification" (overfitting and regularization)
Regularization, Ridge, Lasso	`regularization`, `ridge-regression`, `lasso-regression`	MLS C1 W3 (regularization)
Logistic regression, sigmoid, decision boundary	`logistic-regression`	MLS C1 W3 (logistic regression, decision boundary, cost function)

W5 — Model Evaluation

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Bias–variance tradeoff	`bias-variance-tradeoff`	MLS C2 W3 "Advice for applying machine learning" (bias and variance, learning curves); DLS *Improving Deep NNs* W1
Train / dev (validation) / test split	IBM ML with Python, "Evaluating and Validating ML Models"	DLS *Improving Deep NNs* W1 "Practical Aspects of Deep Learning"
Confusion matrix, precision/recall, skewed classes	`confusion-matrix`	MLS C2 W3 (metrics for skewed datasets)
Loss functions (MSE, log loss)	`loss-function`	MLS C1 W3 (cost function for logistic regression)
Cross-validation, data leakage	IBM ML with Python, "Evaluating and Validating ML Models"	—

W6 — kNN, Naive Bayes, SVM

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
k-nearest neighbors	`knn`	NLP C1 W4 "Machine Translation and Document Search" (K-nearest neighbors)

More

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Naive Bayes, Laplace smoothing	`naive-bayes`	NLP C1 W2 “Sentiment Analysis with Naïve Bayes” (Bayes’ rule, **Laplacian smoothing**, log likelihood)
SVM, margin, kernel trick	`support-vector-machine`	IBM ML with Python, “Building Supervised Learning Models”
Supervised learning overview	`supervised-learning`	MLS C1 W1

W7 — Unsupervised Learning

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
K-Means, elbow method (IBM’s page explains the elbow with WCSS)	`k-means-clustering`	MLS C3 W1 “Unsupervised learning” (K-means objective, initialization, choosing the number of clusters)
Hierarchical clustering, dendrograms	`hierarchical-clustering`	—
DBSCAN	IBM ML with Python, “Building Unsupervised Learning Models”	—
PCA, dimensionality reduction	`principal-component-analysis`, `dimensionality-reduction`	MLS C3 W2 (optional: PCA); NLP C1 W3 (visualization with PCA)
Clustering / unsupervised overview	`clustering`, `unsupervised-learning`	MLS C3 W1

W8 — Decision Trees

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Decision trees, entropy, information gain, regression trees	`decision-trees`	MLS C2 W4 “Decision trees” (information gain, regression trees)
Random forests	`random-forest`	MLS C2 W4 (random forests)

W9 — Ensemble Learning

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Ensembles overview	`ensemble-learning`	MLS C2 W4 (random forests, XGBoost)
Bagging, bootstrap	`bagging`	—
Boosting, XGBoost	`boosting`, `xgboost`	MLS C2 W4 (XGBoost)

W10 — Recommender Systems

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Recommender overview	`recommendation-engine`	MLS C3 W2 “Recommender systems”
Collaborative filtering, mean normalization	`collaborative-filtering`	MLS C3 W2 (collaborative filtering, mean normalization)

More

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Content-based filtering	`content-based-filtering`	MLS C3 W2 (content-based filtering, finding related items)
Cosine similarity	`cosine-similarity`	NLP C1 W3 "Vector Space Models" (Euclidean distance and cosine similarity)
Bag of words, word embeddings, Word2Vec, GloVe	`bag-of-words`, `word-embeddings`	DLS *Sequence Models* W2 "NLP & Word Embeddings" (embedding matrix, Word2Vec, GloVe)

W11 — Deep Learning

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Neural networks, forward propagation	`neural-networks`	DLS *Neural Networks and Deep Learning* W2–W4
Backpropagation, gradient descent, activation functions, vanishing gradient	`backpropagation`; IBM *Intro to DL & NN with Keras*, "Basics of Deep Learning"	DLS *Neural Networks and Deep Learning* W3 (forward/backpropagation, activation functions)
Vanishing / exploding gradients, initialization, dropout	IBM *Intro to DL & NN with Keras*, "Basics of Deep Learning"	**AI Notes: *Initializing neural networks* (free)**; DLS *Improving Deep NNs* W1 (dropout, vanishing/exploding gradients)
Mini-batch vs. stochastic GD, momentum, Adam	`stochastic-gradient-descent`	**AI Notes: *Parameter optimization in neural networks* (free)**; DLS *Improving Deep NNs* W2 "Optimization Algorithms"
Batch normalization	—	DLS *Improving Deep NNs* W3
CNN	`convolutional-neural-networks`	DLS *Convolutional Neural Networks* W1 (convolution, padding, stride, pooling)
RNN, LSTM	`recurrent-neural-networks`, `lstm`	DLS *Sequence Models* W1 (vanishing gradients, GRU, LSTM)
Transformers, attention	`transformer-model`, `attention-mechanism`	DLS *Sequence Models* W4 "Transformer Network"
Autoencoders, GANs	`autoencoder`, `generative-adversarial-networks`	IBM *Intro to DL & NN with Keras*, "Deep Learning Models" (autoencoders)

W12 — Intro to NLP

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
NLP overview	`natural-language-processing`	—
Tokenization	`tokenization`	NLP C1 W1 (preprocessing)
Stemming vs. lemmatization	`stemming-lemmatization`	NLP C1 W1 (preprocessing)

More

Topic	Easier (IBM)	Deeper (DeepLearning.AI)
Topic modeling, **LDA**	`topic-modeling`, `latent-dirichlet-allocation`	— (no DeepLearning.AI course found that lists LDA; use the deck + IBM)
N-gram language models, Markov assumption	—	NLP C2 *Probabilistic Models* M3 "Autocomplete and Language Models" (n-gram LMs, smoothing)
Word embeddings, CBOW, skip-gram	`word-embeddings`	NLP C2 M4 (CBOW); DLS *Sequence Models* W2 (Word2Vec, GloVe, debiasing)
RNN, LSTM, Transformer, attention	`lstm`, `transformer-model`, `attention-mechanism`	DLS *Sequence Models* W1, W4
Large language models	`large-language-models`	—

More